

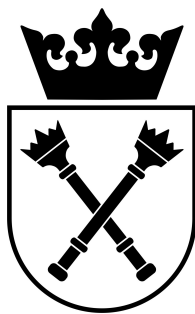
UNIwersytet Jagielloński

PYTANIA DO EGZAMINU LICENCJACKIEGO

NA KIERUNKU INFORMATYKA

BRYK

Karol CHRZĄSTEK
Małgorzata DYMEK
Michał PIOTROWSKI
Dominik WOŁEK
Dagmara ZDYBAŁ



Rok akademicki 2019/2020

Spis treści

Matematyczne podstawy informatyki

1 Zasada indukcji matematycznej.

- Dla $T(n)$ będącego funkcją/formą zdaniową jeżeli (1) zachodzi $T(0)$, (2) $\forall n \in \mathbb{N} \ T(n) \Rightarrow T(n+1)$ to $T(n)$ jest prawdziwa dla każdego $n \in \mathbb{N}$.
- W dowodzie udowadniamy że $M = \{n \in \mathbb{N} : T(n) \text{ zachodzi}\}$ jest równe $\mathbb{N}$ przy pomocy **Aksjomatu 5 liczb naturalnych (Peano)** – zbiór liczb naturalnych taki, że (1) 1 jest elementem tego zbioru, (2) dla każdej liczby ze zbioru należy do niego również jej następnik, zawiera wszystkie liczby naturalne.

2 Porządki częściowe i liniowe. Elementy największe, najmniejsze, maksymalne i minimalne.

- Relacja **częściowego porządku** ($\leq, \preceq, \prec$) to relacja **zwrotna** ($\forall x \in X \ xRx$), **przechodnia** ($\forall x, y, z \in X \ xRy \wedge yRz \Rightarrow xRz$) i **antysymetryczna** ($\forall x, y \in X \ xRy \wedge yRx \Rightarrow x = y$).
 - **Elementy porównywalne** to takie, które są w relacji ze sobą.
 - **Element maksymalny**, jeśli $\forall a \in A \ m \preceq a \Rightarrow a = m$.
 - **Element minimalny**, jeśli $\forall a \in A \ a \preceq m \Rightarrow a = m$.
 - **Element największy**, jeśli $\forall a \in A \ a \preceq m$.
 - **Element najmniejszy**, jeśli $\forall a \in A \ m \preceq a$.
- **Diagram Hassego** - graf skierowany przedstawiający częściowy porządek w zbiorze, w odpowiedni sposób przedstawiony graficznie.
- Relacja **liniowego porządku** to relacja **zwrotna**, **przechodnia**, **antysymetryczna** i **spójna** ($\forall x, y \in X \ xRy \vee yRx \vee x = y$).

3 Relacja równoważności i zbiór ilorazowy.

- Relacja równoważności jest **zwrotna**, **symetryczna** i **przechodnia**.

- **Klasa abstrakcji** elementu x (względem relacji R) to: $[x]_R = \{y \in X : xRy\}$. Element x to reprezentant klasy abstrakcji $[x]_R$.
- **Zbiór ilorazowy** zbioru X przez relację R to zbiór wszystkich klas abstrakcji.

4 Metody dowodzenia twierdzeń: wprost, nie wprost, przez kontrapozycję.

Dla prawdziwości zdania:

$$p \Rightarrow q$$

Dowód wprost: $p \Rightarrow q$.

Dowód przez kontrapozycję: $(p \Rightarrow q) \Leftrightarrow (\neg q \Rightarrow \neg p)$ (prawo kontrapozycji).

Dowód nie wprost: $p \Leftrightarrow (\neg p \Rightarrow (q \wedge \neg q))$

5 Metody numeryczne rozwiązywania równań nieliniowych: bisekcji, siecznych, Newtona.

5.1 Metoda połowienia (bisekcji)

Założenia: f jest funkcją ciągłą w przedziale $[a, b]$, $f(a)f(b) < 0$.

Z własności Darboux funkcji ciągłych, funkcja f ma miejsce zerowe w przedziale $[a, b]$.

Definicja 5.1 *Algorytm bisekcji polega na obliczeniu $f(c_k)$, gdzie $c_k = \frac{a_k + b_k}{2}$ i zastąpieniu przez c_k tej z liczb a_k, b_k dla której funkcja f ma taki sam znak.*

$$(a_{k+1}, b_{k+1}) = (c_k, b_k) \text{ jeżeli } f(a_k)f(c_k) > 0$$

$$(a_{k+1}, b_{k+1}) = (a_k, c_k) \text{ jeżeli } f(b_k)f(c_k) > 0$$

Jeżeli $f(c_k) = 0$ to kończymy obliczenia.

- Metoda bisekcji jest **niezawodna**, ale **wolno zbieżna**.
- Kryteria zakończenia obliczeń: osiągnięta **dokładność**, **bliskość** wartości funkcji do 0, ilość **iteracji**.

- **dokładność** przybliżenia w k-tej iteracji: $|x_k - \alpha| \leq \frac{|b-a|}{2^k}$.

5.2 Metoda siecznych

Założenia: f jest funkcją ciągłą w przedziale $[a, b]$, $f(a)f(b) < 0$.

$$x_{k+1} = x_k - \frac{f(x_k)(x_k - x_{k-1})}{f(x_k) - f(x_{k-1})}$$

gdzie $x_0 = a$, $x_1 = b$.

- W metodzie siecznych rezygnujemy z założenia, że funkcja na końcach przedziału ma różne znaki.
- **Może się zdarzyć, że metoda wyprodukuje ciąg rozbieżny.**
- **Szybsza zbieżność** ciągu iteracji, jeśli zaczynamy od dobrego przybliżenia.

5.3 Metoda Newtona (stycznych)

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Zmodyfikowana metoda Newtona:

$$x_{n+1} = x_n - \frac{f'(x_n) \pm \sqrt{(f'(x_n))^2 - 2f(x_n)f''(x_n)}}{f''(x_n)}$$

- Metoda Newtona w wielu sytuacjach daje **bardzo szybką zbieżność**.
- Podstawową wadą tej metody jest konieczność obliczenia pochodnej funkcji.
- Zmodyfikowana metoda Newtona daje bardzo szybką zbieżność, jeśli x_n jest już dobrym przybliżeniem pierwiastka.
- Koszt wyznaczenia kolejnego przybliżenia w zmodyfikowanej metodzie może przerastać korzyści płynące z szybszej zbieżności.

Wielowymiarowa metoda Newtona.

Jeśli $f = (f_1, f_2, \dots, f_n) : \mathbb{R}^n \rightarrow \mathbb{R}^n$ jest różniczkowalną funkcją wielu zmiennych możemy przybliżyć ją lokalnie:

$$f(x) \approx f(x_0) + Df(x_0)(x - x_0)$$

gdzie

$$Df(x_0) = \begin{bmatrix} \frac{\delta f_1}{\delta x_1}(x_0) & \frac{\delta f_1}{\delta x_2}(x_0) & \dots & \frac{\delta f_1}{\delta x_n}(x_0) \\ \frac{\delta f_2}{\delta x_1}(x_0) & \frac{\delta f_2}{\delta x_2}(x_0) & \dots & \frac{\delta f_2}{\delta x_n}(x_0) \\ \dots & \dots & \dots & \dots \\ \frac{\delta f_n}{\delta x_1}(x_0) & \frac{\delta f_n}{\delta x_2}(x_0) & \dots & \frac{\delta f_n}{\delta x_n}(x_0) \end{bmatrix}$$

Rozwiązując równanie $f(x_0) + Df(x_0)(x - x_0) = 0$ otrzymujemy:

$$x^{(i+1)} = x^{(i)} - [Df(x^{(i)})]^{-1} f(x^{(i)})$$

$$[Df(x^{(i)})](x^{(i+1)} - x^{(i)}) = -f(x^{(i)})$$

6 Rozwiązywanie układów równań liniowych: metoda eliminacji Gaussa, metody iteracyjne Jacobiiego i Gaussa-Seidla.

6.1 Metoda eliminacji Gaussa

Obliczając rząd macierzy metodą Gaussa należy za pomocą operacji elementarnych na wierszach sprowadzić macierz do macierzy schodkowej. Wtedy wszystkie niezerowe wiersze są liniowo niezależne.

Metody iteracyjne

Ogólna postać metody iteracyjnej:

$$Ax = b$$

$$Qx^{n+1} = (Q - A)x^n + b = \tilde{b}$$

$$\begin{bmatrix} 5 & -2 & 3 \\ 2 & 4 & 2 \\ 2 & -1 & -4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 10 \\ 0 \\ 0 \end{bmatrix}$$

6.2 Metoda iteracyjna Jacobiego

6.2.1 Algebraicznie

$$\begin{cases} x_1^{N+1} = \frac{1}{5}(10 + 2x_2^N - 3x_3^N) \\ x_2^{N+1} = \frac{1}{4}(-2x_1^N - 2x_3^N) \\ x_3^{N+1} = -\frac{1}{4}(x_2^N - 2x_1^N) \end{cases}$$

6.2.2 Macierzowo

$$Q = D \quad (\text{diagonalna})$$

6.3 Metoda iteracyjna Gaussa-Seidla

6.3.1 Algebraicznie

$$\begin{cases} x_1^{N+1} = \frac{1}{5}(10 + 2x_2^N - 3x_3^N) \\ x_2^{N+1} = \frac{1}{4}(-2x_1^{N+1} - 2x_3^N) \\ x_3^{N+1} = -\frac{1}{4}(x_2^{N+1} - 2x_1^{N+1}) \end{cases}$$

6.3.2 Macierzowo

$$Q = L + D \quad (\text{diagonalna i dolnotrójkątna})$$

7 Wartości i wektory własne macierzy: numeryczne algorytmy ich wyznaczania.

Definicja 7.1 Jeżeli $Ax = \lambda x$ dla $x \neq 0$ to λ jest **wartością własną** A , a x - **wektorem własnym** A odpowiadającym λ .

Definicja 7.2 *Metoda potęgowa* służy do wyznaczania **największej wartości własnej** macierzy. Korzysta z założenia: $x^{N+1} = A^{N+1}x^0$.

1. W i -tej iteracji: $x_i = Ay_{i-1}$, $y_i = \frac{x_i}{\|x_i\|_2}$
2. i -te przybliżenie największej wartości własnej: $\lambda_i = \frac{\langle x_i, y_{i-1} \rangle}{\langle y_{i-1}, y_{i-1} \rangle}$

Definicja 7.3 *Odwrotna metoda potęgowa* służy do wyznaczenia **najmniejszej wartości własnej** macierzy.

1. $Ax_1 = x_0$, $Ax_i = x_{i-1}$
2. i -te przybliżenie najmniejszej wartości własnej: $\frac{1}{\lambda_i} = \frac{\langle x_i, x_{i-1} \rangle}{\langle x_{i-1}, x_{i-1} \rangle}$

Definicja 7.4 *Metoda QR* pozwala na przybliżone wyznaczenie **wszystkich wartości własnych** macierzy. Dla:

$$A_i = Q_i R_i, \quad A_{i+1} = R_i Q_i$$

A_i dąży do macierzy przekątniowej z wartościami własnymi na przekątnej. Q , R to macierze z **rozkładu QR**:

$$A = QR, \quad Q - \text{ortonormalna } (QQ^T = I), \quad R - \text{górnortrójkątna}$$

$$A = [a_1 \dots a_n], \quad Q = [q_1 \dots q_n], \quad q_1 = \frac{a_1}{\|a_1\|_2}, \quad q_i = \frac{\tilde{q}_i}{\|\tilde{q}_i\|_2}$$

$$\tilde{q}_i = a_i - \sum_{j=1}^{i-1} r_{ji} q_j, \quad r_{ji} = \frac{\langle a_i, q_j \rangle}{\langle q_j, q_j \rangle}, \quad r_{jj} = \|\tilde{q}_j\|_2$$

8 Interpolacja wielomianowa: metody Lagrange’a i Hermite’a. Efekt Rungego.

8.1 Interpolacja wielomianowa

Interpolacja wielomianowa jest numeryczną **metodą przybliżania funkcji**. Wielomian p o możliwie najniższym stopniu interpoluje wartości y_k w węzłach x_k gdy przyjmuje pożądane wartości w odpowiednich punktach. Jeśli są to wartości pewnej funkcji f to mówimy też że p interpoluje f .

8.2 Wzór interpolacyjny Lagrange’a

Wyrażamy wielomian p jako sumę

$$p(x) = \sum_{k=0}^n y_k l_k(x), \quad p(x) = \sum_{i=0}^n y_i \prod_{j=0 \wedge j \neq i}^n \frac{x - x_j}{x_i - x_j}.$$

w której l_k są wielomianami zależnymi od węzłów $x_0, x_1, \dots, x_n$ ale nie od wartości $y_0, y_1, \dots, y_n$. Gdyby jedna z nich, mianowicie y_i , była równa 1, a pozostałe by zniknęły, to mielibyśmy równość

$$\delta_{ij} = p_n(x_j) = \sum_{k=0}^n y_k l_k(x_j) = \sum_{k=0}^n \delta_k l_k(x_j) = l_i(x_j) \quad (0 \leq j \leq n)$$

(δ_{ij} jest deltą Kroneckera). Łatwo znaleźć wielomian l_i o tej własności. Jego zerami są wszystkie węzły oprócz x_i , czyli dla pewnej stałej c jest

$$l_i(x) = c(x - x_0) \dots (x - x_{i-1})(x - x_{i+1}) \dots (x - x_n)$$

a c wynika z warunku $l_i(x_i) = 1$.

8.3 Interpolacja Hermite’a

Interpolacja umożliwiająca znalezienie **wielomianu przybliżającego według wartości funkcji** na n zadanych węzłach oraz na wartościach jej pochodnych. Węzeł zadany bez pochodnej jest węzłem pojedynczym, a węzeł z podanymi pochodnymi k -tego stopnia jest węzłem $k + 1$ -krotnym.

8.3.1 Algorytm

Budujemy tabelę z użyciem **różnic dzielonych** z tym wyjątkiem, że przy węzłach k -krotnych, $k > 1$, gdzie, de facto, nie można obliczyć różnicy dzielonej, podstawia się $\frac{f^{(k)}(x_i)}{k!}$. (W tabeli przedstawiony jest 3-krotny węzeł x_1).

Różnica dzielona jest zdefiniowana rekurencyjnie:

$$f[x_i] = f(x_i), \quad f[x_i, \dots, x_{i+j+1}] = \frac{f[x_{i+1}, \dots, x_{i+j+1}] - f[x_i, \dots, x_{i+j}]}{x_{i+j+1} - x_i}$$

x_i	$f(x_i)$	$f[x_{i-1}]$	$f[x_{i-2}, x_{i-1}, x_i]$	$f[x_{i-3}, x_{i-2}, x_{i-1}, x_i]$	$\dots$	$f[x_{i-n}, \dots, x_i]$
x_0	$f(x_0)$					
x_1	$f(x_1)$	$f[x_0, x_1]$				
x_1	$f(x_1)$	$f'(x_1)$	$f[x_0, x_1, x_1]$			
x_1	$f(x_1)$	$f'(x_1)$	$\frac{f''(x_1)}{2!}$	$f[x_0, x_1, x_1, x_1]$		
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	
x_n	$f(x_n)$	$f[x_{n-1}, x_n]$	$f[x_{n-2}, x_{n-1}, x_n]$	$f[x_{n-3}, x_{n-2}, x_{n-1}, x_n]$	$\dots$	$f[x_0, \dots, x_n]$

Definiując a_i jako wartości na przekątnej, $i = 1, 2, \dots, m$ oraz $\bar{x}_i = \{x_j : j < i\}$.

$$w(x) = \sum_{i=0}^m a_i \prod_{j=0}^{i-1} (x - \bar{x}_j),$$

8.4 Efekt Rungego

Efekt Rungego - pogorszenie jakości interpolacji wielomianowej, mimo **zwiększenia liczby węzłów**.

- Pogorszenie szczególnie widoczne na końcach przedziałów.
- Typowe dla interpolacji za pomocą wielomianów wysokich stopni przy stałych odległościach węzłów oraz przy interpolacji funkcji nieciągłych albo odbiegających znacząco od funkcji gładkiej.
- Dla uniknięcia efektu, stosuje się interpolację z węzłami coraz gęściej upakowanymi na krańcach przedziału interpolacji. Np. węzłami interpolacji n -punktowej wielomianowej powinny być miejsca zerowe wielomianu Czebyszewa n -tego stopnia.

9 Zmienne losowe dyskretne. Definicje i najważniejsze rozkłady.

Definicja 9.1 *Zmienne dyskretne.*

$$P(x) = P(X = x)$$

Obliczanie prawdopodobieństwa: $P(X \in A) = \sum_{x \in A} P(x)$.

Skumulowana funkcja rozkładu: $F(x) = P(X \leq x) = \sum_{y \leq x} P(y)$.

Całkowite prawdopodobieństwo: $\sum_x P(x) = 1$.

Wartość oczekiwana (średnia): $EX = \sum_x xP(x)$.

Wariancja (kwadrat odchylenia stand.): $VarX = \sigma^2 = E[(X - \mu)^2]$.

Rozkład	P(x)	EX	VarX	
Bernoulli(p)		p	pq	próba
	$P(x) = \begin{cases} p, & \text{for } x = 1 \\ q = (1 - p), & \text{for } x = 0 \end{cases}$			
Binomial(n, p)	$P(x) = \binom{n}{x} p^x (1 - p)^{n-x}$ for $x = 0, 1, \dots$	np	npq	liczba sukcesów z n prób
Geometric(p)	$P(x) = (1 - p)^{x-1} p$ for $x = 1, 2, \dots$ $P(X > k) = (1 - p)^k$	$\frac{1}{p}$	$\frac{1-p}{p^2}$	liczba prób do sukcesu
Poiss(λ)	$P(x) = e^{-\lambda} \frac{\lambda^x}{x!}$ for $x = 0, 1, \dots$	λ	λ	rozkład zdarzeń rzadkich

$$X_i \sim \text{Bernoulli}(p), S_n = \sum_{i=1}^n X_i, 0.05 \leq p \leq 0.95$$

10 Zmienne losowe ciągłe. Definicje i najważniejsze rozkłady.

Definicja 10.1 Zmienne ciągłe.

$$f(x) = F'(x)$$

Obliczanie prawdopodobieństwa: $P(X \in A) = \int_A f(x)dx$.

Skumulowana funkcja rozkładu: $F(x) = P(X \leq x) = \int_{-\infty}^x f(y)dy$.

Całkowite prawdopodobieństwo: $\int_{-\infty}^{\infty} f(x)dx = 1$.

Wartość oczekiwana: $EX = \int xf(x)dx$.

Wariancja: $VarX = \int_{-\infty}^{\infty} (x - \mu)^2 f(x)dx$.

Rozkład	f(x), F(x)	EX	VarX	
Unif(a,b)	$f(x) = \frac{1}{b-a}$ for $a \leq x \leq b$ $F(x) = \begin{cases} 0, & \text{for } x < a \\ \frac{x-a}{b-a}, & \text{for } a \leq x < b \\ 1, & \text{for } x \geq b \end{cases}$	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$	
Exp(λ)	$f(x) = \lambda e^{-\lambda x}$ for $x \geq 0$ $F(x) = 1 - e^{-\lambda x}$	$\frac{1}{\lambda}$	$\frac{1}{\lambda^2}$	modelowanie czasu, brak pamięci
Gamma(α, λ)	$f(x) = \frac{\lambda^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\lambda x}$ $F(x) = \frac{\lambda^\alpha}{\Gamma(\alpha)} \int_0^x t^{\alpha-1} e^{-\lambda t} dt$	$\frac{\alpha}{\lambda}$	$\frac{\alpha}{\lambda^2}$	łączny czas α niezależnych zdarzeń $\sim Exp(\lambda)$
N(μ, σ)	$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$ $F(x) = \Phi(x)$ dla N(0,1)	μ	σ^2	

$$T \sim Gamma(\alpha, \lambda), X \sim Poiss(\lambda t), \quad P(T \leq t) = P(X \geq \alpha)$$

$$Binomial(n, p) = N(np, \sqrt{np(1-p)})$$

11 Łańcuchy Markowa. Rozkład stacjonarny.

$$P = \begin{bmatrix} p_{11} & p_{12} & \dots & p_{1n} \\ p_{21} & p_{22} & \dots & p_{2n} \\ \dots & \dots & \dots & \dots \\ p_{n1} & p_{n2} & \dots & p_{nn} \end{bmatrix}$$

Rozkład w czasie h : $P_h = P_0 * P^h$

Rozkład stacjonarny: $\pi P = \pi, \sum \pi_i = 1$

12 Testy statystyczne: test z, test t-Studenta, test chi-kwadrat.

- **Z-testów** używamy do sprawdzenia czy dana próba pasuje do zadanej populacji lub do porównywania dwóch dużych prób ($n > 30$).
- **T-testów** używamy do porównywania dwóch małych prób ($n < 30$) testowych ze sobą, mogą również służyć do porównywania próby do zadanej wartości (np. średniej). Próby mogą być zależne lub niezależne.
- **Chi-kwadrat** używamy do ustalania **goodness of fit** dla próbki względem populacji lub do zbadania niezależności.

T-testy

Hipoteza zerowa	Warunki	Statystyka	Stopnie swobody
$\mu = \mu_0$	Rozmiar próby n ; nieznana σ	$t = \frac{\bar{X} - \mu_0}{\frac{s}{\sqrt{n}}}$	$n - 1$
$\mu_X - \mu_Y = D$	Rozmiary prób n, m nie- znane, równe $\sigma_X = \sigma_Y$	$t = \frac{\bar{X} - \bar{Y} - D}{s_p \sqrt{\frac{1}{n} + \frac{1}{m}}}$	$n + m - 2$
$\mu_X - \mu_Y = D$	Rozmiary prób n, m ; nieznane, różne $\sigma_X \neq \sigma_Y$	$t = \frac{\bar{X} - \bar{Y} - D}{\sqrt{\frac{s_X^2}{n} + \frac{s_Y^2}{m}}}$	aproxymacja Satterthwaite

Z-testy

Hipoteza ze- rowa	Parametr, estymator	jeśli H_0 jest prawdziwa:		Statystyka
H_0	$\theta, \hat{\theta}$	$E(\hat{\theta})$	$Var(\hat{\theta})$	$Z = \frac{\hat{\theta} - \theta_0}{\sqrt{Var(\hat{\theta})}}$
$\mu = \mu_0$	$\mu, \bar{X}$	μ_0	$\frac{\sigma^2}{n}$	$Z = \frac{\bar{X} - \mu_0}{\frac{\sigma}{\sqrt{n}}}$
$p = p_0$	$p, \hat{p}$	p_0	$\frac{p_0(1-p_0)}{n}$	$Z = \frac{\hat{p} - p_0}{\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}}$
$\mu_X - \mu_Y = D$	$\mu_X - \mu_Y,$ $\bar{X} - \bar{Y}$	D	$\frac{\sigma_X^2}{n} + \frac{\sigma_Y^2}{n}$	$Z = \frac{\bar{X} - \bar{Y} - D}{\sqrt{\frac{\sigma_X^2}{n} + \frac{\sigma_Y^2}{n}}}$
$p_1 - p_2 = D$	$p_1 - p_2,$ $\hat{p}_1 - \hat{p}_2$	D	$\frac{p_1(1-p_1)}{n} + \frac{p_2(1-p_2)}{m}$	$Z = \frac{\hat{p}_1 - \hat{p}_2 - D}{\sqrt{\frac{\hat{p}_1(1-\hat{p}_1)}{n} + \frac{\hat{p}_2(1-\hat{p}_2)}{m}}}$
$p_1 = p_2$	$p_1 - p_2,$ $\hat{p}_1 - \hat{p}_2$	0	$p(1-p) \left(\frac{1}{n} + \frac{1}{m} \right)$ gdzie $p = p_1 = p_2$	$Z = \frac{\hat{p}_1 - \hat{p}_2}{\sqrt{\hat{p}(1-\hat{p}) \left(\frac{1}{n} + \frac{1}{m} \right)}}$ gdzie $\hat{p} = \frac{n\hat{p}_1 + m\hat{p}_2}{n+m}$

Rozkład obserwacji o rozkładzie normalnym i wspólnej wariancji σ^2

$$\frac{(n-1)s^2}{\sigma^2} = \sum_{i=1}^n \left(\frac{X_i - \bar{X}}{\sigma} \right)^2 \sim Chi-square(n-1) \sim Gamma\left(\frac{n-1}{2}, \frac{1}{2}\right)$$

Zatem przedział ufności:

$$\left[\frac{(n-1)s^2}{\chi_{\frac{\alpha}{2}}^2}, \frac{(n-1)s^2}{\chi_{1-\frac{\alpha}{2}}^2} \right]$$

Testy Chi kwadrat

H_0	H_A	Test statistic	Rejection region	P-value
$\sigma^2 = \sigma_0^2$	$\sigma^2 > \sigma_0^2$ $\sigma^2 < \sigma_0^2$ $\sigma^2 \neq \sigma_0^2$	$\frac{(n-1)s^2}{\sigma_0^2}$	$\chi_{obs}^2 > \chi_{\alpha}^2$ $\chi_{obs}^2 < \chi_{\alpha}^2$ $\chi_{obs}^2 \geq \chi_{\frac{\alpha}{2}}^2$ or $\chi_{obs}^2 \leq \chi_{\frac{\alpha}{2}}^2$	$P\chi^2 \geq \chi_{obs}^2$ $P\chi^2 \leq \chi_{obs}^2$ $2 \min(P\chi^2 \geq \chi_{obs}^2, P\chi^2 \leq \chi_{obs}^2)$

Statystyka Chi-kwadrat

$$\chi^2 = \sum_{k=1}^N \frac{(Obs(k) - Exp(k))^2}{Exp(k)}, R = [\chi_\alpha^2, +\infty], P = P\chi^2 \geq \chi_{obs}^2$$

Rule of thumb: $Exp(k) \geq 5$ for all $k = 1, \dots, N$.

Test Chi-kwadrat niezależności A i B

$$\chi_{obs}^2 = \sum_{i=1}^k \sum_{j=1}^m \frac{(Obs(i, j) - \hat{Exp}(i, j))^2}{\hat{Exp}(i, j)}, \hat{Exp}(i, j) = \frac{(n_{i.})(n_{.j})}{n}$$

13 Wzór Bayesa i jego interpretacja.

$$P(E|F) = \frac{P(E \cap F)}{P(F)} = \frac{P(F|E)P(E)}{P(F)}$$

$$P(E_1 \cap \dots \cap E_n) = \prod_{i=1}^n P(X_i | X_1, \dots, X_{i-1})$$

$$P(E) = \sum_{i=1}^n P(E|F_i)P(F_i) \text{ dla } \bigcap_{i=1}^n F_i = \Omega$$

14 Istnienie elementów odwrotnych względem mnożenia w strukturze $(Z_m, +, *)$ w zależności od liczby naturalnej m . Rozszerzony algorytm Euklidesa.

Twierdzenie 14.1 *Tożsamość Bézouta*

Dla niezerowych liczb całkowitych a oraz b o największym wspólnym dzielniku d , istnieją liczby całkowite x oraz y , nazywane współczynnikami Bézouta, które spełniają równanie

$$ax + by = d$$

Definicja 14.2 *Elementy odwrotne modulo*

W strukturze $(Z_m, +, *)$ element odwrotny względem mnożenia dla x to taki x^{-1} , że:

$$x \cdot x^{-1} = 1 \bmod m$$

Element odwrotny istnieje, jeśli $NWD(x, m) = 1$

Aby obliczyć element odwrotny do x w Z_m należy użyć rozszerzonego algorytmu Euklidesa.

14.1 Algorytm Euklidesa

Oblicza największy wspólny dzielnik a, b :

z modulo:	z odejmowaniem:
<pre>while b != 0: • $r = a \% b$ • $a, b = b, r$ return a</pre>	<pre>while a != b: • if $a > b$: $a = a - b$ • else: $b = b - a$ return a</pre>

14.2 Rozszerzony algorytm Euklidesa

Oprócz obliczenia największego wspólnego dzielnika, algorytm znajduje także współczynniki Bézouta.

Mamy następujące ciągi:

$$\begin{aligned}
 r_0 &= a, r_1 = b & r_{i+1} &= r_{i-1} - q_i r_i \\
 x_0 &= 1, x_1 = 0 & x_{i+1} &= x_{i-1} - q_i x_i \\
 y_0 &= 0, y_1 = 1 & y_{i+1} &= y_{i-1} - q_i y_i
 \end{aligned}$$

gdzie q_i oznacza wynik całkowitego dzielenia r_{i-1} przez r_i . Możemy zauważyć, że $r_i = x_i * a + y_i * b$.

15 Ortogonalność wektorów w przestrzeni $\mathbb{R}_n$; związki z liniową niezależnością. Metoda ortonormalizacji Grama-Schmidta

Definicja 15.1 *Ortogonalność.*

Dla wektorów x, y w przestrzeni $\mathbb{R}_n$ z zadany iloczynem skalarnym $\cdot$

$$x \perp y \Leftrightarrow x \cdot y = 0$$

Definicja 15.2 *Liniowa niezależność*

Wektory $x_1, x_2, \dots, x_n$ w przestrzeni $\mathbb{R}^n$ nazywamy *liniowo niezależnymi* jeśli

$$a_1 * x_1 + a_2 * x_2 + \dots + a_n * x_n = 0 \Rightarrow a_1 = a_2 = \dots = a_n = 0$$

W przestrzeni $\mathbb{R}_n$ zbiór liniowo niezależnych nie może zawierać więcej niż n wektorów.

Każdy układ ortogonalny wektorów w przestrzeni $\mathbb{R}^n$ jest liniowo niezależny.

Definicja 15.3 *Baza przestrzeni $\mathbb{R}^n$*

Zbiór wektorów $B \subset \mathbb{R}^n$ nazywamy *bazą przestrzeni $\mathbb{R}^n$* jeśli:

1. Jest liniowo niezależny
2. Generuje przestrzeń $\mathbb{R}^n$ tzn. każdy wektor $\in \mathbb{R}^n$ można zapisać jako kombinację liniową wektorów z B

Każdy ortogonalny układ n wektorów w $\mathbb{R}^n$ jest bazą.

Definicja 15.4 Ortonormalność

Zbiór wektorów A przestrzeni $\mathbb{R}^n$ z zadany iloczynem skalarnym $\cdot$ jest ortonormalny, jeśli

$$\forall x, y \in A \quad x \cdot y = \begin{cases} 0, & x \neq y \\ 1, & x = y \end{cases}$$

Zatem wektory należące do A są **wersorami**.

15.1 Ortonormalizacja Grama-Schmidta

Ortonormalizacja to proces przekształcenia układu niezależnych liniowo wektorów w układ wektorów ortonormalnych.

Projekcję wektora $\mathbf{v}$ na wektor $\mathbf{u}$ definiujemy jako:

$$\text{proj}_{\mathbf{u}} \mathbf{v} = \frac{\mathbf{v} \cdot \mathbf{u}}{\mathbf{u} \cdot \mathbf{u}} \mathbf{u}$$

1. Dla zbioru wektorów $A = \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ oblicz $\{\mathbf{u}_1, \dots, \mathbf{u}_n\}$ gdzie

$$\mathbf{u}_i = \mathbf{v}_i - \sum_{j=1}^{i-1} \text{proj}_{\mathbf{u}_j} \mathbf{v}_i, \quad \text{tzn: } \mathbf{u}_1 = \mathbf{v}_1, \quad \mathbf{u}_2 = \mathbf{v}_2 - \text{proj}_{\mathbf{u}_1} \mathbf{v}_2 \text{ itd.}$$

2. Wektory $\{\mathbf{u}_1, \dots, \mathbf{u}_n\}$ podziel przez ich normy, uzyskując $\{\mathbf{e}_1, \dots, \mathbf{e}_n\}$ - układ wektorów ortonormalnych.

16 Liczby Stirlinga I i II rodzaju i ich interpretacja.

Definicja 16.1 *Liczby Stirlinga I rodzaju.*

Przez $c(n, k)$ ($[n]_k$) oznaczamy **liczbę permutacji zbioru n -elementowego, które mają rozkład na dokładnie k cykli rozłącznych.**

Dla dowolnego $n \geq 1$ mamy:

1. $c(0, 0) = 1$
2. $c(n, 0) = c(0, n) = 0$
3. $c(n, k) = c(n-1, k-1) + (n-1) * c(n-1, k)$

W szczególności: $c(n, n) = 1, c(n, 1) = (n-1)!, \sum_{k=0}^n c(n, k) = n!$.

Definicja 16.2 *Liczby Stirlinga II rodzaju.*

Przez $S(n, k)$ ($\{ \{ \}_k^n \}$) oznaczamy **liczbę rozmieszczeń n rozróżnialnych kul na k nierozróżnialnych stosach** w taki sposób, aby żaden **stos nie był pusty**.

Dla dowolnego $n \geq 1$ mamy:

1. $S(0, 0) = 1$,
2. $S(n, 0) = S(0, n) = 0$,
3. $S(n, k) = S(n - 1, k - 1) + k \times S(n - 1, k)$.

w szczególności $S(n, n) = S(n, 1) = 1$.

17 Twierdzenia Eulera i Fermata; funkcja Eulera.

Definicja 17.1 *Funkcja Eulera (Tocjent)*

Funkcja przypisująca każdej **liczbie naturalnej** **liczbę mniejszych liczb względnie pierwszych z nią**.

$$\sum_{m|n} \varphi(m) = n$$

Własności

1. $\varphi(n) \leq n - 1$ dla każdego $n > 1$
2. $\varphi(p) \leq p - 1$ dla każdego p będącego liczbą pierwszą
3. $\varphi(mn) = \varphi(m)\varphi(n)$ jeśli $\text{NWD}(m, n) = 1$ (m i n względnie pierwsze)
4. $\varphi(p^k) = p^{k-1} \cdot (p - 1)$ jeśli p jest liczbą pierwszą
5. $\varphi(n) = n \left(1 - \frac{1}{p_1}\right) \left(1 - \frac{1}{p_2}\right) \cdots \left(1 - \frac{1}{p_n}\right)$ gdzie $p_1, p_2, \dots, p_n$ są czynnikami pierwszymi liczby n
6. jeżeli $n = \prod_{i=1}^k p_i^{k_i}$ jest rozkładem liczby n na czynniki pierwsze $\varphi(n) = \prod_{i=1}^k \varphi(p_i^{k_i})$

Definicja 17.2 Małe Twierdzenie Fermata jeżeli p - pierwsza, to:

1. $a^{p-a} \equiv 0 \pmod{p} \quad \forall a \in \mathbb{Z},$
2. $a^{p-1} \equiv 1 \pmod{p} \quad \forall a : a, p \text{ względnie pierwsze.}$

Definicja 17.3 Twierdzenie Eulera

dla $m \in \mathbb{Z}_+, a \in \mathbb{Z}$ względnie pierwszych:

$$a^{\varphi(m)} \equiv 1 \pmod{m}$$

18 Konfiguracje i t-konfiguracje kombinatoryczne.

Definicja 18.1 Konfiguracją kombinatoryczną B o parametrach (n, k, r) nazywamy **rodzinę k -elementowych podzbiorów** w X , jeżeli **każdy element** $x \in X$ występuje w dokładnie r podbiorach.

1. $n = |X|$
2. $|B_i| = k \quad \forall i = 1, \dots, b$
3. $n \cdot r = b \cdot k \Rightarrow k | n \cdot r$

Konfiguracja istnieje wtw. $\frac{n \cdot r}{k} \leq \binom{n}{k}$.

Definicja 18.2 *T-konfiguracja* używa r_t oznaczające ilość wystąpień t -elementowych podzbiorów X .

$$r_{t-1} = r_t \cdot \frac{n-t+1}{k-t+1}$$

Jeśli B jest t -konfiguracją kombinatoryczną, to B jest s -konfiguracją kombinatoryczną dla $s = 1, 2, \dots, t-1$.

$$r_s = r_t \cdot \frac{(n-s)(n-s-1)\dots(n-t+1)}{(k-s)(k-s-1)\dots(k-t+1)}$$

Warunek konieczny (nie wystarczający) dla istnienia t -konfiguracji ($t \geq 2$):

$$(k-s)(k-s-1)\dots(k-t+1) \mid r_t(n-s)(n-s-1)\dots(n-t+1) \text{ dla } 0 \leq s \leq (t-1)$$

19 Cykl Hamiltona, obwód Eulera, liczba chromatyczna - definicje i twierdzenia.

19.1 Cykl Hamiltona

- **Ścieżka Hamiltona** to ścieżka przechodząca przez wszystkie wierzchołki.
- **Cykl Hamiltona** to nim cykl przechodzący przez wszystkie wierzchołki. Taki graf nazywamy **hamiltonowskim**.
- Jeśli dla dowolnych **różnych, niesąsiednich** wierzchołków $u, v \in V$ zachodzi:
 - $d(u) + d(v) \geq n$, to G jest **hamiltonowski** ($|V_G| \geq 3$).
 - $d(u) + d(v) \geq n - 1$, to G ma **ścieżkę Hamiltona** ($|V_G| \geq 2$).
- Jeśli **minimalny stopień** wierzchołka wynosi co najmniej $\frac{n}{2}$, to G jest **hamiltonowski** ($|V_G| \geq 3$).

19.2 Obwód Eulera

- **Droga Eulera** to droga w której każda krawędź użyta jest dokładnie raz.

- **Obwód Eulera** to droga Eulera w której pierwszy i ostatni wierzchołek są identyczne. Graf ten nazywamy wtedy **eulerowskim**.

- Jeśli G spójny to:

G jest grafem **eulerowskim** $\Leftrightarrow$ stopień każdego wierzchołka w G jest **parzysty**

G ma **drogę** Eulera $\Leftrightarrow G$ ma dokładnie **0 lub 2** wierzchołki stopnia **nieparzystego**

19.3 Liczba chromatyczna

- **Kolorowanie wierzchołkowe** grafu $G = (V, E)$ przy użyciu (co najwyżej) k kolorów to funkcja $c : V \rightarrow \{1, \dots, k\}$ spełniająca warunek

$$c(u) \neq c(v) \quad \forall u, v \in V : uv \in E$$

- **Liczba chromatyczna** to najmniejsza liczba kolorów dla której istnieje kolorowanie wierzchołkowe G . Oznaczamy $\chi(G)$.

1. $\chi(G) \leq d_{\max}(G) + 1$.
 2. $\chi(G) = 1 \Leftrightarrow |E(G)| = 0$
 3. $\chi(T) = 2$ dla każdego drzewa T o przynajmniej dwóch wierzchołkach
 4. $\chi(C_{2k}) = 2, \chi(C_{2k+1}) = 3$ dla grafu cyklicznego
 5. $\chi(K_n) = n$ dla grafu pełnego
- G spójny $\Rightarrow \chi(G) \leq d_{\max}(G)$, o ile G nie jest grafem pełnym ani cyklem o nieparzystej liczbie wierzchołków.

20 Algorytm Forda-Fulkersona wyznaczania maksymalnego przepływu.

- **Siecią przepływową** nazywamy graf skierowany z wyróżnionym **źródłem** i **ujściem** oraz **funkcją pojemności krawędzi**.
- **Przepływem** nazywamy funkcję f spełniającą następujące warunki:

1. $\forall \vec{uv} \in \vec{E} : f(\vec{uv}) \leq c(\vec{uv})$,
2. $\forall v \in V \setminus \{s, t\} : \sum_{\vec{uv} \in \vec{E}} f(\vec{uv}) = \sum_{\vec{vw} \in \vec{E}} f(\vec{vw})$.

- **Wartość przepływu** – $|f| = \sum_{\vec{s}\vec{u} \in \vec{E}} f(\vec{s}\vec{u}) = \sum_{\vec{w}\vec{t} \in \vec{E}} f(\vec{w}\vec{t})$
- **Sieć rezydualna** to sieć zawierająca również krawędzie przeciwne do krawędzi w $\vec{E}$ z funkcją pojemności określoną wzorem:

$$\begin{aligned} c_f(\vec{u}\vec{v}) &= c(\vec{u}\vec{v}) - f(\vec{u}\vec{v}) \quad \text{dla } \vec{u}\vec{v} \in \vec{E} \text{ t. że } c(\vec{u}\vec{v}) > 0 \\ c_f(\vec{u}\vec{v}) &= f(\vec{u}\vec{v}) \quad \text{dla } \vec{u}\vec{v} \notin \vec{E} \text{ t. że } \vec{v}\vec{u} \in \vec{E} \text{ i } f(\vec{u}\vec{v}) > 0 \end{aligned}$$

oraz $c_f(\vec{u}\vec{v}) = 0$ w pozostałych przypadkach (ile jeszcze może płynąć w przód/ile płynie wstecz).

- **Ścieżka rozszerzająca** dla przepływu f to dowolna ścieżkę skierowaną łączącą wierzchołki źródło z ujściem krawędziami z niepełną pojemnością.
- **Algorytm Forda-Fulkersona** znajduje **maksymalny przepływ** w sieci:
 1. przyjmujemy początkowo dowolny (np. zerowy) przepływ f ,
 2. budujemy sieć rezydualną G_f ,
 3. dopóki w G_f istnieje ścieżka rozszerzająca P określamy jej przepływ c_p i uaktualniamy wartości funkcji c_f sieci rezydualnej.
- **Przekrój** sieci przepływowej to para rozłącznych zbiorów (S, T) dzielących V na podzbiór z źródłem i podzbiór z ujściem. **Pojemność przekroju**:

$$c(S, T) = \sum_{(u,v) \in \vec{E} \cap (S \times T)} c(\vec{u}\vec{v})$$

- **Maksymalna wartość przepływu** jest równa **minimalnej pojemności przekroju**.

21 Rozwiązywanie równań rekurencyjnych przy użyciu funkcji tworzących (generujących) oraz przy użyciu równania charakterystycznego.

Definicja 21.1 *Liniowym równaniem rekurencyjnym rzędu r (o stałych współczynnikach) nazywamy równanie postaci:*

$$x_{n+r} = c_1 x_{n+r-1} + \cdots + c_r x_n + f(n), \quad c_i \in \mathbb{R}, c_r \neq 0, f: \mathbb{N} \rightarrow \mathbb{R}$$

*Jeśli funkcja $\forall n \ f(n) = 0$, to powyższe równanie nazywamy **jednorodnym**.*

Rozwiązani szczególne to dowolny ciąg spełniający zależność rekurencyjną.

Rozwiązanie ogólne to wzór ogólny pozwalający wyznaczyć wszystkie rozwiązania szczególne.

Definicja 21.2 *Funkcja tworząca (generująca) ciągu (a_n) to szereg*

$$\sum_{n=0}^{\infty} a_n x^n.$$

Jest to tylko inny zapis ciągu (wyraz a_n to współczynnik przy x^n).

Operacje na funkcjach tworzących.

1. Funkcje tworzące można **dodawać, odejmować i mnożyć przez liczbę**.
2. **Mnożeniu dwóch funkcji tworzących** odpowiada operacja "splotu", tzn. jeśli $A(x) = \sum_{n=0}^{\infty} a_n x^n$ i $B(x) = \sum_{n=0}^{\infty} b_n x^n$, to $A(x) * B(x) = \sum_{n=0}^{\infty} c_n x^n$, gdzie $c_n = \sum_{k=0}^n a_k b_{n-k}$.
3. Jeśli funkcja tworząca $A(x)$ ma "element odwrotny", tzn. taką funkcję tworzącą $B(x)$ dla której $A(x) * B(x) = 1$, to mówimy że $A(x)$ jest **odwracalna** i piszemy $\frac{1}{A(x)} = B(x)$.
4. Funkcje tworzące można **różniczkować** według wzoru:

$$\left(\sum_{n=0}^{\infty} a_n x^n \right)' = \sum_{n=1}^{\infty} n a_n x^{n-1} = \sum_{n=0}^{\infty} (n+1) a_{n+1} x^n$$

5. Zachodzą wzory na **pochodne** znane z analizy, np. $(A(x)*B(x))' = A'(x)B(x) + A(x)B'(x)$.

6. Operacja "**całkowania**" (odwrotna do różniczkowania):

$$\int (\sum_{n=0}^{\infty} a_n x^n) = \sum_{n=0}^{\infty} \frac{a_n}{n+1} x^{n+1} = \sum_{n=0}^{\infty} \frac{a_n - 1}{n} x^n$$

Wyraz a_n można "odzyskać" z funkcji tworzącej $A(x)$ wykonując jej n -krotne różniczkowanie.

Twierdzenie 21.3 *Zachodzi: Dla dowolnej liczby naturalnej $m \geq 1$ zachodzi wzór:*

$$\frac{1}{(1-x)^m} = \sum_{n=0}^{\infty} \binom{n+m-1}{n} x^n \quad \forall m \in \mathbb{N}$$

$$(1+x)^\alpha = \sum_{n=0}^{\infty} \binom{\alpha}{n} x^n \quad \forall \alpha \in \mathbb{R}, |x| < 1$$

Definicja 21.4 ***Równanie charakterystyczne** równania rekurencyjnego. Weźmy równanie rekurencyjne jednorodne z danymi A, B :*

$$a_n = Aa_{n-1} + Ba_{n-2}$$

Założmy, że ma ono rozwiązanie postaci $a_n = t^n$. Podstawiając otrzymujemy:

$$t^n = At^{n-1} + Bt^{n-2}$$

$$t^2 = At^1 + B$$

$$t^2 - At^1 - B = 0$$

Jest to równanie charakterystyczne. r_1, r_2 – pierwiastki równania, C, D – stałe:

$$a_n = \begin{cases} Cr_1^n + Dr_2^n, & r_1 \neq r_2 \\ (C + Dn)r_1^n, & r_1 = r_2 \end{cases}$$

22 Ciąg i granica ciągu liczbowego, granica funkcji.

22.1 Ciągi.

Definicja 22.1 *Ciąg liczbowy* – funkcja $\mathbb{N} \rightarrow \mathbb{R}$, oznaczamy przez (a_n) , zbiór wartości $\{a_n\}$, n -ty wyraz a_n .

Definicja 22.2 *Granice.*

Granica właściwa ciągu: $\lim_{n \rightarrow \infty} a_n = a, \quad a \in \mathbb{R} \Leftrightarrow$

$$\forall \varepsilon > 0 \quad \exists n_0 \in \mathbb{N} \quad \forall n \in \mathbb{N} \quad [(n > n_0) \Rightarrow (|a_n - a| < \varepsilon)]$$

Granica niewłaściwa ciągu: $\lim_{n \rightarrow \infty} a_n = +(-)\infty \Leftrightarrow$

$$\forall \varepsilon > (<)0 \quad \exists n_0 \in \mathbb{N} \quad \forall n \in \mathbb{N} \quad [(n > n_0) \Rightarrow (a_n > (<)\varepsilon)]$$

Twierdzenie 22.3 *Twierdzenia.*

1. **O ograniczoności ciągu zbieżnego.** Jeśli ciąg jest zbieżny do granicy właściwej, to jest ograniczony.
2. **O równoważności granic:** $\lim_{n \rightarrow \infty} a_n = 0 \Leftrightarrow \lim_{n \rightarrow \infty} |a_n| = 0$.
3. **O dwóch ciągach.** Weźmy ciągi $(a_n), (b_n) : a_n \leq b_n \forall n \geq n_0$, wtedy:

$$\begin{aligned} \lim_{n \rightarrow \infty} a_n = \infty &\Rightarrow \lim_{n \rightarrow \infty} b_n = \infty \\ \lim_{n \rightarrow \infty} b_n = -\infty &\Rightarrow \lim_{n \rightarrow \infty} a_n = -\infty \end{aligned}$$

4. **O trzech ciągach.** Weźmy ciągi $(a_n), (b_n), (c_n) : a_n \leq b_n \leq c_n \forall n \geq n_0$:

$$\lim_{n \rightarrow \infty} a_n = \lim_{n \rightarrow \infty} c_n = b \Rightarrow a_n \leq b_n \leq c_n \quad \forall n \geq n_0$$

5. **O ciągu monotonicznym i ograniczonym.** Jeżeli ciąg (a_n) jest nie-malejący dla $n \geq n_0$ oraz ograniczony z góry, to jest zbieżny do granicy właściwej $\sup\{a_n : n \geq n_0\}$. Nierosnący analogicznie.

22.2 Funkcje.

Definicja 22.4 *Heinego granicy właściwej funkcji w punkcie.* $x_0 \in \mathbb{R}$, f – określona na sąsiedztwie $S(x_0)$:

$$\begin{aligned} \lim_{x \rightarrow x_0} f(x) &= g \\ \Leftrightarrow \\ \forall (x_n): \{x_n\} \subset S(x_0) \quad &[(\lim_{n \rightarrow \infty} x_n = x_0) \Rightarrow (\lim_{n \rightarrow \infty} f(x_n) = g)]. \end{aligned}$$

Definicja 22.5 *Cauchy’ego granicy właściwej funkcji w punkcie.* $x_0 \in \mathbb{R}$, f określona przynajmniej na sąsiedztwie $S(x_0)$:

$$\begin{aligned} \lim_{x \rightarrow x_0} f(x) &= g \\ \Leftrightarrow \\ \forall \varepsilon > 0 \quad \exists \delta > 0 \quad \forall x \in S(x_0) \quad &[(|x - x_0| < \delta) \Rightarrow (|f(x) - g| < \varepsilon)]. \end{aligned}$$

Granica niewłaściwa:

$$\begin{aligned} \lim_{x \rightarrow x_0} f(x) &= \infty \\ \Leftrightarrow \\ \forall \varepsilon > 0 \quad \exists \delta > 0 \quad \forall x \in S(x_0) \quad &[(|x - x_0| < \delta) \Rightarrow (f(x) > \varepsilon)]. \end{aligned}$$

Twierdzenie 22.6 *Warunek konieczny i wystarczający istnienia granicy.*

$$\lim_{x \rightarrow x_0^-} f(x) = \lim_{x \rightarrow x_0^+} f(x)$$

Twierdzenie 22.7 O nieistnieniu granicy funkcji w punkcie. Jeśli istnieją ciągi $(x'_n), (x''_n)$ spełniające warunki:

1. $\lim_{n \rightarrow \infty} x'_n = x_0$, przy czym $x'_n \neq x_0 \quad \forall n \in \mathbb{N}$ oraz $\lim_{n \rightarrow \infty} f(x'_n) = g'$,
2. $\lim_{n \rightarrow \infty} x''_n = x_0$, przy czym $x''_n \neq x_0 \quad \forall n \in \mathbb{N}$ oraz $\lim_{n \rightarrow \infty} f(x''_n) = g''$,
3. $g' \neq g''$,

to granica $\lim_{x \rightarrow x_0} f(x)$ nie istnieje (właściwa ani niewłaściwa).

Twierdzenie 22.8 O nieistnieniu granicy funkcji w nieskończoności. Jeżeli istnieją ciągi $(x'_n), (x''_n)$ spełniające warunki:

1. $\lim_{n \rightarrow \infty} x'_n = \infty$ oraz $\lim_{n \rightarrow \infty} f(x'_n) = g'$,
2. $\lim_{n \rightarrow \infty} x''_n = \infty$ oraz $\lim_{n \rightarrow \infty} f(x''_n) = g''$,
3. $g' \neq g''$,

to nie istnieje granica $\lim_{x \rightarrow x_0} f(x)$ (właściwa ani niewłaściwa).

Twierdzenie 22.9 O dwóch funkcjach. Weźmy $f, g : f(x) \leq g(x) \quad \forall x \in S(x_0)$, wtedy:

$$\lim_{x \rightarrow x_0} f(x) = \infty \Rightarrow \lim_{x \rightarrow x_0} g(x) = \infty$$

Granice specjalne

$$\lim_{x \rightarrow +\infty} a^x x^\alpha \stackrel{[0, \infty]}{=} 0 \text{ dla } a \in (0, 1), \alpha \geq 0$$

$$\lim_{x \rightarrow 0} \frac{\sin x}{x} \stackrel{[\frac{0}{0}]}{=} 1 \text{ oraz } \lim_{x \rightarrow 0} \frac{\arcsin x}{x} \stackrel{[\frac{0}{0}]}{=} 1$$

$$\lim_{x \rightarrow 0} (1+x)^{\frac{1}{x}} \stackrel{1^\infty}{=} e \text{ oraz } \lim_{x \rightarrow \infty} (1+x)^{\frac{1}{x}} \stackrel{\infty^0}{=} 1$$

$$\lim_{x \rightarrow 0} \frac{a^x - 1}{x} \stackrel{[\frac{0}{0}]}{=} \ln a \text{ dla } a > 0, \text{ w szczególności } \lim_{x \rightarrow 0} \frac{e^x - 1}{x} \stackrel{[\frac{0}{0}]}{=} 1$$

$$\lim_{x \rightarrow 0} \frac{\log_a(1+x)}{x} \stackrel{[\frac{0}{0}]}{=} \frac{1}{\ln a}, \text{ w szczególności } \lim_{x \rightarrow 0} \frac{\ln(1+x)}{x} \stackrel{[\frac{0}{0}]}{=} 1$$

$$\lim_{x \rightarrow \pm\infty} \left(1 + \frac{a}{x}\right)^x \stackrel{[1^\infty]}{=} e^a, \text{ dla } a \in \mathbb{R}$$

$$\lim_{x \rightarrow 0} \frac{(1+x)^a - 1}{x} \stackrel{[\frac{0}{0}]}{=} a, \text{ dla } a \in \mathbb{R}$$

Twierdzenie 22.10 Reguła de L’Hostpiala. Jeżeli funkcja f i g spełniają warunki:

1. $\lim_{x \rightarrow x_0} f(x) = \lim_{x \rightarrow x_0} g(x) = 0 \quad (\infty)$, przy czym $g(x) \neq 0$ dla $x \in S(x_0)$
2. istnieje granica $\lim_{x \rightarrow x_0} \frac{f'(x)}{g'(x)}$ (właściwa lub niewłaściwa),

to

$$\lim_{x \rightarrow x_0} \frac{f(x)}{g(x)} = \lim_{x \rightarrow x_0} \frac{f'(x)}{g'(x)}$$

23 Ciągłość i pochodna funkcji. Definicja i podstawowe twierdzenia.

23.1 Ciągłość.

Definicja 23.1 *Funkcja ciągła w punkcie.* $x_0 \in \mathbb{R}$, f określona przynajmniej na otoczeniu $O(x_0)$.

$$f \text{ ciągła w } x_0 \Leftrightarrow \lim_{x \rightarrow x_0} f(x) = f(x_0).$$

Funkcja jest ciągła na zbiorze, jeżeli jest ciągła w każdym punkcie tego zbioru.

Twierdzenie 23.2 *Warunek konieczny i wystarczający ciągłości funkcji.* Funkcja jest ciągła w punkcie wtw, gdy jest w tym punkcie ciągła lewostronnie i prawostronnie.

Definicja 23.3 *Nieciągłość funkcji.* $x_0 \in \mathbb{R}$, f określona przynajmniej na otoczeniu $O(x_0)$. Funkcja f jest nieciągła w punkcie x_0 wtedy i tylko wtedy, gdy nie istnieje granica $\lim_{x \rightarrow x_0} f(x)$ albo gdy $\lim_{x \rightarrow x_0} f(x) \neq f(x_0)$.

Nieciągłość pierwszego rodzaju:

$$\lim_{x \rightarrow x_0^-} f(x) \neq f(x_0) \quad \text{lub} \quad \lim_{x \rightarrow x_0^+} f(x) \neq f(x_0).$$

1. typu "skok": $\lim_{x \rightarrow x_0^-} f(x) \neq \lim_{x \rightarrow x_0^+} f(x)$.

2. typu "luka": $\lim_{x \rightarrow x_0^-} f(x) = \lim_{x \rightarrow x_0^+} f(x) \neq f(x_0)$.

Nieciągłość drugiego rodzaju. Jeżeli co najmniej jedna z granic

$$\lim_{x \rightarrow x_0^-} f(x), \quad \lim_{x \rightarrow x_0^+} f(x)$$

nie istnieje lub jest niewłaściwa.

Twierdzenie 23.4 *Twierdzenia*

1. **Weierstrassa o ograniczoności funkcji ciągłej.** Jeżeli funkcja jest ciągła na przedziale domkniętym i ograniczonym, to jest na nim ograniczona.
2. **Weierstrassa o osiągnięciu kresów.** Jeżeli funkcja f jest ciągła na przedziale domkniętym $[a, b]$, to

$$\exists c \in [a, b] \quad f(c) = \inf_{x \in [a, b]} f(x) \quad \text{oraz} \quad \exists d \in [a, b] \quad f(d) = \sup_{x \in [a, b]} f(x)$$

3. **Darboux o przyjmowaniu wartości pośrednich.** Jeżeli funkcja f jest ciągła na przedziale $[a, b]$ oraz spełnia warunek $f(a) < f(b)$, to

$$\forall w \in (f(a), f(b)) \quad \exists c \in (a, b) \quad f(c) = w.$$

23.2 Pochodna.

Definicja 23.5 Iloraz różnicowy. $x_0 \in \mathbb{R}$, f określona przynajmniej na otoczeniu $O(x_0, r)$, gdzie $r > 0$. Ilorazem różnicowym odpowiadającym przyrostowi $0 < |\Delta x| < r$ nazywamy:

$$\frac{\Delta f}{\Delta x} \stackrel{\text{def}}{=} \frac{f(x_0 + \Delta x) - f(x_0)}{\Delta x}$$

Definicja 23.6 Pochodna właściwa funkcji. $x_0 \in \mathbb{R}$, f określona przynajmniej na $O(x_0)$. Pochodna właściwa funkcji f w punkcie x_0 :

$$f'(x_0) \stackrel{\text{def}}{=} \lim_{x \rightarrow x_0} \frac{f(x) - f(x_0)}{x - x_0}$$

$$f'(x_0) \stackrel{\text{def}}{=} \lim_{\Delta x \rightarrow 0} \frac{f(x_0 + \Delta x) - f(x_0)}{\Delta x}$$

Pochodne jednostronne definiuje się alogicznie na jednostronnym otoczeniu.

Twierdzenie 23.7 Warunek konieczny istnienia pochodnej właściwej funkcji. Jeżeli funkcja ma pochodną właściwą w punkcie, to jest ciągła w tym punkcie. Implikacja odwrotna nie jest prawdziwa.

Definicja 23.8 Pochodna funkcji na zbiorze. Funkcja ma pochodną właściwą na zbiorze wtedy i tylko wtedy, gdy ma pochodną właściwą w każdym punkcie tego zbioru.

Twierdzenie 23.9 Zastosowanie różniczki do obliczeń przybliżonych. Jeżeli funkcja f ma pochodną właściwą w punkcie x_0 , to

$$f(x_0 + \Delta x) \approx f(x_0) + f'(x_0)\Delta x$$

Twierdzenie 23.10 Rolle'a. Jeśli funkcja f spełnia warunki:

1. jest ciągła na $[a, b]$
2. ma pochodną właściwą lub niewłaściwą na (a, b) ,
3. $f(a) = f(b)$,

to istnieje punkt $c \in (a, b)$ taki, że:

$$f'(c) = 0.$$

Twierdzenie 23.11 Lagrange'a. Jeżeli funkcja f spełnia warunki:

1. jest ciągła na $[a, b]$,
2. ma pochodną właściwą lub niewłaściwą na (a, b) ,

to istnieje punkt $c \in (a, b)$ taki, że

$$f'(c) = \frac{f(b) - f(a)}{b - a}$$

24 Ekstrema funkcji jednej zmiennej. Definicje i twierdzenia.

Definicja 24.1 *Minimum lokalne funkcji.* Funkcja f ma w punkcie $x_0 \in \mathbb{R}$ minimum lokalne, jeżeli:

$$\exists \delta > 0 \quad \forall x \in S(x_0, \delta) \quad f(x) \geq f(x_0).$$

Minimum lokalne jest właściwe, jeżeli:

$$\exists \delta > 0 \quad \forall x \in S(x_0, \delta) \quad f(x) > f(x_0).$$

Analogicznie definiujemy **maksimum** lokalne (właściwe).

Twierdzenie 24.2 *Fermata, warunek konieczny istnienia ekstremum.* Jeżeli funkcja f ma ekstremum lokalne w punkcie x_0 , oraz istnieje pochodna $f'(x_0)$, to $f'(x_0) = 0$.

Twierdzenie 24.3 *I warunek wystarczający istnienia ekstremum.* Jeżeli funkcja f spełnia warunki:

1. $f'(x_0) = 0$,
2. $\exists \delta > 0 \begin{cases} f'(x) > 0 & \forall x \in S(x_0^-, \delta), \\ f'(x) < 0 & \forall x \in S(x_0^+, \delta), \end{cases}$

to w punkcie x_0 ma maksimum lokalne właściwe. Analogicznie dla minimum.

Twierdzenie 24.4 II warunek wystarczający istnienia ekstremum.
 Jeżeli funkcja f spełnia warunki:

1. $f'(x_0) = f''(x_0) = \dots = f^{(n-1)}(x_0) = 0$,
2. $f^{(n)}(x_0) < 0$,
3. n jest liczbą parzystą, gdzie $n \geq 2$,

to w punkcie x_0 ma maksimum lokalne właściwe. Analogicznie dla minimum,

25 Całka Riemanna funkcji jednej zmiennej.

Definicja 25.1 Całka oznaczona Riemanna. Niech funkcja f będzie ograniczona na przedziale $[a, b]$. Całkę oznaczoną Riemanna z funkcji f na przedziale $[a, b]$ definiujemy wzorem

$$\int_a^b f(x) dx \stackrel{\text{def}}{=} \lim_{\delta(P) \rightarrow 0} \sum_{k=1}^n f(x_k^*) \Delta x_k,$$

o ile po prawej stronie znaku równości granica jest właściwa oraz nie zależy od sposobu podziałów P przedziału $[a, b]$ ani od sposobów wyboru punktów pośrednich x_k^* , gdzie $1 \leq k \leq n$. Ponadto przyjmujemy

$$\int_a^a f(x) dx \stackrel{\text{def}}{=} 0 \quad \text{oraz} \quad \int_b^a f(x) dx \stackrel{\text{def}}{=} - \int_a^b f(x) dx \quad \text{dla } a < b$$

Funkcję, dla której istnieje całka Riemanna, nazywamy całkowaną.

Twierdzenie 25.2 Warunek wystarczający całkowności funkcji. Jeżeli funkcja f jest ograniczona na przedziale $[a, b]$ i ma na tym przedziale skończoną liczbę punktów nieciągłości I rodzaju, to jest na nim całkowna.

Twierdzenie 25.3 *Obliczanie całek przy pomocy sumy całkowej podziału równomiernego.* Jeżeli funkcja f jest całkowalna na przedziale $[a, b]$, to

$$\int_a^b f(x) dx = \lim_{n \rightarrow \infty} \left[\frac{b-a}{n} \sum_{k=1}^n f\left(a + k \frac{b-a}{n}\right) \right]$$

Twierdzenie 25.4 *Newtona - Leibniza, główne twierdzenie rachunku całkowego.* Jeżeli funkcja f jest ciągła na przedziale $[a, b]$, to

$$\int_a^b f(x) dx = F(b) - F(a) = [F(x)]_a^b,$$

gdzie F oznacza dowolną funkcję pierwotną funkcji f na tym przedziale.

26 Pochodne cząstkowe funkcji wielu zmiennych; różniczkowalność i różniczka funkcji.

Definicja 26.1 *Pochodne cząstkowe.* Niech funkcja f będzie określona przynajmniej na otoczeniu punktu (x_0, y_0) . Pochodną cząstkową pierwszego rzędu funkcji f względem x w punkcie (x_0, y_0) określamy wzorem:

$$\frac{\partial f}{\partial x}(x_0, y_0) \stackrel{\text{def}}{=} \lim_{\Delta x \rightarrow 0} \frac{f(x_0 + \Delta x, y_0) - f(x_0, y_0)}{\Delta x}$$

Definicja 26.2 *Funkcja różniczkowalna w punkcie.* Niech istnieją pochodne cząstkowe $\frac{\partial f}{\partial x}(x_0, y_0)$ $\frac{\partial f}{\partial y}(x_0, y_0)$. Funkcja f jest różniczkowalna w punkcie (x_0, y_0) wtw, gdy:

$$\lim_{(\Delta x, \Delta y) \rightarrow (0,0)} \frac{f(x_0 + \Delta x, y_0 + \Delta y) - f(x_0, y_0) - \frac{\partial f}{\partial x}(x_0, y_0)\Delta x - \frac{\partial f}{\partial y}(x_0, y_0)\Delta y}{\sqrt{\Delta x^2 + \Delta y^2}} = 0$$

Definicja 26.3 Różniczka funkcji. Niech funkcja f ma pochodne cząstkowe pierwszego rzędu w punkcie (x_0, y_0) . Różniczką funkcji f w punkcie (x_0, y_0) nazywamy funkcję $df(x_0, y_0)$ zmiennych $\Delta x, \Delta y$ określoną wzorem:

$$df(x_0, y_0)(\Delta x, \Delta y) \stackrel{\text{def}}{=} \frac{\partial f}{\partial x}(x_0, y_0)\Delta x + \frac{\partial f}{\partial y}(x_0, y_0)\Delta y$$

Twierdzenie 26.4 Zastosowanie różniczki funkcji do obliczeń przybliżonych. Niech funkcja f ma ciągle pochodne cząstkowe pierwszego rzędu w punkcie (x_0, y_0) . Wtedy

$$f(x_0 + \Delta x, y_0 + \Delta y) \approx f(x_0, y_0) + df(x_0, y_0)(\Delta x, \Delta y)$$

27 Ekstrema funkcji wielu zmiennych. Definicje i twierdzenia.

Definicja 27.1 *Minimum lokalne funkcji dwóch zmiennych.* Funkcja f ma w punkcie (x_0, y_0) minimum lokalne, jeżeli istnieje jego otoczenie O : $\forall x, y \in O$:

$$f(x, y) \geq f(x_0, y_0)$$

Przy ostrej nierówności mówimy o minimum lokalnym **właściwym**. Analogicznie definiujemy **maksimum** lokalne (właściwe).

Twierdzenie 27.2 *Warunek konieczny istnienia ekstremum.* Jeżeli funkcja f ma ekstremum lokalne w punkcie (x_0, y_0) i istnieją pochodne cząstkowe w tym punkcie, to:

$$\frac{\partial f}{\partial x}(x_0, y_0) = 0, \quad \frac{\partial f}{\partial y}(x_0, y_0) = 0$$

Funkcja może mieć ekstrema tylko w punktach, w których wszystkie jej pochodne cząstkowe pierwszego rzędu się zerują albo w punktach, w których choć jedna z nich nie istnieje.

Twierdzenie 27.3 *Warunek wystarczający istnienia ekstremum.* Niech funkcja f ma ciągle pochodne cząstkowe rzędu drugiego na otoczeniu punktu (x_0, y_0) oraz niech

1. $\frac{\partial f}{\partial x}(x_0, y_0) = 0, \frac{\partial f}{\partial y}(x_0, y_0) = 0$

2. $\det \begin{bmatrix} \frac{\partial^2 f}{\partial^2 x}(x_0, y_0) & \frac{\partial^2 f}{\partial x \partial y}(x_0, y_0) \\ \frac{\partial^2 f}{\partial x \partial y}(x_0, y_0) & \frac{\partial^2 f}{\partial^2 y}(x_0, y_0) \end{bmatrix} > 0$

Wtedy w punkcie (x_0, y_0) funkcja f ma ekstremum lokalne i jest to:

1. **minimum**, gdy $\frac{\partial^2 f}{\partial^2 x}(x_0, y_0) > 0$,

2. **maksimum**, gdy $\frac{\partial^2 f}{\partial^2 x}(x_0, y_0) < 0$.

28 Twierdzenie o zmianie zmiennych w rachunku całkowym; współrzędne walcowe i sferyczne.

Definicja 28.1 *Twierdzenie o zmianie zmiennych w rachunku całkowym. Niech*

1. odwzorowanie $T : \begin{cases} x = \phi(u, v, w) \\ y = \psi(u, v, w) \\ z = \chi(u, v, w) \end{cases}$ przekształca różnowartościowo wnętrze obszaru regularnego Δ na wnętrze obszaru regularnego V ,
2. funkcje ϕ, ψ, χ mają ciągle pochodne cząstkowe rzędu pierwszego na pewnym zbiorze otwartym zawierającym obszar Δ ,
3. funkcja f jest ciągła na obszarze V ,
4. jacobian J_T jest różny od zera wewnątrz obszaru Ω .

Wtedy

$$\iiint_V f(x, y, z) dx dy dz = \iiint_{\Omega} f(\phi(u, v, w), \psi(u, v, w), \chi(u, v, w)) |J_T| du dv dw$$

gdzie

$$J_T(u, v, w) \stackrel{\text{def}}{=} \det \begin{bmatrix} \frac{\partial \phi}{\partial u}(u, v, w) & \frac{\partial \phi}{\partial v}(u, v, w) & \frac{\partial \phi}{\partial w}(u, v, w) \\ \frac{\partial \psi}{\partial u}(u, v, w) & \frac{\partial \psi}{\partial v}(u, v, w) & \frac{\partial \psi}{\partial w}(u, v, w) \\ \frac{\partial \chi}{\partial u}(u, v, w) & \frac{\partial \chi}{\partial v}(u, v, w) & \frac{\partial \chi}{\partial w}(u, v, w) \end{bmatrix}$$

Obszar regularny: złożony z obszarów normalnych, czyli "ogrodzonych" funkcjami ciągłymi.

Definicja 28.2 *Współrzędne walcowe.*

$$W : \begin{cases} x = \varrho \cos \varphi \\ y = \varrho \sin \varphi \\ z = h \end{cases} \quad |J_W| = \varrho$$

φ - miara kąta między rzutem promienia wodzącego punktu P na płaszczyznę xOy , a dodatnią częścią osi Ox , $0 \leq \varphi \leq 2\pi$ albo $-\pi < \varphi \leq \pi$

ϱ - odległość rzutu punktu na xOy od punktu $(0,0,0)$, $0 \leq \varrho < \infty$

h - oznacza odległość punktu P od płaszczyzny xOy , $-\infty < h < \infty$

Definicja 28.3 *Współrzędne sferyczne.*

$$S : \begin{cases} x = \varrho \cos \varphi \cos \psi \\ y = \varrho \sin \varphi \cos \psi \\ z = \varrho \sin \psi \end{cases} \quad |J_S| = \varrho^2 \cos \psi$$

φ - oznacza miarę kąta między rzutem promienia wodzącego punktu P na płaszczyznę xOy , a dodatnią częścią osi Ox , $0 \leq \varphi \leq 2\pi$ albo $-\pi < \varphi \leq \pi$

ψ - oznacza miarę kąta między promieniem wodzącym punktu P , a płaszczyznę xOy , $-\frac{\pi}{2} \leq \psi \leq \frac{\pi}{2}$

ϱ - oznacza odległość punktu P od początku układu współrzędnych, $0 \leq \varrho < \infty$

Teoretyczne podstawy informatyki

29 Metody dowodzenia poprawności pętli.

Asercja - warunek logiczny wyrażający zależności między zmiennymi algorytmu w kontekście ich aktualnego stanu.

- **początkowa** - określa warunek wejściowy algorytmu.
- **końcowa** - określa wyniki algorytmu.
- Asercje początkowa i końcowa stanowią **specyfikację algorytmu**

Algorytm A:

- ma własność **określoności obliczeń** względem warunku α , jeśli dla każdego danych spełniających warunek α działanie algorytmu A nie zostanie zerwane.
- ma **własność stopu** względem warunku α , jeśli dla każdego danych spełniających warunek α działanie algorytmu A nie ciągnie się w nieskończoność (oznaczenie $\text{stop}(\alpha, A)$).
- jest **częściowo poprawny** względem warunku początkowego α i końcowego β , gdy dla każdego danych spełniających α , **jeżeli** działanie algorytmu dochodzi do końca, wyniki spełniają β :

$$cp(\alpha, z \leftarrow w, \beta) \iff (\alpha \Rightarrow \beta|_{z \leftarrow w})$$

- jest **całkowicie** (semantycznie) **poprawny** względem warunku początkowego α i końcowego β , jeśli ma własność określoności obliczeń względem α , ma własność stopu względem α oraz jest częściowo poprawny względem α i β :

$$sp(\alpha, A, \beta) \iff obl(\alpha, A) \wedge stop(\alpha, A) \wedge cp(\alpha, A, \beta)$$

30 Odwrotna Notacja Polska: definicja, własności, zalety i wady, algorytmy.

- notacja postfiksowa,
- jednoznacznie wyznacza kolejność wykonywania działań,
- pozwala na całkowitą rezygnację z nawiasów.

- Zalety:
 - ułatwione obliczenia na komputerze - wykorzystuje jedynie stos
 - prosty algorytm konwersji między standardowym zapisem infiksowym a ONP wykorzystywany w wielu algorytmach wykorzystujących jako wejście zapis działań brak konieczności stosowania nawiasów
- Wady:
 - mniej czytelny dla człowieka (wymaga przyzwyczajenia)
 - podczas zapisu na kartce “12 34” + może wyglądać jak “123 4” + (xD)

30.1 Algorytm obliczenia wartości wyrażenia ONP

Dla wszystkich symboli z wyrażenia ONP wykonuj:

- jeśli symbol jest liczbą, to odłóż go na stos,
- jeśli symbol jest operatorem (funkcją):
 - zdejmij ze stosu oczekiwaną liczbę elementów (parametrów),
 - odłóż na stos wynik działania operatora.

Zdejmij ze stosu wynik.

30.2 Algorytm konwersji z notacji infiksowej do ONP

Wykonuj dopóki dane na wejściu.

- Weź kolejny element z wejścia.
- Jeśli element jest operandem, to dopisz element na wyjście.
- Wpp (element jest operatorem, przecinkiem lub nawiasem):
 - Jeśli element jest operatorem o1, to dopóki:
 - * lewa łączność(o1) i priorytet $\leq$ od operatora na stosie, lub
 - * prawa łączność(o1) i priorytet $<$ od operatora na stosie
 wstaw o1 na wyjście.
 - Lub jeśli element == '(' to wstaw element na stos.
 - Lub jeśli element == ')' to przepisuj elementy ze stosu na wyjście aż do '(', nawias usuń.

Przepisz elementy ze stosu na wyjście.

31 Modele obliczeń: maszyna Turinga.

Definicja 31.1 *Formalna definicja maszyny Turinga.* Maszynę Turinga opisujemy poprzez krotkę:

$$MT = \langle Q, \Sigma, \delta, \tau, q_0, B, F \rangle$$

gdzie:

- Q - skończony zbiór stanów,
- $q_0 \in Q$ - stan początkowy,
- $F \subset Q$ - zbiór stanów końcowych,
- τ - skończony zbiór dopuszczalnych symboli,
- $b \in \tau$ - symbol pusty,
- $\Sigma \subset \tau \setminus \{b\}$ - zbiór symboli wejściowych,
- δ - funkcja sterująca.
 - W wersji niedeterministycznej postaci maszyny Turinga:

$$\delta \subset (Q \times \tau) \times (\tau \times \{\leftarrow, \rightarrow, \downarrow\} \times Q)$$

- W wersji deterministycznej:

$$\delta : Q \times \tau \mapsto \tau \times \{\leftarrow, \rightarrow, \downarrow\} \times Q$$

Tablica sterująca to dwuwymiarowa tablica indeksowana: stanami w jednym wymiarze, a symbolami taśmy w drugim. Elementy tablicy zawierają trójki działania maszyny, czyli wartości funkcji sterowania δ .

32 Modele obliczeń: automat skończony, automat ze stosem.

32.1 Automat skończony deterministyczny

Definicja 32.1 *Automat skończony deterministyczny* oznaczamy piętką parametrów:

$$\mathcal{A} = (S, A, f, s_0, T)$$

S – zbiór stanów, A – alfabet, $T \subseteq S$ – zbiór terminali, $s_0 \in S$ – stan początkowy
 $f : S \times A \rightarrow S$ – funkcja przejścia

Każdy język skończony jest akceptowany przez pewien deterministyczny automat skończony.

Definicja 32.2 Automaty $\mathcal{A}_1$ i $\mathcal{A}_2$ są **równoważne**, jeżeli rozpoznają ten sam język, czyli:

$$L(\mathcal{A}_1) = L(\mathcal{A}_2)$$

Definicja 32.3 Każdy automat $\mathcal{A} = (S, A, f, s_0, T)$ wyznacza w wolnym monoidzie A^* **prawą kongruencję automatową** określoną w następujący sposób:

$$\forall u, v \in A^*, u \sim_{\mathcal{A}} v \Leftrightarrow f(s_0, u) = f(s_0, v)$$

Automat ilorazowy to automat, w którym stanami są klasy równoważności.

Definicja 32.4 $L \subset A^*$ język, $u \in A^*$ słowo. **Pochodną Brzozowskiego (residuum)** z języka L względem słowa u nazywamy język

$$u^{-1}L = \{w \in A^* : uw \in L\}$$

Definicja 32.5 Monoidem przejść automatu $\mathcal{A}$ nazywamy monoid:

$$\mathcal{M}(\mathcal{A}) = \tau_{\mathcal{A}}(A^*) \subset S^S = \{\tau_{\mathcal{A}}(a) : a \in A^*\}$$

będący zbiorem generatorów (unikalnych reprezentacji) automatu $\mathcal{A}$.

32.2 Automat skończony niedeterministyczny

Definicja 32.6 *Automat skończony niedeterministyczny* różni funkcja przejścia f zdefiniowana:

$$f : S \times A \rightarrow \mathcal{P}(S)$$

Słowo x jest akceptowane gdy $f^*(S_0, x) \cap T \neq \emptyset$

32.3 Automat ze stosem

Definicja 32.7 *Automat ze stosem* to

$$\mathcal{AS} = (A, A_S, Q, f, s_0, z_0, Q_F)$$

A – alfabet, A_S – alfabet stosu, Q – zbiór stanów,

$Q_F \subset Q$ – zbiór stanów końcowych

$q_0 \in Q$ – stan początkowy, $z_0 \in A_S$ – symbol początkowy stosu

$f: A_s \times Q \times (A \cup \{1\}) \rightarrow \mathcal{P}_{sk}(A_S^* \times Q)$ – funkcja przejść

33 Złożoność obliczeniowa - definicja notacji: O , Ω , Θ .

Definicja 33.1 Niech $f, g, h : \mathbb{N} \rightarrow \mathbb{R}_+ \cup \{0\}$, wtedy:

- $f(n) = O(g(n))$ – f jest **co najwyżej rzędu** g , gdy

$$\exists c > 0, n_0 \in \mathbb{N} \forall n \geq n_0 f(n) \leq cg(n).$$

- $f(n) = \Omega(g(n))$ – f jest **co najmniej rzędu** g , gdy

$$g(n) = O(f(n)).$$

- $f(n) = \Theta(g(n))$ – f jest **dokładnie rzędu** g , gdy

$$f(n) = O(g(n)) \wedge f(n) = \Omega(g(n)).$$

34 Złożoność obliczeniowa - pesymistyczna i średnia.

Definicja 34.1 *Niech:*

D_n - zbiór danych rozmiaru n ,

$t(d)$ - liczba operacji dominujących,

X_n - zmienna losowa dla $t(d) \in D_n$,

p_{kn} - rozkład prawdopodobieństwa zmiennej X_n .

Optymistyczna złożoność czasowa:

$$Opt(n) = \inf\{t(d) : d \in D_n\}$$

Średnia złożoność czasowa:

$$A(n) = ave(X_n) = \sum_{k \geq 0} k p_{nk}$$

Pesymistyczna złożoność czasowa:

$$W(n) = \sup\{t(d) : d \in D_n\}$$

Definicja 34.2 *Koszt amortyzowany to średni czas wykonania przypadający na jedną operację w pesymistycznym ciągu operacji. Koszt amortyzowany różni się od kosztu średniego tym, że bierze pod uwagę pesymistyczny ciąg operacji i nie jest metodą probabilistyczną.*

35 Metoda "dziel i zwyciężaj": zalety i wady.

1. **Dziel** problem na mniejsze podproblemy, aż do przypadków bazowych.
2. **Zwyciężaj** rozwiązując przypadki bazowe.
3. **Łącz** wyniki otrzymane z podproblemów.

35.1 Zalety

- Optymalne użycie pamięci podręcznej (*cache*): problemy dostatecznie małe rozwiązywane są w pamięci nieprzekraczającej wielkości pamięci podręcznej.

- Znacząco lepsza złożoność obliczeniowa od bardziej prymitywnych podejść.

35.2 Wady

- Rekurencja nieliniowa - przekształcenie w algorytm iteracyjny jest nietrywialne (w szczególności nie zostanie to wykonane przez kompilator).
- Potencjalnie wysoka złożoność pamięciowa algorytmu rekurencyjnego / iteracyjnego ze stosem. Ponadto rozmiar stosu może przekroczyć pamięć komputera.
- Możliwość wielokrotnego rozwiązywania identycznych problemów (programowanie dynamiczne, memoization)/

36 Lista: ujęcie abstrakcyjne, możliwe implementacje i ich złożoności.

Definicja 36.1 *Lista jest abstrakcyjną strukturą danych (ADT), która posiada następujące właściwości*

- *Przechowuje elementy **tego samego typu***
- *Dostęp do danych jest **sekwencyjny***
- *Elementy w liście są **liniowo uporządkowane***
- *W sensie matematycznym lista jest skończonym ciągiem elementów ustalonego typu.*

Wyróżnia się dwa podstawowe sposoby implementacji listy:

- **Tablica** - wykorzystuje pamięć w sposób **statyczny**, posiada zmienne wskazujące na jej maksymalny rozmiar, ilość zajętych miejsc.
- **Lista wiązana**:
 1. Lista wiązana **jednokierunkowa** – z każdego elementu możliwe jest przejście do jego następnika.
 2. Lista wiązana **dwukierunkowa** – z każdego elementu możliwe jest przejście do jego następnika lub poprzednika.
 3. Lista **z głową**, **cykliczna** itd.

37 Kolejka i kolejka priorytetowa: ujęcie abstrakcyjne, możliwe implementacje i ich złożoności.

37.1 Kolejka

Definicja 37.1 *Kolejka **FIFO** (First In First Out) to lista, w której:*

- Wstawianie nowego elementu odbywa się na końcu kolejki **rear**.
- Usuwanie elementu odbywa się na początku kolejki **front**.
- Wyróżnia się dwa podstawowe sposoby implementacji kolejki:
 - Tablica cykliczna,
 - Lista wiązana.
- Wszystkie operacji w czasie stałym.

37.2 Kolejka Priorytetowa

Definicja 37.2 *Kolejka Priorytetowa to lista, w której:*

- Usuwany jest zawsze element o największej (najmniejszej) wartości.
- Wyróżnia się trzy podstawowe sposoby implementacji kolejki priorytetowej:
 - Kopiec Max (Min)
 - Lista/Tablica Nieuporządkowana - **NIEEFEKTYWNE!**
 - Lista/Tablica Uporządkowana Rosnąco (Malejąco) - **NIEEFEKTYWNE!**

38 Algorytmy sortowania QuickSort i MergeSort: metody wyboru pivota w QS; złożoności.

38.1 QuickSort.

```
def quickSort(arr, start, end):  
    if (start < end):
```



```

pivot = partition(arr, start, end)
quickSort(arr, start, pivot - 1)
quickSort(arr, pivot + 1, end)

```

```

def partition (arr, start, end):
    pivot = arr[end]
    i = (start - 1)

    for j in range [start, end- 1]:
        if (arr[j] < pivot):
            i++;
            swap arr[i] and arr[j]

    swap arr[i + 1] and arr[end])
    return (i + 1)

```

Złożoność: pesymistyczna - $O(n^2)$, średnia i optymistyczna - $O(n \log_2 n)$.

Sposoby wybrania pivota: pierwszy, ostatni lub losowy element; **mediana** z pierwszego, środkowego i ostatniego

38.1.1 MergeSort.

```

def mergeSort (arr, start, end):
    if end > start:
        middle = (start+end)/2

        mergeSort (arr, start, middle)
        mergeSort (arr, middle+1, end)

    merge (arr, start, middle, right)

```

Złożoność: pesymistyczna, średnia, optymistyczna - $O(n \log n)$.

39 Algorytm sortowania bez porównań (sortowanie przez zliczanie, sortowanie kubełkowe oraz sortowanie pozycyjne).

39.1 CountSort.

```
def countSort (arr):  
    count = []  
    for a in arr:  
        count[a] += 1  
  
    i = 0;  
    for j in range [0, arr.len]:  
        while (count[i] == 0):  
            i++  
        arr[j] = i  
        count[i]--
```

Złożoność: $O(n + k)$.

39.2 BucketSort.

```
bucketSort (arr):  
    n = arr.len  
    buckets = [{} for i in range [1,n]]  
  
    for a in arr:  
        buckets[n*arr[i]].add(arr[i])  
  
    for b in buckets:  
        sort(b)  
  
    i = 0  
    for b in buckets:  
        for a in b:  
            arr[i++] = a
```

Złożoność: pesymistyczna - $O(n^2)$, średnia - $O\left(n + \frac{n^2}{k} + k\right)$.

39.3 RadixSort.

```
def radixSort (arr):  
    for all digits ascending:  
        countSort(arr, digit)
```

Złożoność: $O(d(n + k))$, gdzie d jest liczbą cyfr.

40 Reprezentacja drzewa binarnego za pomocą porządków (preorder, inorder, postorder).

```
def preorder (v):  
    func(v)  
    preorder(v.left)  
    preorder(v.right)  
  
def inorder (v):  
    inorder(v.left)  
    func(v)  
    inorder(v.right)  
  
def postorder (v):  
    postorder(v.left)  
    postorder(v.right)  
    func(v)
```

Możemy odtworzyć wyjściowe drzewo, jeśli mamy inorder i post- lub pre-order.

41 Algorytmy wyszukiwania następnika i poprzednika w drzewach BST; usuwanie węzła.

Definicja 41.1 *Binarne drzewo poszukiwań (BST)* - dynamiczna struktura danych będąca drzewem binarnym, w którym lewe poddrzewo każdego węzła zawiera wyłącznie elementy o kluczach mniejszych niż klucz węzła a prawe poddrzewo zawiera wyłącznie elementy o kluczach nie mniejszych niż klucz węzła.

41.1 Wyszukiwanie następnika i poprzednika w BST.

Definicja 41.2 *Następnik* danego węzła to węzeł, który jest odwiedzany jako **następny** w przypadku przechodzenia drzewa metodą **inorder**. Sposób wyznaczenia nie wymaga porównywania żadnych kluczy, jest to w praktyce:

- skrajnie lewy element prawego poddrzewa węzła, jeśli istnieje; wpp
- pierwszy napotkany przodek węzła, dla którego węzeł jest w lewym poddrzewie.

Definicja 41.3 *Poprzednik* danego węzła to węzeł, który jest odwiedzany jako **poprzedni** w przypadku przechodzenia drzewa metodą **inorder**. Sposób wyznaczenia nie wymaga porównywania żadnych kluczy, jest to w praktyce:

- skrajnie prawy element lewego poddrzewa węzła, jeśli istnieje; wpp
- pierwszy napotkany przodek węzła, dla którego węzeł jest w prawym poddrzewie.

41.2 Usuwanie węzła z BST.

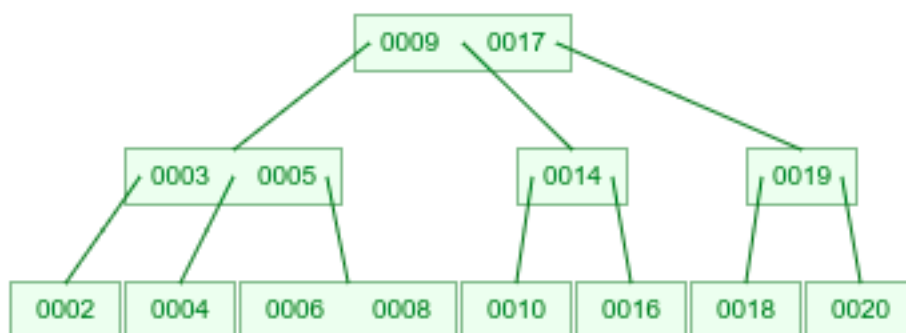
Twierdzenie 41.4 *Usuwanie* węzła z BST w trzech przypadkach:

1. usuwany węzeł jest **liściem** – **usuwamy**,
2. usuwany węzeł ma **jednego** syna – podstawiamy **syna**,
3. usuwany węzeł ma **dwóch** synów – podstawiamy jego **następnik**.

42 B-drzewa: operacje i ich złożoność.

- B-drzewa to **rozszerzone drzewa BST** zoptymalizowane pod kątem operacji wejścia/wyjścia.
- Mają ustalony **rzęd**. B-drzewo rzędu K musi spełniać poniższe warunki:
 - Korzeń jest liściem lub ma od 2 do K synów
 - Wszystkie liście są na tym samym poziomie $\rightarrow$ wysokość $\in [\log_{\frac{K}{2}} \frac{n}{\frac{K}{2}}, \log_K \frac{n}{K}]$
 - Każdy węzeł wewnętrzny (oprócz korzenia) ma $s \in [\lceil K/2 \rceil, K]$ synów i $s - 1$ kluczy (wartości)

- **Złożoność** wszystkich operacji na B-drzewach – $O(K \log_K n)$.
 - wyszukiwanie – podobne jak w BST, do przeszukiwania węzłów używamy binsearcha
 - wstawianie – wyszukujemy odpowiedni liść i do niego wstawiamy; jeśli się przepełni ($\geq K$), to jedną z wartości wstawiamy do rodzica a pozostałe dzielimy na dwa nowe liście (rekurencja)
 - usuwanie – znajdujemy żądany element, zastępujemy największym mniejszym lub najmniejszym większym i równoważymy drzewo
- Odmianą B-drzew są **drzewa B+** – wszystkie wartości trzymane są w liściach, liście powiązane w listę.

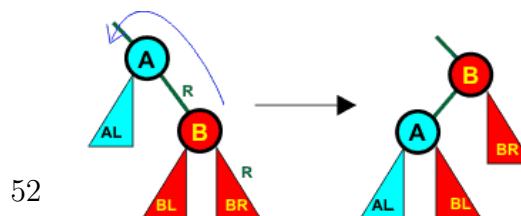


43 Drzewa AVL: rotacje, operacje z wykorzystaniem rotacji i ich złożoność.

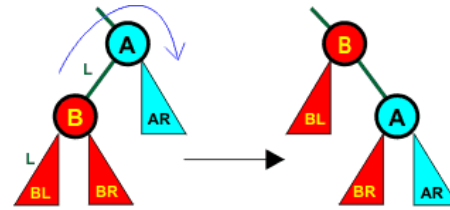
- odmiana drzew BST, w której **różnica między wysokością prawego i lewego poddrzewa** danego węzła wynosi **maksymalnie 1**.
- Operacje wyszukiwania, wstawiania oraz usuwania mają **złożoność** $O(\log_2 n)$.

43.1 Rotacje

Rotacja RR



Rotacja LL



- **Rotacja RR** – gdy dla węzła A prawe poddrzewo z korzeniem B zaburza równowagę, ale samo poddrzewo B jest zrównoważone ("pojedyncza nierównowaga na prawo"),
- **Rotacja LL** – analogicznie dla lewego poddrzewa ("pojedyncza nierównowaga na prawo"),
- **Rotacja RL** (LL $\rightarrow$ RR) – prawe poddrzewo B niezrównoważone przez jego lewe poddrzewo C ("podwójna nierównowaga prawo-lewo"),
- **Rotacja LR** (RR $\rightarrow$ LL) – lewe poddrzewo B niezrównoważone przez jego prawe poddrzewo C ("podwójna nierównowaga lewo-prawo").

44 Algorytmy przeszukiwania wszerz i w głąb w grafach.

Przejście przez wszystkie **osiągalne** z wierzchołka startowego s wierzchołki grafu G .

- złożoność obliczeniowa – $\theta(e + v)$
- złożoność pamięciowa – $\theta(v)$

44.1 Przeszukiwanie grafu wszerz (*breadth first search*, BFS)

Algorytm

Dopóki kolejka nie jest pusta:

1. Zdejmij x z kolejki
2. Pomaluj x na czarno
3. Dla każdego niepomalowanego na czarno v , sąsiadującego z x wrzuć v do kolejki

44.1.1 Cechy

- **Kompletność:** BFS dla **nieskończonego grafu** zwróci wszystkie poprawne odpowiedzi, w przeciwieństwie do DFS.
- **Zastosowania:**
 - Najkrótsza droga do s
 - Wykrywanie cykli grafu nieskierowanego
 - Sprawdzanie dwudzielności grafu
 - Algorytmy sieci przepływowych
 - Garbage collection

44.2 Przeszukiwanie grafu włąb (*depth first search*, DFS)

Algorytm

Dla danego wierzchołka x (zaczynając od s):

1. Pomaluj x na czarno
2. Wywołaj się rekurencyjnie (odłóż x na **stos**) dla każdego niepomalowanego na czarno v , sąsiadującego z x

44.2.1 Cechy

- **Niekompletność** dla grafów **nieskończonych**
- **Zastosowania**
 - Minimalne drzewo rozpinające (dla grafów nieważonych)
 - Wykrywanie cykli w grafie skierowanym/nieskierowanym
 - Sortowanie topologiczne
 - Algorytmy sieci przepływowych
 - Garbage collection

45 Algorytmy wyszukiwania najkrótszej ścieżki (Dijkstra oraz Bellmana-Forda).

Algorytmy pozwalające na wyliczenie najkrótszych ścieżek od wierzchołka startowego s .

Funkcja relaksująca sprawdza, czy ścieżka z u do w przez v jest krótsza od aktualnie najkrótszej ścieżki.

$relax(u, v, w)$:

Jeśli $D[u, w] > D[u, v] + D[v, w]$ to:

$$D[u, w] = D[u, v] + D[v, w]$$

$$P[u, w] = v$$

45.1 Algorytm Dijkstry

- Algorytm zachłanny
- Nie działa poprawnie dla grafów z wagami ujemnymi
- **Złożoność:**
 - Implementacja tablicowa – $O(|V|^2)$
 - Implementacja z kopcem binarnym – $O(|E| \log(|V|))$

Algorytm

$P = \{\}$ – słownik wierzchołek : rodzic

D – słownik odległości od s

$S = \{v \in V : D[v]\}$ – kolejka priorytetowa nieodwiedzonych wierzchołków względem ich odległości.

$$D[s] = 0; D[v] = \infty \quad \forall v \in V \setminus \{s\}.$$

Dopóki S niepusta:

1. Usuń wierzchołek v o najmniejszej wartości z S
2. Dla każdego wierzchołka w sąsiadującego z v wywołaj $relax(s, v, w)$

45.2 Algorytm Bellmana-Forda

- Działa poprawnie dla grafów z wagami ujemnymi (bez cykli o wagach ujemnych)
- Pozwala decydować, czy graf ma cykle ujemne
- Używany w protokole RIP
- Algorytm można zoptymalizować: jeśli w danej iteracji nie dokonał zmiany, należy przerwać jego działanie (ścieżki już są najkrótsze)
- **Złożoność** – $O(|V| \cdot |E|)$

Algorytm

$P = \{\}$ – słownik wierzchołek : rodzic

D – słownik odległości od s

$D[s] = 0; D[v] = \infty \forall v \in V \setminus \{s\}$.

Dla każdego wierzchołka w $V \setminus \{s\}$:

Wywołaj $relax(s, u, v)$ dla każdej krawędzi $(u, v) \in E$

Na koniec można wykonać sprawdzenie, czy w grafie nie ma cykli o wadze ujemnej:

$$\forall (u, v) \in E : D[v] > D[u] + e_{u \rightarrow v} \Rightarrow \text{istnieje cykl o wadze ujemnej}$$

45.3 Algorytm Floyda-Warshalla

Pozwala wyliczyć najkrótsze ścieżki pomiędzy wszystkimi wierzchołkami w grafie G (nieposiadającym ujemnych cykli). Inicjujemy słownik odległości jak poniżej:

$$D[(u, w) : u, w \in V] = \begin{cases} 0, & \text{jeśli } u = w \\ e_{u \rightarrow w}, & \text{jeśli } (u, w) \in E. \\ \infty, & \text{jeśli } (u, w) \notin E \end{cases}$$

I wywołujemy $relax(u, v, w)$ dla każdej trójki $(u, v, w) : u, v, w \in V$. **Złożoność** – $\theta(v^3)$.

45.3.1 Cechy

- Prostota
- Pozwala obliczyć za jednym razem wszystkie ścieżki pomiędzy wierzchołkami
- Działa dla grafów z krawędziami ujemnymi (ale nie z cyklami ujemnymi)
- Złożoność jest niezależna od gęstości grafu

46 Programowanie dynamiczne: podział na podproblemy, porównanie z metodą "dziel i zwyciężaj".

Programowanie dynamiczne:

- Podział problemu na mniejsze podproblemy,
- Wyniki obliczeń zapisywane w tabeli (brak redundantnych obliczeń)
- Niejednokrotnie stosowanie techniki programowania dynamicznego daje w rezultacie algorytm **pseudowielomianowy**.
- **Metody programowania dynamicznego:**
 - Metoda **zstępująca** z zapamiętywaniem polega na **rekurencyjnym wywoływaniu** funkcji z **zapamiętywaniem** wyników.
 - Metoda **wstępująca** polega na rozwiązywaniu **wszystkich możliwych podproblemów**, zaczynając od tych o najmniejszym rozmiarze; nie ma stosu wywołań.
- **Zastosowanie** przy problemach typu:
 - Problem posiada powtarzające się podproblemy
 - Podproblemy musi cechować **własność optymalnej podstruktury** – optymalne rozwiązanie problemu jest funkcją optymalnych rozwiązań podproblemów.
 - Zastosowanie brute force prowadzi do **ponadwielomianowej** liczby rozwiązań podproblemów, podczas gdy sama liczba różnych podproblemów jest wielomianowa.

47 Algorytm zachłanny: przykład optymalnego i nieoptymalnego wykorzystania.

Algorytmy zachłanne w każdym kroku podejmują taką decyzję, która w danej chwili jest najkorzystniejsza (**lokalnie optymalna**) licząc, że doprowadzi to do znalezienia rozwiązania **globalnie optymalnego**.

- Nie zawsze znajdują rozwiązanie optymalne
- Są podzbiorem algorytmów **heurystycznych**
- Są to algorytmy **deterministyczne** – nie ma w nich losowości

Przykład - Pakowanie plecaka

```
posortuj nierosnąco przedmioty według wartości  $c[j]/w[j]$ 
aktualna_waga:=0

for i:=1 to n do
  if  $w[i] + aktualna\_waga \leq W$  then
    dodaj i-ty przedmiot do plecaka
    aktualna_waga := aktualna_waga +  $w[i]$ 
```

Dane wejściowe, dla których algorytm będzie działał

	Działanie	
	optymalne	nieoptymalne
Pojemność	8	7
Dane wejściowe	5, 3, 2, 1, 1, 1	3, 3, 2, 2
Rozwiązanie	5 + 3 + 2 :)	3 + 3 = 6 :(optymalne: 3 + 2 + 2 = 7

48 Kolorowania wierzchołkowe (grafów planarnych) i krawędziowe grafów, algorytmy i ich złożoności.

48.1 Kolorowanie grafów

- Klasyczny problem kolorowania (krawędziowego i wierzchołkowego) jest **NP-trudny** – nie istnieją wielomianowe algorytmy kolorujące grafy zawsze w sposób optymalny.

Przykładowy algorytm kolorowania krawędziowego:

1. Użyj BFS do poruszania się po grafie.
2. Wybierz wierzchołek i nadaj różne kolory jego krawędziom.
3. Przejdź do kolejnego wierzchołka.
4. Powtarzaj aż wszystkie krawędzie zostaną pokolorowane.

Algorytmy kolorowania wierzchołkowego:

- **Algorytm SL** (smallest last):
 1. Znajdź wierzchołek o minimalnym stopniu i usuń go z grafu.
 2. Powtarzaj krok pierwszy tak długo, aż graf będzie pusty (zapamiętaj kolejność usuwanych wierzchołków).
 3. Koloruj wierzchołki zachłannie, zgodnie z ustaloną wcześniej kolejnością (zaczynając od wierzchołków usuniętych później).

Algorytm SL jest **statyczny**, **złożoność** - $O(|V| + |E|)$.

- **Algorytm SLF** (saturated largest first)
dopóki istnieją niepokolorowane wierzchołki wykonuj operacje:
 1. znajdź wierzchołek o maksymalnym stopniu spośród wierzchołków o maksymalnym stopniu nasycenia (liczbie kolorów sąsiadów)
 2. pokoloruj znaleziony wierzchołek zachłannie

Złożoność algorytmu SLF wynosi $O(|E| \log |V|)$.

48.2 Kolorowanie wierzchołkowe grafów planarnych

Twierdzenie 48.1 *Liczba chromatyczna grafu planarnego jest równa lub mniejsza niż 4*

Każdy graf planarny można pokolorować 5 kolorami w czasie liniowym.

49 Algorytmy wyszukiwania minimalnego drzewa rozpinającego: Boruvki, Prima i Kruskala.

Algorytm Boruvki - $O(E \log V)$.

1. Weź wszystkie wierzchołki, ale bez krawędzi - każdy wierzchołek jest osobnym drzewem.
2. Dla każdego drzewa znajdź krawędź o najmniejszej wadze w oryginalnym grafie, która łączy je z innym drzewem.
3. Dodaj tę krawędź do MST.
4. Powtarzaj, dopóki drzewa nie połączyły się w jedno.

Algorytm Prima - $O(V^2)$ lub $O(E \log V)$, jeśli użyta została lista sąsiedztwa i kopiec.

1. Zainicjalizuj MST jednym wierzchołkiem, wybranym losowo z grafu.
2. Spośród krawędzi łączących MST z wierzchołkami jeszcze w nim niebędącymi wybierz tę, która ma najmniejszą wagę, i dodaj ją do MST.
3. Powtarzaj krok 2 aż wszystkie wierzchołki znajdą się w MST.

Algorytm Kruskala - $O(E \log E)$ jeśli $E > V$ lub $O(E \log V)$ wpp.

1. Posortuj krawędzie niemalejąco pod względem wag.
2. Wybierz krawędź o najmniejszej wadze. Sprawdź czy tworzy cykl z MST utworzonym do tej pory. Jeśli nie, uwzględnij tę krawędź w MST. W przeciwnym wypadku, odrzuć.
3. Powtarzaj krok 2 aż otrzymasz $V - 1$ krawędzi w MST.

Złożoność: Sortowanie krawędzi zajmuje $O(E \log E)$. Po sortowaniu, iterujemy po krawędziach i używamy algorytmu find-union który może zająć $O(\log V)$. Zatem ogólna złożoność to $O(E \log E + E \log V)$. Wartość E jest mniejsza niż V^2 , więc $O(\log V) = O(\log E)$.

50 Najważniejsze algorytmy wyznaczania otoczki wypukłej zbioru punktów w układzie współrzędnych (Grahama, Jarvisa, algorytm przyrostowy (quickhull)).

Algorytm Grahama - złożoność $O(n \log n)$

1. Wybierz punkt (ozn. O) o najmniejszym y (przy równości mniejszy x).
2. Posortuj pozostałe punkty leksykograficznie względem:
 - kąta pomiędzy wektorem OP_i a osią X,
 - długości wektora OP_i .
3. Przeglądaj listę posortowanych punktów:
 - Od bieżącej pozycji weź trzy kolejne punkty (ozn. A, B, C).
 - Jeśli B leży wewnątrz trójkąta AOC, to znaczy, że nie należy do otoczki. Usuń B z listy i cofnij się o jedną pozycję (o ile $B \neq O$).
 - Wpp może należeć do otoczki wypukłej. Przejdź do następnej trójki.

4. Pozostałe na liście punkty stanowią otoczkę wypukłą.

Algorytm Jarvisa - pesymistyczna - $O(n^2)$, średnia $O(kn)$ (k - punkty w otoczce).

1. P_1 - punkt o najmniejszej współrzędnej y (przy równości mniejszy x),
2. $P_0 := [-\infty, y(P_1)]$,
3. $i := 1$,
4. powtarzaj:
 - P_{i+1} - punkt N , dla którego kąt $P_{i-1}P_iN$ jest największy,
 - można odrzucić punkty leżące po prawej stronie wektora,
 - jeśli $N = P_1$, koniec iterowania,
 - $i := i + 1$,
5. ostatecznie otoczkę tworzą punkty $P_{1...i}$.

Quickhull - pesymistyczna $O(n^2)$; średnia $O(n \log n)$.

QuickHull(P):

1. Znajdź w P punkty o minimalnym i maksymalnym x (A oraz B).
2. Podziel P na dwa podzbiory S_1 i S_2 znajdujące się nad i pod prostą AB .
3. Wywołaj rekurencyjnie $QuickHull(A, B, S_1)$ i $QuickHull(B, A, S_2)$.

QuickHull(A, B, P) - funkcja pomocnicza:

1. Jeśli P jest pusty - koniec.
2. Jeśli P ma jeden element, ten punkt należy do otoczki - koniec.
3. W przeciwnym razie:
 - Do otoczki należy punkt $C \in P$ najbardziej oddalony od prostej AB .
 - Odrzuć wszystkie punkty z wnętrza trójkąta ABC .
 - Znajdź zbiór S_1 punktów znajdujących się po prawej stronie prostej AC oraz analogiczny zbiór S_2 dla prostej BC .
 - Wywołaj rekurencyjnie $QuickHull(A, C, S_1)$ i $QuickHull(B, C, S_2)$.

51 Problemy P, NP, NP-zupełne i zależności między nimi. Hipoteza P vs. NP.

Definicja 51.1 Problem P (ang. *deterministic polynomial, deterministycznie wielomianowy*) - problem decyzyjny, dla którego rozwiązanie można **znaleźć** w czasie wielomianowym.

Definicja 51.2 Problem NP (ang. *nondeterministic polynomial*, *niedeterministycznie wielomianowy*) – problem decyzyjny, dla którego rozwiązanie można **zweryfikować** w czasie wielomianowym.

Twierdzenie 51.3 $P \stackrel{?}{=} NP$. Każdy problem P jest NP, jednak nie wiadomo czy istnieje problem NP niebędący P .

Definicja 51.4 Problem NP-zupełny (ang. *NP-Complete*) – taki problem NP, do którego można w czasie wielomianowym zredukować dowolny inny problem NP. Czasami zamiast redukcji w czasie wielomianowym używa się redukcji w pamięci logarytmicznej.

Stąd wynika, że jeśli potrafimy rozwiązać jakikolwiek problem NP-zupełny w czasie wielomianowym, to potrafimy rozwiązać w takim czasie wszystkie problemy NP.

52 Automat minimalny, wybrany algorytm minimalizacji.

52.1 Automat minimalny

Definicja 52.1 Automat $\mathcal{A} = (S, A, f, s_0, T)$ rozpoznający język L jest minimalny, jeśli posiada najmniejszą liczbę stanów spośród wszystkich automatów rozpoznających język L .

Definicja 52.2 Dla dowolnego języka $L \in \mathcal{REC}(A^*)$ automat

$$\mathcal{A}_{P_L} = (A^*/P_L^r, A, f^*, [1]_{P_L^r}, T),$$

gdzie $T = \{[w]_{P_L^r} : w \in L\}$ jest automatem minimalnym rozpoznającym język L

Definicja 52.3 *Konstrukcja automatu minimalnego z wykorzystaniem prawej kongruencji automatowej*

$L \subset A^*$ - dowolny język

$\Theta_L \subset A^* \times A^*$ - relacja równoważności dzieląca A^* na dwie klasy:

$$L \text{ oraz } A^* \setminus L$$

ρ_i dla $i \in \mathbb{N}$ - zstępujący ciąg relacji:

$$\rho_1 = \Theta_L,$$

$$\rho_i = \{(u, w) \in A^* \times A^* : \forall a \in A \cup \{\mathbf{1}\} : (ua, wa) \in \rho_{i-1}\} \text{ dla } i = 2, \dots$$

Wtedy

$$\bigcap_{i \in \mathbb{N}} \rho_i = P_L^r$$

52.2 Minimalizacja automatu

Wejście L - język

Wyjście: automat minimalny $\mathcal{A} = (S_L, A, f_F, s_L, T_L)$ taki, że $L\mathcal{A} = L$

- $S_L \leftarrow \{L\}$; Put($\mathcal{L}, L$);
- while $\mathcal{L} \neq \emptyset$ do
 - $M \leftarrow \mathbf{zdejmij}(\mathcal{L})$;
 - foreach $a \in A$ do
 - * $N \leftarrow a^{-1}M$;
 - * if $N \cap S_L = \emptyset$ then
 - $S_L \leftarrow S_L \cup \{N\}$; Put($\mathcal{L}, N$);
- foreach $M \in S_L$ do
 - foreach $a \in A$ do $f_L(M, a) \leftarrow a^{-1}M$;
- $s_L \leftarrow L$; $T_L \leftarrow \{u^{-1}L : u \in L\}$;
- return $\mathcal{A}'$;

53 Lemat o pompowaniu dla języków regularnych.

Definicja 53.1 Niech $L \subset A^*$ będzie językiem rozpoznawalnym. Istnieje liczba naturalna $N \geq 1$ taka, że dowolne słowo $w \in L$ o długości $|w| \geq N$ można rozłożyć na katenację:

$$w = v_1uv_2$$

gdzie $v_1, v_2 \in A^*, u \in A^+, |v_1u| < N$ oraz

$$v_1u^*v_2 \subset L$$

Definicja 53.2 Jeśli rozpoznawalny język $L \subset A^*$ jest nieskończony, to istnieją $v_1, v_2 \in A^*, u \in A^+$, takie, że

$$v_1u^*v_2 \subset L$$

Schemat dowodu: zakładamy, że L jest regularny; znajdujemy takie słowo, że dla dowolnego podziału lemat nie zachodzi.

54 Warunki równoważne definicji języka regularnego: automat, prawa kongruencja syntaktyczna, wyrażenia regularne.

54.1 Wyrażenia regularne

Definicja 54.1 Rodzina regularnych podzbiorów

Niech A będzie dowolnym zbiorem. Rodzina $REG(A^*)$ regularnych podzbiorów A^* to najmniejsza rodzina R podzbiorów A^* taka, że:

1. $\emptyset \in R, \forall a \in A \ a \in R$
2. jeśli $X, Y \in R$, to: $X \cup Y, X \cdot Y \in R$
3. jeśli $X \in R$, to: $X^* = \bigcup_{n=0}^{\infty} X^n \in R$

Definicja 54.2 Wyrażenie regularne

Niech A będzie dowolnym alfabetem, a zbiór $\{+, \star, \emptyset, (,)\}$ alfabetem takim, że $A \cap \{+, \star, \emptyset, (,)\} = \emptyset$.

Słowo $\alpha \in (A \cup \{+, \star, \emptyset, (,)\})^*$ jest wyrażeniem regularnym nad alfabetem A wtedy i tylko wtedy, gdy:

1. $\alpha = \emptyset$
2. $\alpha = a \in A$
3. α jest w postaci $(\beta + \gamma), (\beta\gamma), \gamma^*$, gdzie β, γ są wyrażeniami regularnymi nad alfabetem A

Rodzinę wyrażen regularnych nad A nazywamy $\mathcal{WR}$.

- **Automat skończony** to para $\mathcal{A} = (S, f)$, gdzie S – zbiór stanów, f – funkcja przejść.
- **Język rozpoznawalny** to taki dla którego istnieje automat skończony rozpoznający każdego jego słowo. Ozn. $REC(A^*)$ – rodzina wszystkich języków rozpoznawalnych nad alfabetem A .
- **Prawa kongruencja** jest wyznaczana przez automat $\mathcal{A} = (S, f)$ z s_0 na wolnym monoidzie A^* :

$$\forall u, v \in A^* \quad u \sim_{\mathcal{A}} v \Leftrightarrow f(s_0, u) = f(s_0, v)$$

Ma skończony indeks (liczbę klas równoważności) wtw gdy $\mathcal{A}$ skończony.

- **Automat ilorazowy** $\mathcal{A}_{\sim_{\mathcal{A}}}$ z generatorem $[1]_{\sim_{\mathcal{A}}}$ jest wyznaczany przez prawą kongruencję w następujący sposób:

$$\mathcal{A}_{\sim_{\mathcal{A}}} = (A^*/_{\sim_{\mathcal{A}}}, f^*), \text{ gdzie } f^*([w]_{\sim_{\mathcal{A}}}, u) = [wu]_{\sim_{\mathcal{A}}}$$

Jest automatem skończonym wtedy i tylko gdy relacja p ma skończony indeks.

Twierdzenie 54.3 Twierdzenie Kleenego

Dla każdego skończonego alfabetu A

$$REC(A^*) = REG(^*)$$

Twierdzenie 54.4 Dla dowolnego języka regularnego $L \in A^*$ następujące warunki są równoważne:

- $L = L(G)$ dla gramatyki G typu 3 (regularnej, lewoliniowej)
- $L = L(G)$ dla gramatyki G prawoliniowej
- $L = L(\mathcal{A})$ dla automatu deterministycznego $\mathcal{A}$
- $L = L(\mathcal{A}_{ND})$ dla automatu niedeterministycznego $\mathcal{A}_{ND}$
- $L = L(\mathcal{A}_{ND}^p)$ dla automatu niedeterministycznego $\mathcal{A}_{ND}^p$ z pustymi przejściami
- $L = \|\alpha\|$ dla wyrażenia regularnego α nad alfabetem A
- $L \in REG(A^*)$
- monoid syntaktyczna $M(L)$ jest skończony
- liczba różnych pochodnych Brzozowskiego dla języka L jest skończona
- L jest sumą wybranych klas równoważności pewnej prawej kongruencji na A^* o skończonym indeksie oraz $L = \bigcup_{w \in L} [w]_p$
- istnieje skończony monoid M i homomorfizm

$$\varphi : A^* \rightarrow M : L = \varphi^{-1}(\phi(L))$$

55 Automaty niedeterministyczne i deterministyczne (w tym ze stosem); determinizacja.

55.1 Determinizacja automatów skończenie stanowych

Twierdzenie 55.1 Język $L \in A^*$ jest rozpoznawany przez automat deterministyczny wtedy i tylko wtedy, gdy jest rozpoznawany przez automat niedeterministyczny.

$\mathcal{A}_{ND} = (S, f, I, F)$ - automat niedeterministyczny

$\mathcal{A}_D = (P(S), \bar{f}, I, T)$ - równoważny automat deterministyczny, gdzie:

$$\bar{f} : P(S) \times A^* \rightarrow P(S)$$

$$\forall Q \in P(S), \forall w \in A^* \quad \bar{f}(Q, w) = \bigcup_{s \in Q} f(s, w)$$

$$T = \{Q \in P(S) \mid Q \cap F \neq \emptyset\}$$

55.2 Determinizm automatu ze stosem

Definicja 55.2 *Deterministyczny automat ze stosem*

Automatem $\mathcal{AS} = (A, A_S, Q, q_0, z_0, f, Q_F)$ nazywamy **deterministycznym**, jeśli dla każdej konfiguracji $(z, q, a) \in A_S^* \times Q \times (A \cup \{1\})$

$$\#f(z, q, a) \leq 1 \text{ oraz} \\ f(z, q, 1) \neq \emptyset \Rightarrow f(z, q, a) = \emptyset \quad \forall a \in A$$

Twierdzenie 55.3 *Każdy język deterministyczny jest jednoznaczny (rozpoznawany przez gramatykę jednoznaczną), ale nie na odwrót.*

Twierdzenie 55.4 *Każdy język deterministyczny jest kontekstowy (rozpoznawany przez niedeterministyczny automat ze stosem), ale nie na odwrót.*

56 Problemy rozstrzygalne i nierozstrzygalne w teorii języków.

Problemy rozpatrywane w kategorii języków

- **Należenie** $\in$ – problem przynależności języka L do jednego z typów języków
- **Inkluzja** $\subset$ – problem zawierania się języka L_i w języku L_k
- **Równoważność** $\equiv$ – problem równoważności dwóch języków $L_1 \equiv L_2$
- **Pustość** $\emptyset$ – problem języka pustego czyli nie generującego żadnego słowa
- **Nieskończoność** ∞ – problem języka nieskończonego
- **Jednoznaczność** – każde słowo ma dokładnie jedno wyprowadzenie lewo/prawostronne.

Definicja 56.1 *Wyprowadzenie lewo/prawostronne* ($v_0 \mapsto_L^* w$, $v_0 \mapsto_P^* w$) słowa $w \in V_T^*$ w gramatyce bezkontekstowej $G = (V_N, V_T, v_0, P)$ to wyprowadzenie $v_0 \mapsto w_1 \mapsto \dots \mapsto w_n = w$ takie, że $\forall i = 0, \dots, n-1$, $w_i \mapsto w_{i+1}$ przez zastąpienie pierwszego z lewej (prawej) symbolu nieterminalnego występującego w słowie w_i .

—	3	2	1	0
Należenie $\in$	T	T	T	N
Inkluzja $\subset$	T	N	N	N
Równoważność $\equiv$	T	N	N	N
Pustość $\emptyset$	T	T	N	N
Nieskończoność ∞	T	T	N	N
Jednoznaczność	T	N	-	-

Oznaczenia:
T - rozstrzygalny
N - nierozstrzygalny

57 Klasy języków w hierarchii Chomsky'ego oraz ich zamkniętość ze względu na operacje bo- olowskie, homomorfizmy, itp.

Definicja 57.1 *Gramatyka to system $G = (V_N, V_T, P, v_0)$, w którym:*

- $V_N \neq \emptyset$ - skończony alfabet nieterminalny
- $V_T \neq \emptyset$ - skończony alfabet terminalny
- $P \subseteq (V_N \cup V_T)^+ \times (V_N \cup V_T)^*$ - skończona relacja, zbiór produkcji (praw)
- $v_0 \in V_N$ - symbol początkowy (startowy)

Ponadto:

- $V_N \cap V_T = \emptyset$
- $\forall (u, v) \in P; \#_{V_N} u \geq 1$
- $(u, v) \in P$ oznaczamy $u \rightarrow v \in P$ lub $u \rightarrow_G v$

Definicja 57.2 Gramatyka $G = (V_N, V_T, P, v_0)$ jest typu (i) dla $i = 0, 1, 2, 3$ wtedy i tylko wtedy, gdy spełnia następujące warunki:

0. **Gramatyka** spełniająca definicję ??.

1. **Kontekstowa** – każde prawo ze zbioru P ma postać

- $u_1vu_2 \rightarrow u_1xu_2$, gdzie $u_1, u_2 \in (V_N \cup V_T)^*$, $v \in V_N, x \in (V_N \cup V_T)^+$
lub
- $v_0 \rightarrow 1$, ale wtedy v_0 nie występuje po prawej stronie w żadnym prawie z P

2. **Bezkontekstowa** – każde prawo ze zbioru P ma postać

$$v \rightarrow x, \quad \text{gdzie } v \in V_N, x \in (V_N \cup V_T)^*$$

3. **Regularna** – każde prawo ze zbioru P ma postać

$$v \rightarrow v'x \text{ lub } v \rightarrow x, \quad \text{gdzie } v, v' \in V_N; x \in V_T^*$$

$$\mathcal{L}_3 \subsetneq \mathcal{L}_2 \subsetneq \mathcal{L}_1 \subsetneq \mathcal{L}_0$$

Zamkniętości języków na operacje

—	3	2	1	0
$\cup$	T	T	T	T
$\cdot$	T	T	T	T
$\star$	T	T	T	T
$\backslash$	T	N	T	N
$\cap$	T	N	T	T

Wytwarzanie oprogramowania

58 Reprezentacja liczb całkowitych; arytmetyka.

58.1 Kod znak-moduł prosty

- Reguły:
 - Sygnalizacja znaku poprzez najwyższą pozycję,
 - Cyfra 0 na pozycji znaku oznacza dodatniość,
 - Brak kodowania cyfr wyrażających wartość.
- Przykład: $q = 10, c = 5, u = 2, n = 8$

$$[+216.7]_{zmp} = 0|00216.70, \quad [-216.7]_{zmp} = 1|00216.70$$

- **Arytmetyka:** skomplikowana i czasochłonna, nie nadaje się do praktycznego stosowania.

58.2 Kod odwrotnościowy

- Reguły:
 - Najbardziej znacząca cyfra jest cyfrą znaku.
 - Zerowa cyfra znaku oznacza dodatniość, najwyższa w systemie - ujemność
 - Cyfry wartości dodatniej pozostają bez zmian
 - Cyfry wartości ujemnej uzupełnione do najwyższej cyfry systemu
- Przykład: $q = 10, c = 7, u = 5$

$$[182073.6954]_{odw} = 0|0182073.69540$$
$$[-182073.6954]_{odw} = [-0|0182073.69540]_{odw} = 9|9917926.30459$$

- **Arytmetyka** – sumowanie: dodaj kody, a w przypadku przepełnienia zwiększ wynik o najmniejszą możliwą liczbę dodatnią.

58.3 Kod uzupełnieniowy

- **Kodowanie** – jak w odwrotnościowym, dla ujemnych z dodaniem 1 na najmniej znaczącym bicie.
- **Przykład**

$$\begin{aligned}[182073.6954]_{uz} &= 0|0182073.69540 \\ [-182073.6954]_{uz} &= [-0|0182073.69540]_{odw} + [0|0000000.00001] = 9|9917926.30460\end{aligned}$$

- **Arytmetyka** – suma kodów uzupełnieniowych jest zawsze kodem uzupełnieniowym sumy. Przepełnienie pomijamy.

58.4 Kod nadmiarowy

- **Kodowanie**
 - Liczba dodatnia ma niezerową cyfrę znaku.
 - Kod nadmiarowy liczby ujemnej to kod uzupełnieniowy z 0 na pozycji znaku.
- **Przykład**

$$\begin{aligned}[182073.6954]_{nad} &= 9|0182073.69540 \\ [-182073.6954]_{nad} &= [9|9917926.30460] + [1|0000000.00000] = 0|9917926.30460\end{aligned}$$

- **Arytmetyka** – suma kodów nadmiarowych to suma liczb z powiększonym o 1 znakiem (korekta: odjęcie 1 od znaku).

59 Reprezentacja liczb rzeczywistych; arytmetyka zmiennopozycyjna.

59.1 Reprezentacja cecha-mantysa

Liczby postaci

$$m \times q^c$$

- q – **podstawa** systemu liczbowego,
- m – **mantysa** – ciąg najbardziej znaczących cyfr liczby.

- c - **cecha** – wykładnik potęgi podstawy systemu
- W rejestrze komputera pamiętane są wyłącznie mantysa oraz cecha, zaś podstawa systemu jest ustalona, zatem nie musi być reprezentowana i nie jest konieczna do działań.
- Różnorodne sposoby zapamiętania w komputerze liczb w postaci cecha-mantysą noszą nazwę **kodowań zmiennopozycyjnych**.

59.2 Kodowanie zmiennopozycyjne

- Powszechnie stosowane w komputerach dla reprezentacji **liczb rzeczywistych**.
- W systemie **dziesiętnym** dla oddzielenia mantysy i cechy powszechne użycie litery **E** (lub e), np. liczba $28.05 = 28.05e0 = 2805e-2 = 2.805e1$ itd.
- W systemie **binarnym** dla oddzielenia mantysy i cechy powszechne użycie litery **D** (lub d), np. liczba $1001.101 = 1001.101d0 = 1.001101d11$.
- Różnorodność reprezentacji wymusza konieczność ujednoznacznienia (**normalizacji**).

59.3 Normalizacja

	normalizacja całkowita	normalizacja do jedynki	normalizacja do ułamka
28.05	2805e-2	2.805e1	.2805e2
$(1001.101)_2$	1001101d-11	1.001101d11	.1001101d100

59.4 Algorytm kodowania zmiennopozycyjnego

- w_m - liczba cyfr reprezentacji mantysy (bez cyfry znaku)
- w_c - liczba cyfr reprezentacji cechy (bez cyfry znaku)
- n - liczba cyfr rejestru

$$n = 1 + w_m + 1 + w_c$$

1. Przedstaw liczbę w systemie o podstawie q .
2. Zaokrąglaj wartość liczby do w_m cyfr.
3. Zaokrąglony wynik sprowadź do znormalizowanej postaci potęgowej $m \cdot q^c$
4. Jeżeli cecha jest zbyt duża (zbyt dodatnia) by zmieścić się w przewidzianym zakresie w_c to zasygnalizuj błąd przepełnienia i zakończ.
5. Jeżeli cecha jest zbyt mała (zbyt ujemna) by zmieścić się w zakresie w_c to zakoduj zerową mantysę oraz zerową wartość cechy i zakończ.
6. Zakoduj stałopozycyjnie mantysę na $w_m + 1$ pozycjach.
7. Zakoduj stałopozycyjnie cechę na $w_c + 1$ pozycjach.

59.5 Arytmetyka

Brak ogólnego działania - dekodowanie i wykonywanie operacji wprost.

Dodawanie:

1. Zmodyfikuj mantysy dla zrównania cech.
2. Wyznacz mantysę sumy w postaci sumy mantys.
3. Zaokrąglaj sumę mantys do liczby cyfr rejestru.
4. Znormalizuj wynik.

Mnożenie:

1. Mantysa iloczynu to iloczyn mantys.
2. Cecha iloczynu to suma cech.
3. Zaokrąglaj mantysę wyniku.
4. Znormalizuj mantysę modyfikując cechę.

60 Różnice w wywołaniu funkcji statycznych, niestatycznych i wirtualnych w C++.

60.1 Funkcje statyczne.

- **Globalne funkcje statyczne** to funkcje o zakresie widoczności ograniczonym do ich *jednostki translacji*, czyli pliku kompilowanego - pliku źródłowego z dołączonymi includami).
- **Metody statyczne klas** to metody których wywołanie jest niezależne od instancjonowania klasy - wszystkie instancje klasy współdzielą kopię statycznych metod i pól. Instancja nie jest nam potrzebna, ale można jej użyć.

60.2 Funkcje niestatyczne.

- **Niestatyczne funkcje globalne** są dostępne ze wszystkich plików, które includują zawierające je plik źródłowy - trzeba zatem uważać na konflikty nazw.
- **Niestatyczne metody klas** wymagają do wywołania instancji klasy - mogą zachowywać się różnie dla różnych instancji; mogą być oznaczone `final` i `override`.

60.3 Funkcje wirtualne.

- **Wirtualne metody klas** są przydatne w polimorfizmie.
- W przypadku zwykłej metody, wywołanie jej dla wskaźnika typu klasy podstawowej zawsze wywoła instancję z klasy podstawowej, nawet jeżeli wskaźnik będzie tak naprawdę wskazywał na klasę pochodną.
- Słowo kluczowe **virtual** **wymusza dedukcję** której metody użyć **na podstawie zawartości**
- **wskaźnika**, a **nie jego typu**. Dedukcja następuje w czasie wykonania programu.

61 Sposoby przekazywania parametrów do funkcji (przez wartość, przez referencję). Zalety i wady.

61.1 Przekazywanie przez wartość.

Przekazywanie argumentu przez wartość oznacza, że argumentem może być **rvalue** (np. wyrażenie arytmetyczne), a przekazanie **lvalue (zmiennej)** powoduje utworzenie **kopii**.

- Zaletą jest **możliwość użycia wyrażenia** w wywołaniu,
- **Tworzenie kopii** może być zaletą lub wadą - umożliwia nam to swobodne modyfikowanie wartości bez modyfikowania oryginalnej zmiennej, ale może stanowić niepotrzebne obciążenie pamięciowe.

61.2 Przekazywanie przez referencję.

Funkcja otrzymuje jako argument **adres** zmiennej, a **nie jej wartość**. Wszelka modyfikacja takiego argumentu wewnątrz funkcji powoduje zmianę skojarzonej z tym argumentem zmiennej.

- Zaletą jest **nietworzenie niepotrzebnych kopii**, możliwość zmiany zmiennej spoza zakresu funkcji (zamiast np. zwracania i przypisywania nowej wartości).

- Nieprzemyślane użycie może spowodować **nieprzewidzianą** przez programistę **zmianę zawartości** zmiennej.
- **Brak możliwości użycia wyrażenia** jako argumentu funkcji - musimy zapisać jego wynik w zmiennej, by móc ją przekazać.

62 Wskaźniki, arytmetyka wskaźników, różnica między wskaźnikiem a referencją w C++.

Wskaźnik – typ zmiennej odpowiedzialnej za przechowywanie **adresu** do innej zmiennej (innego miejsca w pamięci) w obrębie naszej aplikacji. Wskaźnik może wskazywać na jakąś zmienną, strukturę, tablicę czy nawet funkcję.

Operatory związane ze wskaźnikami w C++

- `int * intPtr` – wskaźnik na `int`
- `intPtr = &a` – ampersand jako pobranie adresu
- `cout << *intPtr` – dereferencja

Arytmetyka wskaźników – mając wskaźnik `int *p` wskazujący na hipotetyczny adres 10, wykonując na nim operację `p+2` dostajemy wskaźnik wskazujący na adres `10 + 2*sizeof(int)`. Jest to przydatne szczególnie przy operowaniu na strukturach, których pola są ułożone po kolei w pamięci, np. tablicach (`*(a+i) = [a[i]]`).

Referencja – można ją traktować jako dodatkową nazwę na już istniejący obiekt, coś w rodzaju stałego wskaźnika. Składnia referencji **zabrania wykonywania operacji wskaźnikowych** – **nie można zmienić** referowanego obiektu.

- Łatwiejsza notacja - nie musimy używać operatora wyłuskania (*) do odwołania się do obiektu - odwołanie do referencji powoduje odwołanie do obiektu.
- Większe bezpieczeństwo - brak arytmetyki dla referencji znacznie utrudnia popełnienie błędów związanych z niepoprawnym odwołaniem do pamięci.

Po kompilacji (w kodzie maszynowym) referencje i wskaźniki **nie różnią się**.

63 Podstawowe założenia paradygmatu obiektowego: dziedziczenie, abstrakcja, enkapsulacja, polimorfizm.

Dziedziczenie (kompozycja):

- Umożliwia stworzenie **hierarchii obiektów** w programie.
- Polega na **przejęciu** właściwości i **funkcjonalności** obiektów klasy bazowej i ewentualnej ich **modyfikacji**.
- Np. dziedziczenie klasy "Kot" po klasie "Zwierzę" i nadpisanie wirtualnej metody "dajGłos".

Abstrakcja:

- Dany obiekt w systemie służy jako **model abstrakcyjnego "wykonawcy"**, który może wykonywać pracę, opisywać i zmieniać swój stan, oraz komunikować się z innymi obiektami w systemie, bez ujawniania, w jaki sposób zaimplementowano dane cechy.
- Uogólnienie pewnego zbioru problemów do wspólnego zbioru zachowań i cech **niezależnych od implementacji**.

Enkapsulacja (hermetyzacja, kapsułkowanie):

- **Ukrywanie pewnych danych** składowych lub metod obiektów danej klasy tak, aby były one **dostępne** tylko **metodom wewnętrznym** danej klasy lub funkcjom zaprzyjaźnionym, nie bezpośrednio.
- Zapewnia, że obiekt nie może zmieniać stanu innych obiektów w nieokreślony sposób; dostarcza interfejsu, który określa sposób współpracy z innymi obiektami.
- Gettery, settery i funkcje zaprzyjaźnione.

Polimorfizm (wielopostaciowość):

- **Referencje i wskaźniki obiektów mogą dotyczyć obiektów różnego typu**.
- Dla obiektu zapisanego w pointerze na klasę bazową – domyślnie dedukcja wartości z klasy bazowej, przy wirtualnych metodach - z faktycznego typu obiektu.
- Dwa rodzaje polimorfizmu: **dynamiczny** - wykonywany podczas działania programu, oraz **statyczny** - na etapie kompilacji.

64 Funkcje zaprzyjaźnione i ich związek z przeładowaniem operatorów w C++.

- Funkcja zaprzyjaźniona dla danej klasy to funkcja oznaczona słowem kluczowym *friend*.
- Funkcja ta ma dostęp do niepublicznych (*private* i *protected*) składowych klasy.
- Mechanizm zaprzyjaźnienia jest specyficzny dla języka C++
 - Funkcje zaprzyjaźnione mogą się odnosić wyłącznie do **obiektów globalnych** lub danych przez argumenty
 - **Dowolna ilość klas** może deklarować zaprzyjaźnienie jednej funkcji
 - Funkcje zaprzyjaźnione mogą być **przeładowane**, zarówno w definiowanej klasie, jak i na zewnątrz
 - Funkcje zaprzyjaźnione mogą być **definiowane wewnątrz klasy**, ale nie stanowią jej metody a szczególną definicję zewnętrznej funkcji.
- Przy **przeładowaniu operatorów** często stosuje się *friend* w celu dostępu do pól prywatnych przez funkcje zewnętrzne (np. dla operatora «).

65 Programowanie generyczne na podstawie szablonów w języku C++.

To **paradygmat programowania** w którym algorytmy (funkcje, klasy) są zapisywane **bez specyfikacji typów**, na których działają. W C++ szablony *template* pozwalają na programowanie generyczne.

Zalety szablonów:

- Zmniejszają redundancję
- Wydajny, elastyczny, wielozadaniowy kod (STL)
- Metaprogramowanie
- Polimorfizm rozwiązywany w czasie kompilacji

W C++ istnieją szablony klas, struktur, funkcji (również metod), szablonów.

- Argumenty końcowe szablonu mogą mieć wartości domyślne.
- Możliwość **specjalizacji szablonu** – dostarczenia niestandardowego kodu dla danego zestawu argumentów – np. **vector booli** przechowywany na bitach.

66 Podstawowe kontenery w STL z szerszym omówieniem jednego z nich.

Standard Template Library (STL) to biblioteka C++ zawierająca algorytmy, kontenery, iteratory oraz inne konstrukcje w formie szablonów, gotowe do użycia w programach.

Kontenery zarządzają **pamięcią alokowaną** do przechowywania ich elementów, i zapewniają metody pozwalające na dostęp do nich, bezpośrednio lub przez iteratory (obiekty o właściwościach podobnych do wskaźników).

Cztery rodzaje kontenerów:

- **Kontenery sekwencyjne** – sekwencyjny dostęp do elementów struktury.
 - array, vector, deque, forward_list, list
- **Adaptery** - kontenery wykorzystujące inny kontener zapewniając mu pożądany interfejs, np. `stack` oferujący działania stosu dla np. wektora.
 - stack, queue, priority_queue
- **Kontenery asocjacyjne** – implementują posortowane struktury danych, które da się szybko przeszukiwać (złożoność $O(\log n)$)
 - set, multiset, map, multimap
- **Nieuporządkowane kontenery asocjacyjne** – implementują nieposortowane (haszowane) struktury danych, które mogą być szybko przeszukiwane (zamortyzowana złożoność $O(1)$, w najgorszym przypadku $O(n)$)
 - unordered_set, unordered_multiset, unordered_map, unordered_multimap

Przykładowy kontener: `std::array`:

- wyraża tablicę o **stałej ilości** elementów, ustalonej w czasie kompilacji przez argument **templatki**,
- dostęp **sekwencyjny** do elementów – pewien ustalony porządek,
- **statyczny przydział pamięci** – nie mogą zmienić rozmiaru,
- mogą być traktowane jako **krotki**,
- swap dwóch tablic wymaga ich przepisania w czasie liniowym – za to iteratory odnoszą się ciągle do tych samych adresów

67 Obsługa sytuacji wyjątkowych w C++.

- Wyjątek = obiekt sytuacji wyjątkowej.
- Wyjątki pozwalają zareagować na **nieoczekiwaną sytuację**, w której istnieje **ryzyko niewykonania** określonego zadania.
- Dawniej polegało to na zwróceniu specyficznej wartości, co wymuszało typ wyjątku (jako typ zwracany przez funkcję).
- W C++ wyjątki się "**rzuca**", służy do tego instrukcja **throw**. Rzucić możemy **błąd** (exception, error), ale też **typy** proste (int, char, string, ...) i **obiekty** klas.
- Tam gdzie spodziewamy się wyjątków umieszczamy bloki **try-catch**. Catche rozpatrywane są od góry.

68 Obsługa plików w języku C.

Standardowa biblioteka wejścia/wyjścia języka C udostępnia funkcje do operowania na plikach, tj. odczytu i zapisu danych z plików tekstowych i binarnych (stdio.h, stdlib.h).

1. Otwarcie pliku o określonej nazwie w określonym trybie.

```
FILE *fopen(const char *filename, const char *mode);
```

Dostępne tryby (możliwość łączenia):

- "r" - czytanie, "+" dodaje opcję nadpisywania,
- "w" - nadpisywanie, "+" dodaje odczyt,
- "a" - dopisywanie (ew. tworzenie), "+" dodaje odczyt,
- "t" - tryb tekstowy,
- "b" - tryb binarny.

Funkcja zwraca wskaźnik do pliku (FILE *) lub NULL, gdy pliku nie udało się otworzyć (nie istnieje, jest już otwarty w innym programie itp.). **Po zakończeniu pracy należy plik zamknąć** – funkcja `fclose(file)`.

2. Wykonywanie operacji odczytu/zapisu z możliwością formatowania.


```
int fscanf(FILE *stream, const char *format, ...);
int fprintf(FILE *stream, const char *format, ...);
```

Funkcje działają tak jak scanf i printf, przy czym jako pierwszy argument przyjmują wskaźnik do pliku (strumienia danych).

69 Model wodospadu a model spiralny wytwarzania oprogramowania.

69.1 Model kaskadowy

Zasady

- Tworzenie systemu **sekwencyjnie, w wyizolowanych etapach**;
- Wyjścia z jednego etapu są wykorzystywane jako wejścia do etapu następnego;
- Każdy etap podzielony jest na **dwie części: twórczą i weryfikacji**;
- Ponowna praca, jeśli jest konieczna, jest wykonywana w kolejnych etapach – bardzo **wysoki koszt błędów** popełnionych **we wstępnych etapach**, adaptowanie zmian bardzo kosztowne.
- Koszty opracowania i akceptacji dokumentów są wysokie, powinien być używany tylko jeśli wymagania są jasne i zrozumiałe; jakość silnie związana z ich stabilnością.
- **Marginalizacja roli klienta** w procesie wytwarzania oprogramowania. Uzyskanie produktu zgodnego z wymaganiami silnie zależne od ich stabilności.

Fazy modelu kaskadowego

1. **Planowanie**: cele biznesowe, podstawowe wymagania, założenia, standardy, parametry,
2. **Analiza**: zdefiniowanie przeznaczenia systemu → kompletny, spójny i jednoznaczny model systemu,
3. **Projekt**: dekompozycja na podsystemy, wybór strategii budowania, projektowanie obiektów, wybór komponentów, opis interfejsów,
4. **Implementacja**: tworzenie kodu źródłowego, mapowanie modeli na kod,
5. **Testowanie**: znajdowanie różnic między rzeczywistym elementem a jego modelem,
6. **Pielęgnacja**.

69.2 Model spiralny

- proces ma postać spirali, której każda pętla reprezentuje jedną jego fazę;
- najbardziej wewnętrzna pętla przedstawia początkowe etapy;
- ciągle monitorowanie i pomiar zmian;
- zmiany poddawane są review użytkownika;
- próba minimalizacji ryzyka niepowodzenia;

69.2.1 Fazy modelu spiralnego

Każda pętla spirali podzielona jest na cztery sektory

1. **Ustalanie celów** — definiowanie konkretnych celów wymaganych w tej fazie przedsięwzięcia; identyfikacja ograniczeń i zagrożeń; ustalanie planów realizacji.
2. **Rozpoznanie i redukcja zagrożeń** — przeprowadzenie szczegółowej analizy rozpoznanych zagrożeń, ich źródeł i sposobów zapobiegania; podjęcie kroków zapobiegawczych.
3. **Tworzenie i zatwierdzanie** — tworzenie oprogramowania w oparciu o najbardziej odpowiedni model, wybrany na podstawie oceny zagrożeń.
4. **Ocena i planowanie** — recenzja postępu prac i planowanie kolejnej fazy przedsięwzięcia bądź zakończenie procesu produkcyjnego.

	Model kaskadowy	Model spiralny
sposób działania	sekwencyjny	ewolucyjny
błędy	wykrywane i naprawiane po zakończeniu etapów	wykrywane wcześniej
rozmiar projektu	mniejsze	większe
wymagania	określone na początku	mogą być dodawane po iteracjach
zmiany w czasie projektu	ograniczona możliwość zmian	większa możliwość zmian
koszt	mały	wysoki

70 Diagram sekwencji i diagram przypadków użycia w języku UML.

UML - Unified Modelling Language.

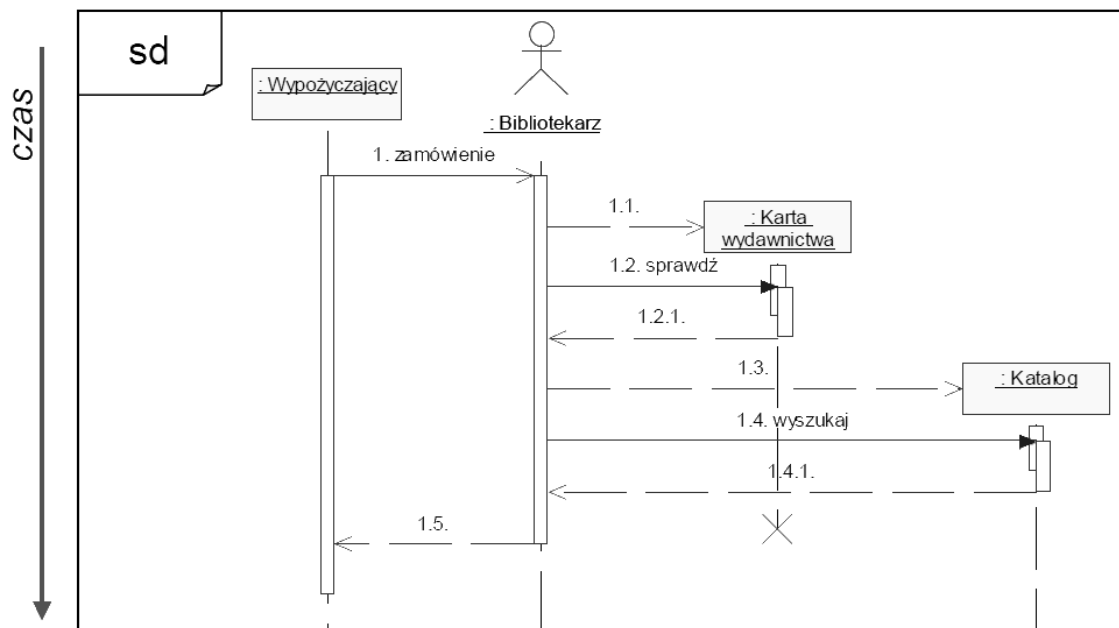
- rodzina notacji graficznych;
- służy do **opisywania i projektowania** systemów informatycznych;
- narodził się z unifikacji wielu obiektowych języków modelowania graficznego;
- nadzorowany przez organizację OMG.

70.1 Diagram sekwencji.

Diagram sekwencji opisuje **interakcje pomiędzy częściami systemu** w postaci sekwencji komunikatów wymienianych między nimi.

Diagramy sekwencji dobrze ukazują **dynamiczne aspekty realizacji scenariuszy**, w których dochodzi do złożonych oddziaływań pomiędzy obiektami.

- **strzałka zwykła** – przesłanie komunikatu asynchronicznego,
- **strzałka z wypełnionym grotem** – wywołanie funkcji,
- **strzałka rysowana linią przerywaną** – powrót z funkcji.



70.2 Diagram przypadków użycia.

Diagram przypadków użycia służy do przedstawiania:

- **interakcji aktora z systemem,**
- **zadań,** jakie wykonuje system,
- **wymagań funkcjonalnych.**

Aktorami mogą być ludzie, systemy zewnętrzne bądź części systemu, które mają wpływ na jego funkcjonowanie, ale są przezeń niezmiennalne (np. zegar systemowy).



71 Klasyfikacja testów.

71.1 Poziomy testów.

Definicja 71.1 *Poziom testów* określa **sposób** testowania ze względu na **postać** testowanego obiektu w kontekście cyklu życia (**co testujemy?**).

TESTY JEDNOSTKOWE	
Podstawa testów	wymagania na moduły, projekt szczegółowy, kod
Typowe obiekty	moduły, programy, funkcje, klasy, procedury

- testowanie modułowe, **unit testing** – przeprowadzane przez **deweloperów**
- defekty szukane w izolacji od reszty systemu
- brak formalnego procesu zarządzania defektami; defekty **usuwane bezpośrednio po znalezieniu**
- wykorzystanie zaślepek i sterowników
- **TTD** – Test Driven Development – napisanie testu, uruchomienie testu, napisanie kodu.

TESTY INTEGRACYJNE	
Podstawa testów	projekt systemu, architektura, przypadki użycia
Typowe obiekty	interfejsy, podsystemy, konfiguracje systemów

- testuje **interfejsy i interakcje**, wymaga **rozumienia architektury**
- może odbywać się na wielu poziomach (np. integracja systemów); im większy zakres integracji, tym trudniej izolować defekty,
- **strategie** testów integracyjnych:
 - **top-down** – najpierw moduł wywołujący, potem wywoływane,
 - **bottom-up** – najpierw moduły wywoływany, potem wywołujący,
 - **funkcjonalne** – top-down wewnątrz danej funkcjonalności,
 - **sekwencja** przeprowadzania **transakcji**,
 - **big-bang** – wszystko integrowane i testowane naraz.

TESTY SYSTEMOWE	
Podstawa testów	wymagania na system, przypadki użycia, specyfikacja funkcjonalna, raporty analizy ryzyka
Typowe obiekty	system, podręczniki użytkownika i operatora, konfiguracja systemu

- sprawdza zachowanie **systemu jako całości** w środowisku testowym podobnym do **produkcyjnego**, zwykle przez **niezależny zespół testerski**

- zakres testu określony w **planie testów**
- często konieczność testowania z **niekompletnymi wymaganiami**

TESTY AKCEPTACYJNE	
Podstawa testów	wymagania użytkownika, wymagania na system, przypadki użycia, procesy biznesowe, raporty analizy ryzyka
Typowe obiekty	Procesy biznesowe w pełni zintegrowanego systemu, procesy operacyjne i utrzymania systemu, procedury, raporty, dane konfiguracyjne

- cel: **uzyskanie zaufania** do systemu; często przeprowadzane przez klienta lub **użytkownika**,
- znajdowanie defektów nie jest głównym celem
- **ocenia gotowość** systemu, ale niekoniecznie ostatni etap testów
- **typowe formy testów akceptacyjnych:**
 - **testy akceptacyjne użytkownika (UAT)** – sprawdzenie gotowości do użycia
 - **testy operacyjne (OAT)** – akceptacja przez administratora systemu (testy backupu, przywracania systemu, zarządzania użytkownikami itp.)
 - **testy akceptacyjne wymagane kontraktem/regulacjami**
 - **testy alfa, beta (polowe)**
 - * **alfa:** przeprowadzane **u producenta**, ale nie przez zespół
 - * **beta:** przeprowadzane **u klienta**, przez użytkownika

71.2 Typy testów.

Definicja 71.2 *Typ testów to zbiór czynności testowych właściwych dla weryfikacji systemu w oparciu o konkretny powód lub cel testów (**jak testujemy?**).*

1. **Testowanie funkcjonalne** – testowanie funkcji – **co system robi.**
2. **Testowanie niefunkcjonalne** – wyrażalne ilościowo testowanie niefunkcjonalnej charakterystyki jakościowej (np. niezawodność, wydajność, użyteczność) – **jak system działa.**

3. **Testowanie strukturalne** – oparte na strukturze (np. kod, graf przepływu sterowania, model procesu biznesowego). Zwykle wykonywane **po testach czarnoskrzynkowych**, aby ustalić **pokrycie**.
4. **Retesty i testy regresji** – **związane ze zmianą**, tzn. potwierdzenie usunięcia defektów (retesty - testy zmienionych fragmentów kodu) oraz poszukiwanie niezamierzonych zmian (regresyjne - testy niezmienionego kodu - często automatyzowane).

72 Model Scrum: struktura zespołu, proces wytwarzania oprogramowania, korzyści modelu.

SCRUM jest: lekki, łatwy do zrozumienia, bardzo trudny do opanowania.

Trzy filary SCRUMu:

- **Adaptacja** - powinna być ciągła. Korekta musi być wykonana jak najszybciej, by ograniczyć dalsze następstwa problemów.
- **Przejrzystość** - wszystkie istotne aspekty procesu muszą być widoczne dla osób odpowiedzialnych za osiągnięte rezultaty.
- **Inspekcja** – poddawane regularnej inspekcji są zarówno scrumowe artefakty jak i postępy prac.

72.1 Role

:

- **Właściciel Produktu**
 - **odpowiedzialny** za maksymalizację wartości produktu i **pracy** Zespołu,
 - jedyną osobą **zarządzającą Rejestrem** Produktu, tzn:
 - * jasne artykułowanie **elementów**, określanie ich **kolejności**;
 - * zapewnianie dostępności i przejrzystości dla wszystkich; opisywanie czym Zespół Scrumowy będzie się zajmował.
- **Zespół deweloperski**
 - złożony z równych **profesjonalistów** – "Deweloperów" (3-9 os); **samoorganizujący** się, wielofunkcyjny.

- ma za zadanie dostarczenie (na zakończenie każdego Sprintu), gotowego do wydania **Przyrostu** produktu,
- odpowiedzialność za wykonywaną pracę ponosi cały Zespół; **brak podziału na podzespoły**.
- **Scrum Master** – odpowiedzialny za to, by Scrum był rozumiany i stosowany, zarówno przez Właściciela, jak i Zespół.

72.2 Zdarzenia

- **Sprint** – stałe przez okres trwania prac **ograniczenie czasowe** (≤ 1 mies)
 - podczas Sprintu wytwarzany jest Przyrost funkcjonalności,
 - niedozwolone są zmiany, które wpłyną na cel Sprintu,
 - niezmienny skład Zespołu Deweloperskiego i jego cel jakościowy.
- **Przerwanie Sprintu** – przez Właściciela w przypadku dezaktualizacji celu Sprintu; zużywa zasoby, bo powoduje przegrupowanie podczas kolejnego Planowania.
- **Planowanie Sprintu** - 8h/mies
 - Część pierwsza: **Co będzie zrobione w tym Sprincie?**
Na podstawie RP, ostatniego przyrostu i odczytów wydajności, oraz przewidywalnej pojemności ZD wyznacza się cel Sprintu i elementy RP do zrobienia.
 - Część druga: **Jak wybrana praca będzie wykonana?**
Stworzenie projektu systemu i planu prac; ZD powinien móc wytłumaczyć Właścicielowi i Scrum Masterowi, w jaki sposób ma zamiar pracować.
- **Codzienny Scrum** - 15 min/d – co zrobiłem od wczoraj, co chcę zrobić do jutra, co mi przeszkadzało.
- **Przegląd Sprintu** - 4h na zakończenie Sprintu
 - WP stwierdza, które funkcjonalności zostały „Ukończone”, a które nie;
 - ZD omawia, co poszło dobrze, jakie były problemy i jak je rozwiązano;
 - ZD prezentuje „Ukończoną” pracę i odpowiada na pytania;
 - WP omawia RP w aktualnej jego postaci. Przewiduje termin zakończenia prac.
 - Cała grupa omawia kolejne kroki.

- **Retrospektywa Sprintu** - inspekcja działań i planowanie usprawnień.
 - co się działo, biorąc pod uwagę ludzi, zależności, procesy i narzędzia;
 - Zidentyfikowanie i uporządkowanie istotnych elementów dobrych i złych,
 - Stworzenie planu wprowadzania w życie usprawnień pracy Zespołu.

72.3 Artefakty

- **Rejestr Produktu**
 - uporządkowana lista wszystkiego, co może być potrzebne w produkcji;
 - jedyne źródło wymaganych zmian;
 - elementy posiadają atrybuty: opis, kolejność i oszacowanie (estymację);
 - odpowiedzialny za RP jest Właściciel Produktu.
- **Rejestr Sprintu**
 - podzbiór elementów RP wybranych do Sprintu rozszerzony o plan dostarczenia Przyrostu produktu.
 - RS jest obrazem pracy, jaką ZD planuje wykonać w trakcie Sprintu.
 - RS należy tylko i wyłącznie do ZD.
- **Monitorowanie postępów Sprintu** – możliwe w każdym momencie Sprintu (Codzienny Scrum).
- **Przyrost** – suma wszystkich elementów RP zakończonych podczas wszystkich Sprintów. Na koniec Sprintu nowy Przyrost musi być „Ukończony”.
- **Definicja Ukończenia** – aby zapewnić przejrzystość, ustala się wspólne dla zespołu pojmowanie, co to znaczy, że praca jest skończona. W miarę dojrzewania Zespołu definicja zawiera coraz bardziej rygorystyczne kryteria celem zapewnienia wyższej jakości.

73 Wymagania w projekcie informatycznym: klasyfikacja, źródła, specyfikacja, analiza.

Wymagania - opis **funkcji** (usług), które mają być **realizowane przez system** i opis **ograniczeń** dla systemu. Wymagania nie opisują jak system ma działać a **co ma wykonywać**.

Klasyfikacja wymagań

- **Funkcjonalne** - opisują jakie funkcje powinien mieć system, np. “wprowadzanie nowej faktury” lub “generowanie raportu miesięcznego”
- **Pozafunkcjonalne** - opisują najczęściej pewne własności danych funkcjonalności, np. “minimum 20 faktur na godzinę”.

Źródła i specyfikacja.

- **System powinien...** – sposób spisywania wymagań w stylu: “System powinien umożliwić wystawianie faktur”.
 - Łatwość spisywania
 - Słaba czytelność
 - Trudne sprawdzanie kompletności, spójności
 - Obecnie nie używany
- **Funkcje systemu** – opisywanie poszczególnych funkcji systemu podobnie do funkcji matematycznych.
 - Słaba czytelność
 - Trudne do zrozumienia
 - Obecnie nie używany
- **Przypadki użycia** – scenariusze główne i poboczne interakcji użytkownika z systemem.
 - Łatwość spisywania
 - Czytelność
 - Łatwość zrozumienia i wyobrażenia sobie przyszłego systemu
- **Historyjki użytkownika** – **Who? What? Why?**.

Analiza

- **Celem** analizy wymagań jest stworzenie modelu systemu, zwanego modelem analitycznym – ustrukturalnianie i **formalizowanie wymagań**, tworzenie **UMLi**.
- **Model analityczny** - reprezentuje tworzony system z **perspektywy użytkownika**; opis co system powinien robić.

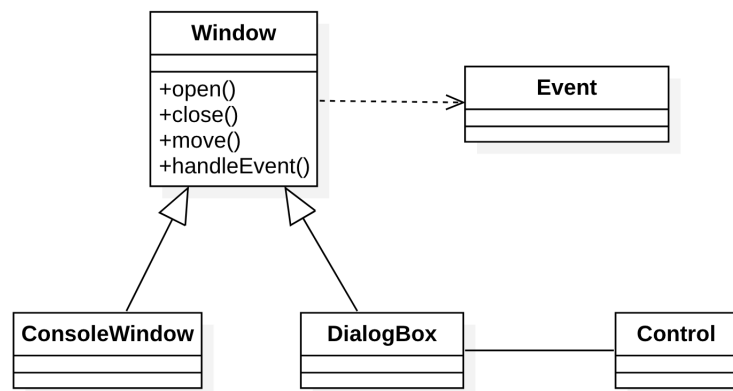
- **Model dynamiczny** (diagramy sekwencji i stanów) - koncentruje się na **zachowaniu** systemu.
- **Analityczny model obiektowy** (diagramy klas) - odzwierciedla **indywidualne koncepcje** korzystania z systemu, ich właściwości i relacje między nimi

74 Analiza obiektowa: modele obiektowe i dynamiczne, obiekty encyjne, brzegowe i sterujące.

74.1 Analityczny model obiektowy.

Diagram klas ma za zadanie przedstawić:

- strukturę systemu w modelach obiektowych poprzez ilustrację struktury klas i zależności między nimi,
- podział odpowiedzialności pomiędzy klasy i rodzaj wymienianych przez nie komunikatów.



Klasy składają się z **nazwy**, **atrybutów**, **operacji** i opcjonalnie **odpowiedzialności**.

Widoczność pól

+ public
 - private
 # protected
 ~ package

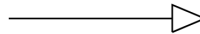
- **Asocjacja**



- **Zależność** – klasa używa innej klasy. Przykład: EventHandler i Event.



- **Generalizacja** – dziedziczenie lub uogólnienie; zależność między klasą pochodną a klasą bazową.



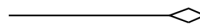
- **Realizacja** – zależność między implementacją a interfejsem. Przykład: ArrayList i List.



- **Agregacja** – zależność między częścią a całością. Przykład: cegła i mur.



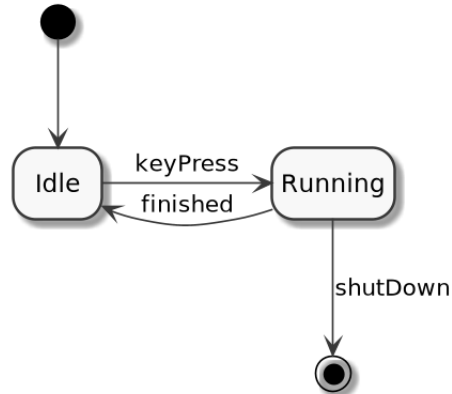
- **Kompozycja** (silna agregacja) – zależność między częścią a całością, jednak część nie może istnieć bez całości. Przykład: wydział i uczelnia.



74.2 Modele dynamiczne

- **Diagram sekwencji** – interakcje pomiędzy częściami systemu w postaci sekwencji komunikatów wymienianych między nimi; dynamiczne aspekty realizacji scenariuszy, rozdział ??.
- **Diagram stanów** – graficzna reprezentacja stanów, przejść, zdarzeń i akcji maszyny stanów modelującej dynamiczne aspekty systemu.

Diagram stanów



- **Stan** – sytuacja lub stan obiektu, w którym przez pewien czas spełnia on pewien warunek, wykonuje jakąś czynność lub czeka na jakieś zdarzenie. Wy-
różniamy
 - Podstany – wewnętrzne podzielenie stanu.
 - Stan początkowy – domyślne miejsce początkowe dla danego stanu lub automatu.
 - Stan końcowy – miejsce zakończenia wykonania automatu stanów.
- **Przejście** – związek między dwoma stanami wskazujący, że obiekt w pierw-
szym stanie wykona określone działania i przejdzie w drugi stan, gdy wystąpi
określone zdarzenie i spełnione zostaną określone warunki.
 - Zdarzenie – wystąpienie bodźca, który może wywołać zmianę stanu.
 - Akcja – wykonywalne obliczenie atomowe (np. isFinished).

74.3 Encja-Brzeg-Sterowanie

Trzy typy obiektów:

- **Obiekty brzegowe** – odzwierciedlają interakcje między aktorami a systemem,
- **Obiekty sterujące** – odpowiadają za realizację przypadków użycia,
- **Obiekty encji** – reprezentujące trwałą informację przetwarzaną przez system.

Np. przycisk (brzegowy) → funkcjonalność zmiany daty (sterujący) → zmiana dnia, miesiąca, roku (encje).

75 Wzorce architektury systemów.

75.1 Architektura warstwowa

Efektem dekompozycji hierarchicznej jest **uporządkowany zbiór warstw**. Warstwa stanowi **zgrupowanie podsystemów** oferujących powiązane usługi.

Zalety:

- modyfikowalność
- prostota wykorzystania warstwy
- spójność i przejrzystość
- wielokrotne wykorzystanie
- naturalna ewolucja z tradycyjnego modelu aplikacji

Dwa rodzaje architektury warstwowej:

- **Architektura otwarta** – warstwa może wywołać dowolną z warstw poniżej niej. Przykłady: Serverless, Swing.
- **Architektura zamknięta** – warstwa może wywołać tylko warstwę bezpośrednio pod nią. Przykłady: ISO/OSI, TCP/IP.

Architektura trójwarstwowa – architektura typu klient-serwer, często stosowana przy tworzeniu aplikacji internetowych.

1. Warstwa **prezentacji** – interfejs użytkownika, tłumaczy żądania i wyniki.
2. Warstwa **logiki biznesowej** – koordynuje pracę aplikacji, przetwarza żądania warstwy prezentacji, dokonuje obliczeń.
3. Warstwa **danych** – mechanizmy persystencji danych oraz dostępu do nich.

75.2 Model View Controller

Wzorzec architektoniczny, dzielący system na trzy główne części:

1. **Model** reprezentuje podstawową część funkcjonalności systemu,
2. **Widok** wyświetla dane dostarczone przez model użytkownikowi,
3. **Kontroler** przyjmuje dane wejściowe od użytkownika oraz przetwarza jego żądania.

75.3 Pipeline (filtry i potoki)

- Wzorzec Pipeline pozwala na **uporządkowanie systemu**, która przetwarza **strumienie danych**.
- Każdy krok przetwarzania jest zamknięty w **filtrze**.
- Dane przesyłane są pomiędzy elementami za pomocą **potoków** (pipes).
- Wyjście poprzedniego filtra przez potok jest przesyłane jako wejście następnego.
- Np. pipelnie w Unixie (`|`).

75.4 Client - Server

- Podział systemu na **dostawcę** usług (server) oraz ich **odbiorców** (clients).
- Serwer oczekuje na żądania (request) klientów, przetwarza je, a następnie wysyła odpowiedzi (response).
- Serwer **pasywny**, klient **aktywny**.
- Przykłady: serwer WWW i przeglądarka internetowa, system bankowy.

75.5 Peer-to-Peer

- **Każdy** z podszytemów (hostów, peerów) może pełnić zarówno **funkcję serwera**, jak i **klienta**.
- Architektura systemu jest rozproszona, każdy z hostów ma **te same uprawnienia**.
- Przykłady: torrenty, niektóre wideochaty.

Inżynieria systemów

76 Relacyjny model danych, normalizacja relacji (w szczególności algorytm doprowadzenia relacji do postaci Boyce’a-Codda), przykłady.

76.1 Relacyjny model danych

- Dane w **tabelach**. Tabela może przedstawiać **relację bazodanową**.
- **Wiersz (rekord)** - reprezentuje pewną **encję lub związek**; powinien być **unikalny** wewnątrz tabeli.
- **Atrybuty - kolumny**. Nazwane, mają określony, logicznie niepodzielny typ.
- **Skalar** - pojedyncza wartość, najmniejsza semantycznie jednostka danych; **atomowy w ramach modelu** relacyjnego.
- **Zmienna relacji** - nazwany obiekt, którego wartość zmienia się w czasie. **Wartość zmiennej** w danej chwili nazywa się **wartością relacji**.
- Relacyjna baza to **zestaw znormalizowanych relacji różnych stopni**.
- **Klucz główny** – minimalna kombinacja pól identyfikująca każdy rekord w tabeli w sposób jednoznaczny.

76.2 Relacja bazodanowa

Wg Date’a: Relacja R (w znaczeniu **wartość relacji**) na zbiorze niekoniecznie różnych dziedzin $D_1, D_2, \dots, D_n$ składa się z dwóch części: **nagłówka** (heading) i **treści** (body).

Nagłówek jest to ustalony **zbiór atrybutów**, tj. zbiór par $\langle nazwa_atrybutu : nazwa_dziedziny \rangle$, dziedzina to zbiór wartości skalarych. Atrybuty **nie są uporządkowane**.

Treść jest to **zbiór unikalnych, nieuporządkowanych krotek**, tj. zbiorów par $\langle nazwa_atrybutu : wartosc_atrybutu \rangle$ zawierających wszystkie atrybuty z nagłówka. **Posiada przynajmniej jeden klucz kandydujący**.

Liczebność relacji - liczba krotek w zbiorze.

Stopień relacji - ilość dziedzin.

Rodzaje relacji:

- **Relacja nazwana** jest **nazwaną zmienną relacji** zdefiniowana w SZBD.
- **Relacja podstawowa** (autonomiczna) - wystarczająco ważna, by stanowić **nazwany niezależny byt** w bazie danych. "Przechowuje dane".
- **Relacja wyprowadzana** (pochodna) - relacja, którą można otrzymać ze zbioru nazwanych relacji za pomocą jakiegoś **wyrażenia relacyjnego**.
- **Perspektywa** (widok) - nazwana relacja pochodna.
- **Migawka** (snapshot) - rzeczywista nazwana relacja pochodna.
- **Wynik zapytania** - powstaje w wyniku realizacji zapytania.
- **Wynik pośredni** - powstaje w wyniku realizacji podzapytania.
- **Typy**: jeden do jednego, wiele do jednego, wiele do wielu.

76.3 Postaci normalne

Pierwsza postać normalna 1NF

- Krotki unikalne, nieuporządkowane, istnieje klucz kandydujący. Atrybuty atomowe, wartości skalarne
- *Normalizacja*: wyeliminowania powtarzających się grup – rozbiecie na wartości skalarne.
- *Przykład*: atrybut "imiona" wszystkie imiona danej osoby $\rightarrow$ osobne pola na każde imię

Druga postać normalna 2NF

- Żaden niekluczowy (wtórny) atrybut nie jest zależny funkcyjnie od żadnego podzbioru właściwego klucza kandydującego.
- Każdy atrybut wtórny jest w pełni zależny funkcyjnie od klucza kandydującego.
- *Normalizacja*: wyeliminowanie danych niezależnych od klucza.
- *Przykład*: $\{\text{Imię, nazwisko, płeć}\} \rightarrow \{\text{Imię, nazwisko}\} \{\text{Imię, płeć}\}$.

Trzecia postać normalna 3NF

- **Żaden atrybut niekluczowy nie jest zależny funkcyjnie od innego atrybutu niekluczowego.**
- Dla każdej nietrywialnej zależności funkcyjnej $\{A_1, \dots, A_n\} \rightarrow \{B\}$ zbiór atrybutów $\{A_1, \dots, A_n\}$ jest nadkluczem lub atrybut B jest elementem pewnego klucza kandydującego.
- *Normalizacja*: wyeliminowanie danych niezależnych od klucza:

1. Szukamy wszystkich nietrywialnych, całkowitych zależności funkcyjnych $\{A_1, \dots, A_n\} \rightarrow \{B_i\}$, które naruszają warunek trzeciej postaci normalnej.
 2. Załóżmy, że otrzymujemy zależność $(*) \{A_1, \dots, A_n\} \rightarrow \{B_1, \dots, B_m\}$.
Dzielimy schemat relacji na dwa nierozłączne podzbiory:
 - Pierwszy, zawierający wszystkie atrybuty występujące w zależności $(*)$.
 - Drugi, zawierający atrybuty z lewej strony rozważanej zależności $(*)$ oraz atrybuty nie występujące ani z lewej ani z prawej strony tej zależności.
- *Przykład:* $\{\text{Imię, Nazwisko, Stanowisko, Stawka}\} \rightarrow \{\text{Imię, Nazwisko, Stanowisko}\}, \{\text{Stanowisko, Stawka}\}$

Postać normalna Boyce'a-Codda

- Dla każdej całkowitej nietrywialnej zależności funkcyjnej $A \rightarrow B$ zbiór **A jest nadkluczem**.
- *Normalizacja:* jak dekompozycja do 3PN. **Nie zawsze zachowuje zależności funkcyjne.**

Czwarta postać normalna 4NF

- Dla każdej nietrywialnej zależności wielowartościowej $X \twoheadrightarrow Y$ **zbiór X zawiera klucz relacji**.
- *Normalizacja:* analogicznie jak dla 3PN.

77 Indeksowanie w bazach danych: drzewa B+, tablice o organizacji indeksowej, indeksy haszowe, mapy binarne.

Indeks – struktura danych na dysku umożliwiająca szybkie wyszukiwanie danych w bazie danych na podstawie wartości klucza.

- **Zaleta** Zmniejszenie czasu odszukania potrzebnych danych (minimalizowanie liczby odczytów bloków danych z dysku)
- **Wada:** Zwiększenie czasu zmiany danych, na których założony jest indeks (zmiana takich danych oznacza zmianę w indeksie)

Implementacje indeksów: drzewa B+, tablice haszujące, plik uporządkowany z wyszukiwaniem binarnym.

Typy indeksów (dla plików sekwencyjnych)

- **Główne** (Primary) – na identyfikatorach wierszy,
- **Drugorzędne** (Secondary) – pomocnicze.
- **Gęste** (Dense) – każdy wiersz danych ma swój wpis w indeksie,
- **Rzadkie** (Sparse) – każdy blok danych ma jeden wpis w indeksie. Dane muszą być posortowane według klucza indeksu.
- **Grupujące** (clustering) – ten termin jest wieloznaczny. W odniesieniu do plików sekwencyjnych oznacza grupowanie w jednym bloku **wierszy z taką samą wartością klucza indeksu**; taki indeks budowany jest **na polu niekluczowym** i wymaga **posortowania wierszy według pola indeksowanego**.
- **Wielopoziomowe**.

78 Podstawowe cechy transakcji (ACID). Metody sterowania współbieżnością transakcji, poziomy izolacji transakcji, przykłady.

78.1 Transakcja

Transakcja jest wykonywanym programem, który tworzy logiczną jednostkę przetwarzania w bazie danych.

ACID

- **Atomicity** (niepodzielność) – transakcja wykonywana **w całości albo wcale**.
- **Consistency** (spójność) po zakończeniu transakcji baza musi być w stanie spójnym, tzn. muszą być **zachowane wszystkie więzy** narzucone na dane.
- **Isolation** (izolacja) wykonywana w **izolacji od innych** transakcji.
- **Durability** (trwałość) po zakończeniu transakcji jej **efekty** muszą być **trwałe** w systemie nawet w przypadku awarii.

Stany transakcji

- **Aktywna** – podczas wykonywania operacji.

- **Częściowo zatwierdzona** – ostatnia operacja transakcji została wykonana. Teraz protokół zatwierdzania musi zapewnić **trwałość zmian**. Ew. sprawdzenie czy może zostać zatwierdzona. Jeśli może, to osiągnęła swój **punkt zatwierdzenia**.
- **Nieudana** – wykonywanie transakcji **nie może być kontynuowane**.
- **Przerwana, wycofana** – baza jest **odtworzona** do stanu sprzed rozpoczęcia transakcji.
- **Zatwierdzona** – wszelkie **zmiany** wykonane przez transakcje muszą być **trwałe**. Zatwierdzonej transakcji **nie można już wycofać** (ew. **transakcja kompensująca**).

Implementacja atomowości i trwałości – strategie zarządców buforów:

- **Fix** - nie może być zapisów przed końcem transakcji.
- **No-Fix** - mogą być zapisy przed końcem transakcji. Synchronizacja może być realizowana **cyklicznie** w ramach procesu zwanego **punktem kontrolnym**.
- **Flush** - synchronizacja zmienionych bloków po zakończeniu transakcji.
- **No-Flush** - brak obowiązku synchronizowania zmienionych bloków na koniec transakcji. Synchronizacja może być wykonana później.

W większości systemów wykorzystujących dzienniki transakcji stosowana jest **strategia No-Fix/No-Flush**.

78.2 Techniki sterowania współbieżnością transakcji

Blokady na dane:

- **Blokady wyłączne** (typu X, exclusive, blokady do zapisu)
- **Blokady wspólne** (typu S, shared, blokady do odczytu)

Blokady na poziomie tabeli:

- **IS (intent shared)** – wspólna blokada zapobiegająca – transakcja zamierza założyć blokadę S na poszczególnych wierszach tabeli (ogólnie: węzłach potomnych)
- **IX (intent exclusive)** – wyłączna blokada zapobiegająca – zamiar założenia blokady X na wierszach (ogólnie: węzłach potomnych)
- **SIX (shared intent exclusive)** – wspólna wyłączna blokada zapobiegająca – na tabeli (węzle macierzystym) jest już blokada S, zamierza się założyć blokadę X na wiersze (połączenie S oraz IX).

Poziom izolacji	P0	P1	P2	P3	Blokady X	Blokady S
Locking READ UNCOMMITTED	NIE	TAK	TAK	TAK	długie	nie
Locking READ COMMITED	NIE	NIE	TAK	TAK	długie	krótkie
Locking REPEATABLE READ	NIE	NIE	NIE	TAK	długie	długie
Locking SERIAZABLE	NIE	NIE	NIE	NIE	długie	długie predykatowe

A - oryginalna definicja, P - rozszerzona.

Dirty Write:

P0: $w1[x] \dots w2[x] \dots ((c1 \text{ lub } a1) \text{ i } (c2 \text{ lub } a2))$ w dowolnej kolejności

Dirty Read:

P1: $w1[x] \dots r2[x] \dots ((c1 \text{ lub } a1) \text{ i } (c2 \text{ lub } a2))$ w dowolnej kolejności

A1: $w1[x] \dots r2[x] \dots (a1 \text{ i } c2)$ w dowolnej kolejności

Non-rep or Fuzzy Read:

P2: $r1[x] \dots w2[x] \dots ((c1 \text{ lub } a1) \text{ i } (c2 \text{ lub } a2))$ w dowolnej kolejności

A2: $r1[x] \dots w2[x] \dots c2 \dots r1[x] \dots c1$

Phantom:

P3: $r1[P] \dots w2[y \text{ in } P] \dots ((c1 \text{ lub } a1) \text{ i } (c2 \text{ lub } a2))$ w dowolnej kolejności

A3: $r1[P] \dots w2[y \text{ in } P] \dots c2 \dots r1[P] \dots c1$

Lost update:

P4: $r1[x] \dots w2[x] \dots c2 \dots w1[x] \dots c1$

78.3 Wielowersyjne techniki sterowania współbieżnością

Poziom izolacji Cursor Stability – rozszerzenie sposobu blokowania w poziomie Locking READ COMMITED.

- operacja **rc (czytaj kursor, pobierz wiersz)** dla instrukcji FETCH, blokada (S lub nowy typ blokady do odczytu **scroll lock**) będzie utrzymywana **do chwili przejścia** do innego wiersza lub do **zamknięcia** kursora.
- operacja **wc (write cursor)** powoduje założenie na ten wiersz **długiej blokady X**.

Nowa odmiana problemu P4: **P4C: rc1[x]...w2[x]...c2...wc1[x]...c1**

Poziom izolacji Cursor Stability **eliminuje P4C**, w2[x] będzie wstrzymane do zdjecia blokady (S, scroll lock) przez przejście do innego wiersza lub zamknięcie kursora.

Poziom izolacji Snapshot – transakcja czyta dane (zatwierdzone) z chwili swojego początku, Start-Timestamp.

Protokoły odczytu spójnych wersji czasowych.

- **Snapshot isolation v1** – wszelkie zmiany są wykonywane na lokalnych kopiach i zapisywane na końcu transakcji. Aktualizacje wykonywane przez inne transakcje nie są odczytywane.
- **Snapshot isolation v2** – blokady do zapisu, ponadto przy każdym zapisie transakcja wykonuje sprawdzenie czy ma aktualne dane.
- **Read Committed Snapshot (Read Consistency)** – operacja odczytu czyta ostatnią zatwierdzoną (aktualną) wartość elementu danych.

79 Złączenia, grupowanie, podzapytania w języku SQL.

JOIN – łączenie tabel między kluczem podstawowym i kluczem obcym drugiej.

- Łączenia **wewnętrzne** – **INNER JOIN** – wybiera rekordy posiadające takie same wartości w obu tabelach.
- Łączenia **zewnętrzne** – **LEFT/RIGHT/FULL OUTER JOIN** – zwraca $L \cap R / R \cap L / L \cup R$.

GROUP BY – tworzenie grup rekordów w oparciu o definicję grupowania, wykonywane po filtrowaniu.

W przypadku stosowania **GROUP BY**, w trakcie przetwarzania zapytania, tworzone są dwie sekcje danych.

- **Sekcja grupująca** składa się z atrybutów tworzących definicję grupowania.
- **Sekcja danych surowych** (raw data section) zawierają wszystkie pozostałe dane.

Podzapytania – instrukcje SELECT wewnątrz innej instrukcji SELECT. Serwer bazodanowy w pierwszej kolejności wykona zapytania wewnętrzne. Wynik zapytania wewnętrznego zwracany jest do zapytania zewnętrznego.

- podzapytania **niepowiązane** (nieskorelowane) – wykonywane tylko raz, zwraca jeden wynik; może być wykonane samodzielnie; często używane w WHERE.
- podzapytania **powiązane** (skorelowane) – bezpośrednio powiązane z zapytaniem nadrzędnym poprzez przekazywane przezeń atrybuty; wykonywane osobno dla każdego wiersza; używane do wyliczenia nowego atrybutu.

80 Szeregowalność harmonogramów w bazach danych.

Harmonogram - inaczej historia S zbioru n transakcji $T_1, \dots, T_n$ jest takim ciągiem wszystkich operacji transakcji, że dla każdej transakcji $T_i \in S$ operacje tej transakcji w S muszą występować w takiej samej kolejności, w jakiej występują w T_i . Operacje z różnych transakcji mogą się przeplatać.

- **Harmonogram szeregowy** - operacje każdej transakcji są wykonywane kolejno, **bez przeplatania** operacji z różnych transakcji. Są nieefektywne.
- **Harmonogram szeregowalny** - jego wpływ na stan bazy danych jest taki sam jak pewnego harmonogramu szeregowego.
- **Harmonogram nieszeregowy** - harmonogram, który nie jest sekwencyjny.
- **Harmonogram pełny** - harmonogram zawierający wszystkie operacje z transakcji składowych, gdzie dla dowolnych dwóch **operacji**, które są w **konflikcie**, jedna z nich musi **poprzedzać** drugą w harmonogramie.

Konflikt operacji – dwie operacje są w stanie konfliktu, jeśli spełnione są warunki:

- Należą do **różnych transakcji**,
- Uzyskują dostęp do **tego samego elementu**,
- **Przynajmniej jedna** z operacji jest **operacją zapisu** elementu.
- **Równoważność konfliktowa** – jeżeli **kolejność** wszystkich **operacji konfliktowych** jest w obu harmonogramach **taka sama**. Wyniki harmonogramów równoważnych konfliktowo są takie same.

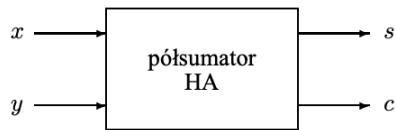
- **Szeregowalność konfliktowa** – jeżeli harmonogram S jest **konfliktowo równoważny** pewnemu harmonogramowi **szeregowemu** S'. Możemy zmieniać kolejność niekonfliktowych operacji w S. **Warunek wystarczający zachowania spójności danych**.
 - **Równoważność perspektywiczna harmonogramów**. Harmonogram S i S' zawierają te same instrukcje i dla każdego elementu danych Q:
 - Jeżeli T_k jest transakcją, która w S czyta Q jako pierwsza, to w S' T_k musi być transakcją, która czyta Q jako pierwsza.
 - Jeżeli w S T_i czyta Q zapisany przez T_j , to w S' T_i czyta Q zapisany przez T_j .
 - Jeżeli w S T_m jest ostatnią transakcją, która zapisuje Q, to w S' T_m jest ostatnią transakcją, która zapisuje Q.
 - **Szeregowalność perspektywiczna**. Harmonogram szeregowalny perspektywicznie jest harmonogramem **równoważnym perspektywicznie** jakiemuś harmonogramowi szeregowemu.
- Szeregowalność perspektywiczna to **szeregowalność konfliktowa z dodatkowymi ograniczeniami** operacji zapisów:
- każda operacja $w(x)$ jest poprzedzona $r(x)$,
 - wartość zapisana przez operację $w(x)$ jest **nie-stałą funkcją** zależną tylko od wartości $r(x)$.

81 Definicja cyfrowego układu kombinacyjnego - przykłady układów kombinacyjnych i ich implementacje.

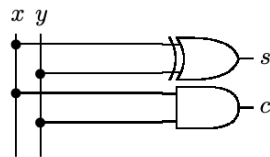
Układ kombinacyjny to taki układ logiczny, w którym **każda kombinacja wartości zmiennych wejściowych jednoznacznie określa kombinację wartości zmiennych wyjściowych**. Opisujemy je za pomocą bramek logicznych lub tablicy prawdy.

Bramka logiczna to układ elektroniczny realizujący funkcję logiczną jednej lub wielu zmiennych (w podstawowej wersji dwóch).

Półsumator (Half Adder).



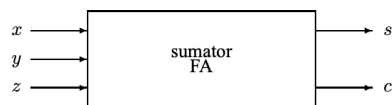
Wejścia		Wyjścia	
x	y	c	s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



$$s(x, y) = \sum(1, 2) = \bar{x}y + x\bar{y} = x \oplus y$$

$$c(x, y) = \sum(3) = xy$$

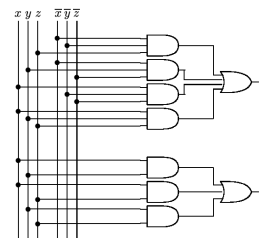
Sumator (Full Adder).



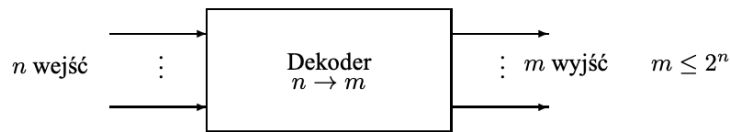
Wejścia			Wyjścia	
x	y	z	c	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$s(x, y, z) = \sum(1, 2, 4, 7) = \bar{x}\bar{y}z + \bar{x}y\bar{z} + x\bar{y}\bar{z} + xyz$$

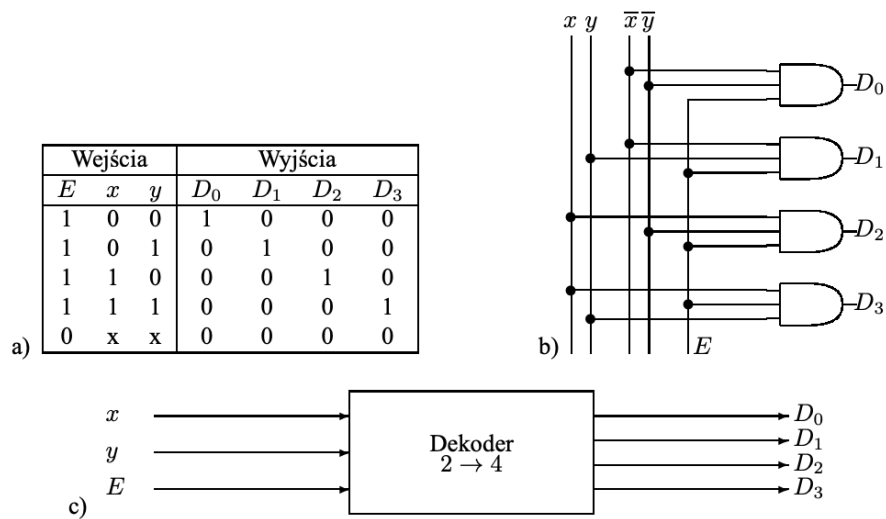
$$c(x, y, z) = \sum(3, 5, 6, 7) = xz + yz + xy$$



Dekoder – układ kombinacyjny mający n wejść i $m \leq 2^n$ wyjść, w którym każdej kombinacji zmiennych wejściowych odpowiada pojawienie się jedynki logicznej tylko na jednym wyjściu, na pozostałych wyjściach zera.



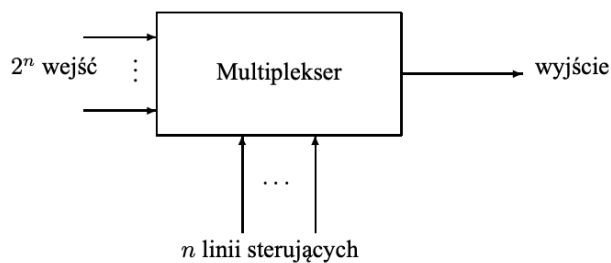
Rys. 3.23. Schemat blokowy dekodera $n \rightarrow m$

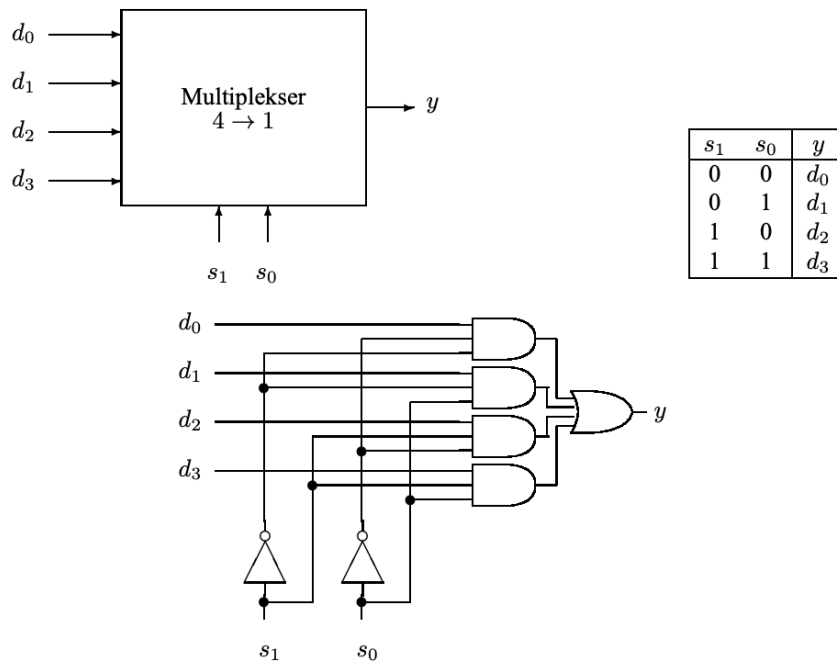


Rys. 3.24. Dekoder $2 \rightarrow 4$ z wejściem E (enable): tablica prawdy (a)), schemat logiczny (b)) i schemat blokowy (c))

Koder to układ kombinacyjny działający odwrotnie do dekodera.

Multiplekser – układ kombinacyjny wybierający informację dwójkową na jednej z linii wejściowych i kierujący ją na jedną linię wyjściową. Wybór linii wejściowej jest określany przez linie sterujące.





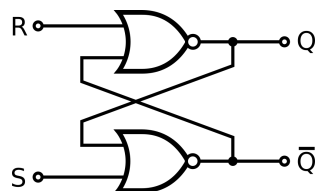
82 Definicja cyfrowego układu sekwencyjnego - przykłady układów sekwencyjnych i ich implementacje.

Układ sekwencyjny – układ, w którym poziomy sygnałów wyjściowych zależą nie tylko od wejścia ale również od stanu poziomów, które występowały uprzednio. Oznacza to, że układy te zawierają elementy pamiętające i nazywane są czasem **układami kombinacyjnymi z pamięcią**.

82.1 Przerzutnik RS (Reset-Set)

- najprostszy przerzutnik: 2xNOR.
- $R \rightarrow \overline{Q}$, $S \rightarrow \overline{Q}_n$ tylko jeśli na którymś sygnał.

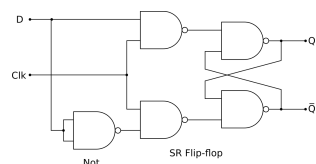
R	S	Q_n	$\overline{Q_n}$
0	0	Q_{n-1}	$\overline{Q_{n-1}}$
0	1	1	0
1	0	0	1
1	1	-	-



82.2 Przerzutnik D (Delay)

- wejścia D oraz C (Clock)
- D przechodzi na wyjście w momencie “tyknięcia” zegara – przerzutnik jest **synchroniczny**

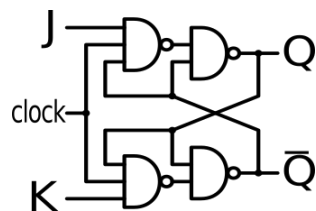
D	Q_{n-1}	Q_n
0	-	0
1	-	1



82.3 Przerzutnik JK

- synchroniczny,
- obsługuje stan 11,
- 3 wejścia: J(S), K(R) oraz Clock, wyjście: Q – stałe dla 00, odwrotne dla 11, równe J wpp.

J	K	Q_{n-1}	Q_n
0	0	0	0
0	0	1	1
1	0	-	1
0	1	-	0
1	1	0	1
1	1	1	0



83 Minimalizacja funkcji logicznych.

Minimalizacja z wykorzystaniem algebry Boole’a – polega na **zapisaniu funkcji w postaci sumy stanów wejść**, dla których funkcja przyjmuje wartość 1, a następnie wykorzystując prawa algebry Boole’a **uproszczenie** jej.

Minimalizacja z wykorzystaniem tablic Karnaugh

Tablica Karnaugh jest cykliczna; składa się z pól reprezentujących wszystkie kombinacje zmiennych naszej funkcji, z czego każde sąsiednie dwa pola różnią się jednym bitem.

Dla dwóch zmiennych

$$x \left\{ \begin{array}{cc} & \overbrace{y} \\ \begin{array}{c} 00 \\ 10 \end{array} & \begin{array}{c} 01 \\ 11 \end{array} \end{array} \right.$$

$$x \left\{ \begin{array}{cc} & \overbrace{y} \\ \begin{array}{c} 0 \\ 2 \end{array} & \begin{array}{c} 1 \\ 3 \end{array} \end{array} \right.$$

$$x \left\{ \begin{array}{cc} & \overbrace{y} \\ \begin{array}{c} \bar{x} \bar{y} \\ x \bar{y} \end{array} & \begin{array}{c} \bar{x} y \\ x y \end{array} \end{array} \right.$$

Dla trzech zmiennych

$$x \left\{ \begin{array}{cccc} & & \overbrace{y} & \\ \begin{array}{c} 000 \\ 100 \end{array} & \begin{array}{c} 001 \\ 101 \end{array} & \begin{array}{c} 011 \\ 111 \end{array} & \begin{array}{c} 010 \\ 110 \end{array} \end{array} \right.$$

$\underbrace{\hspace{10em}}_z$

$$x \left\{ \begin{array}{cccc} & & \overbrace{y} & \\ \begin{array}{c} 0 \\ 4 \end{array} & \begin{array}{c} 1 \\ 5 \end{array} & \begin{array}{c} 3 \\ 7 \end{array} & \begin{array}{c} 2 \\ 6 \end{array} \end{array} \right.$$

$\underbrace{\hspace{10em}}_z$

$$x \left\{ \begin{array}{cccc} & & \overbrace{y} & \\ \begin{array}{c} \bar{x} \bar{y} \bar{z} \\ x \bar{y} \bar{z} \end{array} & \begin{array}{c} \bar{x} \bar{y} z \\ x \bar{y} z \end{array} & \begin{array}{c} \bar{x} y z \\ x y z \end{array} & \begin{array}{c} \bar{x} y \bar{z} \\ x y \bar{z} \end{array} \end{array} \right.$$

$\underbrace{\hspace{10em}}_z$

Przy pomocy tablic Karnaugh możemy minimalizować funkcję o maksymalnie 6 zmiennych.

84 Programowalne układy logiczne PLD (ROM, PAL, PLA).

ROM (Read Only Memory)

k linii adresowych $\rightarrow 2^k$ słów n -bitowych $\rightarrow n$ linii wyjściowych

Zbudowana z dekodera i bramek OR. Jeśli chcemy np. sumator, to wejście do dekodera do xyz, wyjście z dekodera to wartości na s i c realizowane przy pomocy OR.

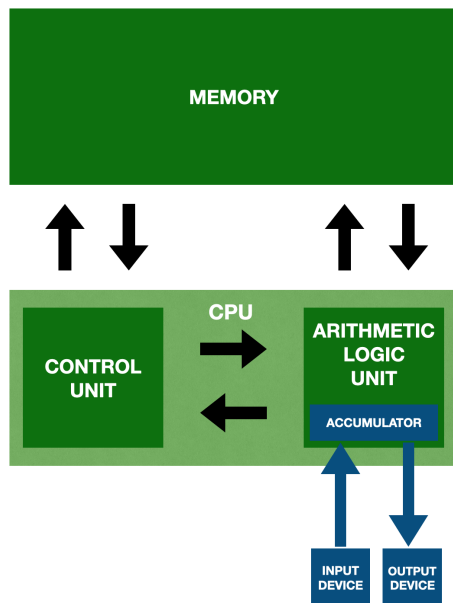
PLA (Programmable Logic Array) zawierają część programowalną matrycę bramek OR oraz programowalną matrycę bramek AND - tzn. realizuje funkcje będące sumą iloczynów, możemy ustawić jakich iloczynów.

PAL (Programmable Array Logic) – zawierają część nieprogramowalną (wbudowane bramki OR) oraz programowalną matrycę bramek AND – tak jak PAL, ale **ustalona ilość OR**.

PLA	PAL
droższe dowolna ilość bramek dużo funkcji	szybsze ustalona ilość OR ograniczona ilość, bo mniej elastyczne

85 Schemat blokowy komputera (maszyna von Neumanna).

- Niemalże wszystkie cyfrowe, elektroniczne współczesne komputery są oparte na architekturze von Neumanna
- Architektura von Neumanna pozwoliła na oddzielenie programu od procesora
- Istnieje architektura Harvardzka, oddzielająca pamięć z rozkazami od pamięci z danymi.
- Cechy:
 - Jednolitość pamięci:
 - * Możliwość uruchomienia samomodyfikującego się programu
 - * Dzielenie pamięci instrukcji z pamięcią danych może prowadzić do bottlenecków (w tym celu wprowadza się cache)
 - Skończona liczba rozkazów
 - Informacje są przetwarzane poprzez sekwencyjny odczyt kolejnych instrukcji z pamięci i ich wykonanie przez procesor



- **Pamięć** - zawiera zarówno dane, jak i instrukcji
- **Procesor** (CPU, Central Processing Unit)
 - **Jednostka kontrolna** (nadzorująca)
 - **Jednostka arytmetyczno-logiczna** – wykonuje podstawowe operacje przetwarzania informacji.
 - * **Akumulator** – obsługuje urządzenia wejścia-/wyjścia (I/O).

86 Zarządzanie procesami: stany procesu, algorytmy szeregowania z wywłaszczaniem.

Program to zbiór instrukcji, element procesu znajdujący się w segmencie kodu, którego stan nie ulega modyfikacji w czasie wykonywania; wykorzystuje dodatkowe zasoby (procesor, pamięć itp).

Proces służy do **organizowania wykonywania programu** w ten sposób, że stanowi on powiązanie **niezbędnych zasobów systemu komputerowego** i umożliwia **kontrolę stanu** tych zasobów związaną z wykonywaniem programu.

Zarządzanie procesami. Fragmenty kodu jądra związane z obsługą procesów:

- **zarządcy procesów** – grupują i wykonują funkcje obsługi procesów.
- **zarządcy zasobów** – kontrolują stan zajętości zasobów i realizację żądań procesów.

Stany procesu – Nowy → Gotowy ↔ Wykonywany (↔ Oczekujący) → Zakończony

- **Nowy** — formowanie procesu – gromadzenie zasobów (z wyjątkiem procesora), oczekiwanie na przyjęcie do kolejki procesów gotowych.

- **Gotowy** — oczekiwanie na przydział kwantu czasu procesora
- **Wykonywany** — wykonywanie instrukcji programu danego procesu i wynikająca z ich wykonywania zmiana stanu odpowiednich zasobów systemu.
- **Oczekujący** — zatrzymanie wykonywania instrukcji programu ze względu na potrzebę przydziału dodatkowych zasobów, konieczność otrzymania danych lub osiągnięcia odpowiedniego stanu przez otoczenie procesu
- **Zakończony** — zakończenie wykonywania programu, zwolnienie większości zasobów i oczekiwanie na możliwość przekazania informacji o zakończeniu. Pozostawanie procesu z stanie zakończony (zombi) spowodowane jest przetrzymywaniem pewnych informacji o procesie po jego zakończeniu (np. statusu).

Wywłaszczenie – przejście Wykonywany $\rightarrow$ Gotowy – może być następstwem:

1. **Upływu kwantu czasu** – przy algorytmie rotacyjnym planowania przydziału procesora.
2. **Pojawienia się procesu gotowego z wyższym priorytetem** – w systemie z **dynamicznymi priorytetami**, które są przeliczane co jakiś czas, przy stosowaniu wywłaszczeniowego podejścia do planowania przydziału procesora.

Szeregowanie procesów (planowanie) – wybór jednego z procesów (lub wątków) gotowych i przekazaniu mu procesora.

- **Funkcja priorytetu** (wyboru) wyznacza aktualny priorytet procesu na podstawie parametrów procesu i stanu systemu.
 - zbiór wytycznych dla planisty,
 - w Windows większy = wyższy, w unix mniejszy = wyższy
 - uwzględnia przede wszystkim stan procesu, może też stan systemu
- **Tryb decyzji** – kiedy są oceniane priorytety oraz dokonywany jest wybór.
 - zbiorem wytycznych odnośnie uruchamiania ekspedytora,
 - **wywłaszczeniowy** lub **niewywłaszczeniowy** (proces może się zrzec, np. po wejściu w stan oczekiwania).
- **Reguła arbitrażu** — reguła rozstrzygania konfliktów w dostępie do procesora.
 - **Arbitraż losowy** przy małej zmienności priorytetów mógłby prowadzić do głodzenia procesów.
 - **Arbitraż cykliczny** – trudny w realizacji przy zmiennych priorytetach, ok przy stałych.
 - **Arbitraż chronologiczny** (FIFO przyjmowania procesów do systemu)
 - najbardziej sprawiedliwy, ale wymaga utrzymania odpowiednich atrybutów procesów.

Algorytmy planowania przydziału procesora:

- **FCFS** (First Come First Served)
 - w systemach obsługi masowej, takich jak kasy sklepowe, banki itd.
 - możliwość oddania procesora innemu procesowi na czas oczekiwania.
- **LCFS** (Last Come First Served)
 - nie wywłaszcza,
 - nie ma praktycznego zastosowania.
- **SJF** (Shortest Job First)
 - preferuje procesy, które mają najmniejsze wymagania odnośnie czasu procesora,
 - "najpierw zadanie z najkrótszą następną fazą procesora",
 - również w systemach masowej obsługi,
 - problem określenia przyszłego zapotrzebowania na procesor.

87 Muteks, semafor, monitor jako narzędzia synchronizacji procesów.

Sekcje krytyczne – fragmenty kodu, które nie mogą zostać wykonane „w tym samym czasie” przez kilka wątków.

Semafor – chroniona zmienna.

- **Semafor binarny** przyjmuje wartości **true** (stan podniesienia, otwarcia) lub **false** (stan opuszczenia, zamknięcia).
- **Semafor ogólny** (zliczający) przyjmuje wartości całkowite nieujemne, zwiększany/zmniejszany przy podniesieniu (V)/opuszczeniu (P).

Muteks (Zamek) – szczególny rodzaj semaforów binarnych (1 - zablokowany, 0 - odblokowany), tylko jeden wątek może być w posiadaniu muteksa.

- **Cecha posiadania** - jeżeli zadanie zablokuje mutex to tylko ono może ten muteks odblokować.

- **Rekurencyjne blokowanie** - określa ile razy zadanie, które posiada dany mutex wykonało na nim operacje blokowania (zapobiega to zakleszczeniu).

Operacje:

- **lock** (`mutex_lock`) – zajęcie mutexu jeśli jest wolny
- **unlock** (`mutex_unlock`) – ustawienie zamka na wolny lub ustawienie wątku z kolejki na gotowy
- **trylock** (`mutex_trylock`) – zajęcie zamka lub kontynuacja przetwarzania

Monitor (zmienna warunkowa) – zmienne i działające na nich procedury zebrane w jednym module (klasie), dostęp do nich tylko za pomocą procedur. W danej chwili tylko jeden proces może wykonywać procedury monitora (FIFO); istnieje możliwość wstrzymania i wznowienia procedur za pomocą zmiennych warunkowych. Może to być np. klasa reprezentująca konto w banku i udostępniająca procedury zmiany stanu.

Operacje:

- **wait** – powoduje wstrzymanie procesu i wstawienie go na koniec kolejki
- **signal** – powoduje wznowienie pierwszego procesu z kolejki, o ile kolejka nie jest pusta

88 Pamięć wirtualna i mechanizm stronicowania.

Pamięć wirtualna jest **organizacją zasobów pamięci**, zrealizowaną w oparciu o tzw. przestrzeń wymiany w pamięci drugiego rzędu (na dysku). Pamięć operacyjna (fizyczna) jest dla tych zasobów tylko pewnym oknem, przechowującym część zawartości na potrzeby bieżącego przetwarzania. Stosowanie pamięci wirtualnej umożliwia **bardziej racjonalne wykorzystanie pamięci operacyjnej**, gdyż programy tworzone są często z nadmiarem w stosunku do typowych potrzeb.

Stronicowanie

- **Arbitralny podział pamięci fizycznej na ramki**, w które ładowane są odpowiednie strony obrazu procesu.
- **Podział logicznej przestrzeni adresowej na strony** o takim samym rozmiarze, jak ramki w pamięci fizycznej.
- **Zalety:**
 - brak problemu fragmentacji zewnętrznej

- wspomaganie dla współdzielenia i ochrony pamięci
- **Wady:**
 - narzut czasowy przy transformacji adresu
 - narzut pamięciowy (na potrzeby tablicy stron)
 - fragmentacja wewnętrzna (niewielka)

Mechanizm stronicowania na żądanie – strony są sprowadzane do pamięci tylko wówczas, gdy następuje odniesienie do komórki o adresie znajdującym się na tej stronie. Jest przede wszystkim z wymianą stron pomiędzy pamięcią pierwszego rzędu (operacyjną, fizyczną) i drugiego rzędu (masową, dyskową).

Problemy zastępowania stron

- **Problem wyboru ofiary** — niewłaściwy wybór ramki-ofiary powoduje wzrost kosztu wymiany. W skrajnym przypadku może dojść do zjawiska migotania (częste odnoszenie się do usuniętej strony).
 - Zakładając, że przyszły ciąg odniesień do pamięci nie jest znany, na podstawie **historii** odniesień należy wybrać najlepszą ramkę.
 - Podstawowa własność programów, na podstawie której można szacować takie prawdopodobieństwo, nazywana jest **lokalnością**.
- **Problem wznawiania rozkazów** — w przypadku wielokrotnego odniesienia do pamięci w jednym cyklu rozkazowym należy zapewnić, że wszystkie adresowane strony są jednocześnie dostępne w ramach w pamięci fizycznej.

Klasyfikacja algorytmów wymiany z względu na **okoliczności sprowadzania i usuwania stron**:

- **Algorytmy wymiany na żądanie** – sprowadzanie i usuwanie odbywa się na żądanie (odwołanie/brak wolnej ramki)
 - **MIN** — zastępowana jest strona, która najdłużej nie będzie używana,
 - **FIFO** – First In First Out,
 - **LIFO** – Last In First Out,
 - **LRU** – Least Recently Used,
 - **LFU** – Least Frequently Used,
 - **MFU** – Most Frequently Used.
- **Algorytmy wymiany ze sprowadzaniem na żądanie** – tylko sprowadzanie odbywa się na żądanie

- **Algorytmy wstępnego sprowadzania** – sprowadzana jest strona żądana, a wraz z nią inne strony

Klasyfikacja algorytmów wymiany ze względu na **sposób zastępowania stron**

- **Zastępowanie lokalne** — zastępuje tylko strony w ramach przydzielonych procesowi, który spowodował błąd strony.
- **Zastępowanie globalne** — zastępuje strony znajdujące się w dostępnej puli ramek w całym systemie (w szczególności zatem usuwa strony innych procesów).

Klasyfikacja algorytmów wymiany ze względu na **przydział ramek dla procesów**: przydział **statyczny** lub **globalny**. **Dobór liczby ramek**:

- **Minimalna liczba ramek** — zdefiniowana przez architekturę komputera (zależna od maksymalnej liczby komórek adresowanych przez jeden rozkaz).
- Liczba ramek przydzielona dla procesu – przydział **równomierny**, **proporcjonalny** lub **zależny od priorytetu** procesu.

89 Systemy plikowe - organizacja fizyczna i logiczna (na przykładzie wybranego systemu uniksopodobnego).

89.1 Organizacja fizyczna

- Przydział miejsca na dysku
 - **Przydział ciągły** — cały plik zajmuje ciąg kolejnych bloków
 - **Przydział listowy** (łańcuchowy) — bloki pliku tworzą listę powiązaną
 - **Przydział indeksowy** — bloki z danymi wskazywane są przez bloki indeksowe, które mogą być zorganizowane w:
 - * schemat listowy
 - * schemat wielopoziomowy
 - * schemat kombinowany
- Zarządzanie **wolną przestrzenią**:
 - **Wektor bitowy** — bity odpowiadają blokom, 1 = wolny blok.
 - **Lista powiązana** — wolny blok zawiera indeks następnego.

- **Grupowanie** — niektóre wolne bloki wypełnione są w całości indeksami innych wolnych bloków, ostatni indeks wskazuje na kolejny blok wypełniony w całości indeksami.
- **Zliczanie** — wykaz wolnych bloków obejmuje indeks pierwszego wolnego bloku oraz liczbę wolnych bloków znajdujących się za nim.
- **Implementacja katalogu** – lista liniowa, tablica haszowa, struktura indeksowa

89.2 Organizacja logiczna

Zadania systemu operacyjnego – w odniesieniu do plików to zapewnienie odwzorowania pomiędzy abstrakcyjnym obrazem informacji a jego reprezentacją na urządzeniu fizycznym.

- **identyfikacja pliku** (hierarchiczna struktura katalogów)
- udostępnienie **interfejsu** operacji plikowych (API)
- realizacja **operacji dostępu** do plików i katalogów z zapewnieniem **bezpieczeństwa, spójności i efektywności**.

Atrybuty pliku: nazwa, typ, rozmiar, lokalizacja (w systemie), ochrona (kontrola dostępu), czasy dostępu.

Logiczna **struktura pliku** określa powiązanie informacji wewnątrz pliku ("struktura informacji"). Np. konkretny bajt może wyznaczać długość nagłówka, który oddziela metadane od informacji.

89.3 UNIX

- Z każdym plikiem związany jest **i-węzeł**, który przechowuje wszystkie atrybuty pliku z wyjątkiem nazwy. I-węzły tworzą tablicę, której rozmiar limituje liczbę plików w systemie.
- Nazwa znajduje się w katalogu obok numeru i-węzła danego pliku.
- Katalogi tworzą **strukturę wielopoziomową** (zawierają wpisy specyfikujące inny katalog).
- **Dane** (zawartość pliku) znajdują się w **blokach** (jednostkach alokacji) o ustalonym rozmiarze.
- Bloki danych identyfikowane są za pośrednictwem **indeksu kombinowanego**.
- **Wolne bloki** zorganizowane są zgodnie z zasadą **grupowania**, w pewnych odmianach wektor bitowy (wtedy również do identyfikacji wolnych i-węzłów).

Format partycji

- **tablica i-węzłów** – ustalona liczba i-węzłów; jeden, ustalony i-węzeł opisuje korzeń drzewa katalogów.
- **bloki danych** – liczba bloków ustalona,

Fizyczna struktura pliku – z każdym powiązany jest jeden i-węzeł zawierający atrybuty:

- **identyfikator** właściciela i grupy,
- **typ pliku** — plik zwykły, katalog, dowiązanie symboliczne, łącze nazwane, urządzenie znakowe, urządzenie blokowe, gniazdo,
- **prawa dostępu** — tradycyjne rwx dla właściciela, grupy i pozostałych,
- **czasy dostępu** — czas modyfikacji pliku, czas modyfikacji i-węzła, czas dostępu,
- **licznik dowiązań** — liczba różnych nazw, pod jakimi występuje plik w systemie,
- **rozmiar** w bajtach,

Pozostała część i-węzła wypełniona jest indeksami bloków z danymi.

Struktura wpisu katalogowego – katalog składa się z wpisów, kojarzących nazwę z i-węzłem. W tradycyjnym podejściu nazwa ograniczona była do 14 znaków, współcześnie dopuszcza się nawet 256 znaków. W katalogu istnieją też wpisy specjalne o nazwie . (kropka) i .. (dwie kropki), skojarzone z numerem i-węzła odpowiednio katalogu bieżącego i nadrzędnego. Wyjątkiem w tym zakresie jest korzeń drzewa katalogów, który nie ma nadkatalogu.

90 Model ISO OSI. Przykłady protokołów w poszczególnych warstwach.

Model OSI (Open Systems Interconnection) utworzony przez Międzynarodową Organizację Normalizacyjną ISO stanowi **model referencyjny**.

Nr warstwy OSI	Nazwa warstwy OSI	Przykładowe protokoły
7	Aplikacji	HTTP/HTTPS, DNS, Telnet
6	Prezentacji	SSL, TLS
5	Sesji	NetBios, SAP
4	Transportu	TCP, UDP
3	Sieci	IPv4, IPv6, IGMP
2	Łącza danych	ARP
1	Fizyczna	Ethernet

Warstwa aplikacji - interfejs między aplikacjami a usługami sieci.

- **HTTP/HTTPS** (Hypertext Transfer Protocol (Secure)) - protokół przesyłania dokumentów hipertekstowych.
- **DNS** (Domain Name System) - system nazw sieciowych (domenowych).
- **Telnet** - protokół do obsługi odległego terminala w architekturze klient-serwer.

Warstwa prezentacji - kompresja, kodowanie i translacja między niezgodnymi schematami kodowania oraz szyfrowanie.

- **SSL** (Secure Socket Layer) - protokół szyfrowania zapewniający poufność i integralność danych. Wykorzystuje szyfrowanie symetryczne z kluczem publicznym, stosuje sumy kontrolne dla zapewnienia integralności danych.
- **TLS** (Transport Layer Security) - jak wyżej. Ustalenie klucza symetrycznego szyfrowaniem niesymetrycznym, symetryczne szyfrowanie danych.

Warstwa sesji - zarządzanie przebiegiem komunikacji podczas połączenia między komputerami.

- **NetBios** - protokół sieciowy zapewniający podstawowy interfejs łączenia aplikacji z innymi aplikacjami w sieci lokalnej.
- **SAP** (Session Announcement Protocol) - protokół wykorzystywany przez usługi powodujące tzw. "burze ogłoszeń" (radio, telewizja internetowa).

Warstwa transportu - kontrola błędów i przepływu danych poza lokalnymi segmentami LAN, protokoły komunikacji procesów na odległych komputerach.

- **TCP** (Transmission Control Protocol) - połączeniowy, niezawodny, strumieniowy.
- **UDP** (User Datagram Protocol) - prosty bezpołączeniowy protokół komunikacyjny, nie zapewnia niezawodności.

Warstwa sieci - określenie trasy przesyłania danych poza lokalnym segmentem sieci LAN, protokoły trasowane.

- **IP** (Internet Protocol) - protokół internetowy będący zbiorem automatycznych kroków nawiązywania łączności.
- **IGMP** (Internet Group Management Protocol) - multicast w IP.

Warstwa łącza danych – grupowanie danych wejściowych (z warstwy fizycznej) w bloki zwane **ramkami** danych („jednostki danych usług warstwy fizycznej”), mechanizmy kontroli poprawności transmisji.

- **ARP** (Address Resolution Protocol) - protokół sieciowy umożliwiający mapowanie logicznych adresów warstwy sieciowej na fizyczne adresy warstwy łącza.

Warstwa fizyczna - standard połączenia fizycznego, charakterystyki wydajnościowe nośników. Same media transmisyjne pozostają poza dziedziną jej zainteresowania (czasem określane są terminem warstwa zerowa).

- **Ethernet** - technologia zawierająca specyfikację przewodów, sygnałów, ramek.
- **DSL** - technologia cyfrowego szerokopasmowego dostępu do Internetu.

91 Adresowanie w protokołach IPv4 i IPv6.

91.1 Adresowanie w IPv4.

Adres IP jest przypisywany do karty sieciowej, nie do komputera. Składają się z czterech części bo 8 bajtów

Istnieją trzy typy adresów IPv4:

- **Adresy jednostkowe** (unicast) – pojedynczy interfejs sieciowy (one-to-one).
- **Adresy rozgłoszeniowe** (broadcast) – wszystkie węzły w tym samym segmencie sieci (one-to-everyone).
- **Adresy grupowe** (multicast) – jeden lub wiele komputerów w jednej lub w różnych segmentach sieci (one-to-many).

W adresie IP zapisanym binarnie można wyróżnić **dwie części**:

- **Identyfikator sieci** (Network ID) - pewna liczba bitów z lewej strony adresu.
- **Identyfikator hosta** (Host ID) - pozostałe bity.

Adres IP, który zawiera **same zera** w części hosta jest traktowany jako **adres sieci**. **Adresy rozgłoszenia do sieci lub podsieci mają jedynki tylko w części hosta**.

Adres ograniczonego rozgłoszenia - 255.255.255.255- adres rozgłoszenia w danym segmencie sieci ograniczonym routerami.

Adresowanie oparte na klasach – pierwszy bajt adresu determinuje do jakiej klasy należy sieć.

Klasa	Adres sieci	Adresy	Zakres 1-go bajtu	Najstarsze bity
A	w.0.0.0	1.0.0.0 - 126.0.0.0	1 – 126	0
B	w.x.0.0	128.0.0.0 - 191.255.0.0	128 – 191	10
C	w.x.y.0	192.0.0.0 - 223.255.255.0	192 – 223	110
D	nie dotyczy	nie dotyczy	224 – 239	1110
E	nie dotyczy	nie dotyczy	240 – 255	11110

- **Adresy klasy D** - przeznaczone są do transmisji grupowych.
- **Adresy klasy E** - zarezerwowane (nie wykorzystywane normalnie do transmisji pakietów).
- **Adresy pętli zwrotnej** (loopback) - postaci 127.x.y.z (na ogół 127.0.0.1). Cały ruch przesyłany na ten adres nie wychodzi z komputera.

Adresowanie bezklasowe – dzielenie na podsieci z **użyciem dowolnej liczby jedynek**. Do określenia sieci należy podać adres sieci oraz **maskę**. Obecnie w Internecie powszechnie jest wykorzystywane adresowanie bezklasowe.

91.2 Adresowanie w IPv6.

- **osiem 16 bitowych sekcji w zapisie szesnastkowym**, oddzielonych dwukropkami, np.: fe80:0000:0000:0000:0202:b3ff:fe1e:8329
- długi ciąg zer może być **raz** połączony i przedstawiony jako "::"
- w środowisku z węzłami IPv4 oraz IPv6 często **w adresie IPv6 umieszcza się również IPv4**. Są dwa typy takich adresów:
 - **Typ 1: IPv4-Compatible IPv6 Address** - 96 bitów zero + adres IPv4
 - **Typ 2: IPv4-Mapped IPv6 Address** (zalecany) - 0000...0000fff + IPv4 address

- Zamiast identyfikatora sieci, w IPv6 używa się **prefiksu** – **pewnej liczby bitów** adresu z lewej strony, standardowo pewna liczba sekcji adresu.

Typy adresów

- **Adres niewyspecyfikowany** - `::/128`
- **Pętla zwrotna** (Loopback) - `::1/128`
- **Multicast** - `ff00::/8` (1111 1111)
- **Link-local unicast** - `fe80::/10` (1111 1110 10)
- **Global unicast**

Klasyfikacja adresów.

1. **adresy jednostkowe (unicast)** - może być przypisany do wielu interfejsów lub wielu komputerów; podzielony na prefiks (typowo 48 bitów dostawcy, 16 podsieci) i identyfikator interfejsu.
2. **adresy pobliskie (anycast)** - z zakresu adresów jednostkowych organizacji, przydzielane do grupy węzłów (np. routerów).
3. **adresy grupowe (multicast)** - zastępują adresy rozgłoszeniowe i grupowe w IPv4, przydzielane do grupy węzłów lecz w przeciwieństwie do anycasta wszystkie węzły odbierają pakiety wysłane do danego adresu - 11111111 + 4 bity flagi + 4 bity zakresu + 112 bitów ID grupy.

Skąd wziąć adres MAC?

- **Unicastowy adres IP na multicastowy** - `ff02:0:0:0:1:ff__:__`, z trzema najmłodszymi bajtami adresu IP
- **Multicastowy MAC** - `33:33:__:__:__` z czterema bajtami adresu multicastowego
- Ramka skonstruowana z adresem MAC, z pakietem IP z multicastowym IP w środku, z unicastowym w komunikacie ICMP
- Switch się znajduje na tyle, że ramka w miarę dochodzi tylko tam gdzie trzeba
- Komputer otrzymujący ramkę sprawdza czy o niego chodzi w komunikacie ICMP, i unicastowo odpowiada

92 Najważniejsze procesy zachodzące w sieci komputerowej od momentu wpisania adresu strony WWW do wyświetlenia strony w przeglądarce (komunikat HTTP, segment TCP, system DNS, pakiet IP, ARP, ramka).

Założmy, że klient wpisał w przeglądarkę internetową adres `www.ksi.sh`.

1. Zapytanie DNS

- (a) Przeglądarka uruchamia wywołanie systemowe odpowiedzialne za rozwiązanie nazwy domenowej na adres IP
- (b) System operacyjny sprawdza, czy żądany adres nie jest zapisany w cache'u
- (c) Jeżeli nie to wysyła on **zapytanie rekurencyjne** do skonfigurowanego w systemie serwera DNS (najczęściej serwera naszego ISP - Internet Service Provider'a) o adres IP związany z domeną `www.ksi.sh`.
- (d) Powyższy serwer DNS sprawdza (w swojej bazie), czy sam nie obsługuje żądanej domeny (nawet niepełnej, np. `.sh`, `ksi.sh`); sprawdza, czy nie ma nazwy w cache'u; rozpoczyna serię **zapytań iteracyjnych do autorytatywnych** serwerów poszczególnych domen (takich, które obsługują tylko iteracyjne zapytania serwerów, a nie komputery).
- (e) Ostatecznie serwer DNS naszego ISP zapisuje znaleziony adres w swoim cache'u i odsyła go do naszego komputera.

2. Tworzenie pakietów zgodnie ze stosem TCP/IP

- (a) Komputer z przeglądarką tworzy **zapytanie HTTP** o stronę `www.ksi.sh`
- (b) Komputer tworzy **pakiet IP** i opakuje w ten pakiet zapytanie HTTP
- (c) Komputer na podstawie swojego lokalnego adresu IP i ustawionej maski sprawdza, czy docelowy komputer znajduje się w jego sieci LAN; jeśli nie, to odczytuje ze swojej konfiguracji IP bramy domyślnej
- (d) Komputer sprawdza, czy nie ma adresu MAC IP bramy zapisanego w pamięci cache u siebie. Jeżeli nie, wysyła on na rozgłoszeniowy adres MAC **ARP Request**.
- (e) Brama odpowiada mu **unicastem** (na jego MAC) **ARP Response** ze swoim MACiem.
- (f) Komputer tworzy ramkę z MACami i ją wysyła do sieci

3. Przekazywanie pakietów do serwera WWW i z powrotem

- (a) Switch, do którego podłączony jest nasz komputer odbiera wysłane przez niego dane i zagląda do ramki
- (b) Sprawdza on docelowy adres MAC i zgodnie ze swoją tablicą przełączania przekazuje tę ramkę na właściwy port lub rozsyła do wszystkich portów
- (c) W końcu ramka trafia do naszej bramy
- (d) Brama jako router zagląda do pakietu IP i sprawdza zgodnie ze swoją tablicą routing'u do jakiego kolejnego routera lub sieci skierować ten pakiet
- (e) Gdy już znajdzie adres następnego skoku, router konstruuje kolejną ramkę, opakuje w nią pakiet IP i wysyła
- (f) Tak dzieje się do momentu aż pakiet nie dotrze do serwera WWW. Ten odczytuje żądanie HTTP, generuje odpowiedź HTTP i wysyła analogicznie.

93 Działanie przełączników Ethernet, sieci VLAN, protokół STP.

Przełącznik Ethernet'owy (Switch)

1. Gdy switch odbiera na jakimś porcie ramkę, sprawdza on czy adres MAC odbiorcy znajduje się w tablicy CAM:
 - (a) Jeżeli tak - ramka wysyłana jest tylko na odpowiedni port
 - (b) Jeżeli nie - Switch postąpi podobnie jak koncentrator sieciowy (hub) - roześle tę ramkę do wszystkich portów. Podobnie postąpi, jeżeli adres MAC odbiorcy jest adresem broadcastowym.
2. Switch odczytuje z ramki adres MAC nadawcy i wiąże go z portem, na który przyszła ta ramka, oraz zapisuje tę informację w tablicy CAM

Tryby pracy:

1. **Store-and-Forward** – przełącznik pobiera całą ramkę, analizuje czy nie jest błędna i przesyła ją dalej. Ten tryb wprowadza największe opóźnienie.
2. **Cut-through** (dwa podtypy):
 - (a) **Fast-forward switching** - po otrzymaniu adresu MAC miejsca docelowego ramki, już rozpoczyna się jej nadawanie. Jest to najszybsza metoda, ale nie sprawdza czy nie nastąpiła kolizja i ramka jest niepoprawna.

- (b) **Fragment-free switching** - ramka jest przesyłana do portu docelowego dopiero po otrzymaniu 64 bajtów (rozmiar najmniejszej poprawnej ramki w Ethernetie). Zapewnia to, że ramka nie koliduje z inną

Sieci VLAN – sposób oddzielenia od siebie pewnej grupy komputerów tak jakby były w LANie.

- podłączanie grup do osobnych switchy problematyczne – dla każdej grupy osobne urządzenie, fizyczne połączenie itd.
- **VLANy** pozwalają na **jednym switchu wydzielić** wiele osobnych “pod-switch’y” – grup portów
 - Nie musimy fizycznie przełączać kabli, gdy chcemy zmienić grupę
 - Utworzenie kolejnej grupy nic nas nie kosztuje
 - Jednym kablem możemy przysyłać wiele VLAN’ów
- Ramki w przypadku technologii VLAN mogą być **tagowane** – zawierać informację do którego VLANu należą, lub **nietagowane**.
- Porty i połączenia, na których wysyłane/odbierane są ramki **nietagowane** nazywamy **dostępowymi**, natomiast te obsługujące **tagowane** nazywamy **trunkowymi**. Standardowo komputery rozumieją tylko ramki nietagowane.
- **Router na kiju** – router podłączony do trunkowego portu switcha, obsługujący routing między VLANami, używa firewalli do ograniczania ruchu. Router posiada dla każdego VLANU **wirtualne interfejsy**.
- Żeby ograniczyć koszt komunikacji przez router na kiju używa się **przełączników warstwy trzeciej**, które są w stanie wykonywać routing między podłączonymi VLAN’ami (z firewallem).

STP - Spanning-Tree Protocol - protokół umożliwiający uniknięcie pętli w sieci LAN oraz mogący zapewnić redundantne ścieżki komunikacji przydatne przy awarii.

- umożliwia tworzenie **redundantnych ścieżek** między przełącznikami, ale w danej chwili tylko jedno ze zduplikowanych połączeń jest **aktywne**.
- aktywne połączenia tworzą strukturę drzewiastą z jednym przełącznikiem wybranym jako **korzeń**
- STP wykorzystuje specjalne **ramki BPDU** (Bridge Protocol Data Units), które są **regularnie** przesyłane między przełącznikami; mają specjalny multicastowy MAC docelowy; otrzymane ramki BPDU nie są wprost przekazywane

do innych portów, **przełącznik buduje swoje ramki** z otrzymanymi informacjami, np. koszt dotarcia do root-switch'a.

Działanie STP:

1. **Wybór korzenia** – realizowany przez BPDU na podstawie konfigurowalnego priorytetu przełącznika. Wygrywa najniższy priorytet, następnie najniższy adres MAC.
2. **Wybór portów do korzeni** – do przekazywania ramek do korzenia; bezpośrednio połączony do korzenia, lub ten o najniższym koszcie (przez ramki BPDU).
3. **Wybór portów wyznaczonych** (designated ports) – uczestniczących w wymianie informacji w drzewie rozpinającym; jeden na segment LAN.
4. **Blokada niepotrzebnych portów** – portów redundantnych, czy nie root and designated.

Stan połączeń jest analizowany na podstawie cyklicznie wysyłanych ramek BPDU, które są przesyłane cyklicznie.

94 Rola routerów i podstawowe protokoły routingu (RIP, OSPF).

Mechanizm routowania – sposób obsługi routowania przez warstwę IP; przeglądanie przez system operacyjny tablicy routowania i podejmowania decyzji co do przesyłania datagramów IP.

Polityka routowania – działania procesu routowania podejmowane w celu ustanowienia i modyfikacji tablicy routowania; realizowana z wykorzystaniem protokołów routowania.

Pożądane cechy protokołów routowania:

- Wyznaczenie **najlepszej trasy** do punktu docelowego
- **Odporność** (robustness) na nieprzewidziane, niepożądane zdarzenia: "kopara", niewłaściwa konfiguracja, obciążenia sieci
- **Szybkie osiągnięcie zbieżności**, tj. jednakiej topologii sieci w routerach.
- Dopasowanie do **zmian**, np. podłączenie, odłączenie danej sieci; zmiana szerokości pasma, opóźnienia.

Podział ze względu na sposób działania:

- **protokoły wektora odległości** (DV, distance-vector) – oparte na algorytmie Bellmana-Forda (najkrótsze ścieżki w grafie) – RIP, RIP2, IGRP, EIGRP (wg Cisco hybrydowy).
- **protokoły stanu łącza** (link-state) – stan łącza składa się z wielu atrybutów, np. prędkość transmisji, opóźnienie, adres IP, sąsiedzi – OSPF, OSPF2.

94.1 RIP

- **Metryką** w RIP jest **liczba skoków** – 16 to nieskończoność, więc źle dla dużych sieci.
- **Autosumaryzacja**, system **klasowy** adresacji ip.
- **Liczniki RIP**:
 - **Update** timer (30s) – wektor odległości (routing update)
 - **Invalid** timer (180s) – zegar od ostatniego uaktualnienia danej trasy.
 - **Hold-down** timer (180s) – po osiągnięciu wartości progowej przez invalid timer trasa jest ustawiana w stan hold-down.
 - **Flush** timer (240s) – zegar od ostatniej informacji o danej trasie. Po upływie trasa jest usuwana.
- **Zalety**: prostota, łatwa konfiguracja.
- **Wady**:
 - Wolno rozprzestrzenianie się informacji o zmianach w topologii
 - Wolne osiągnięcie zbieżności
 - Częste przesyłanie dużych porcji informacji (wektory odległości)

W RIP2 mamy maski podsieci, dzielenie podsieci na podsieci o zmiennym rozmiarze, prostą autentykację (hasła) i adresy następnego skoku w komunikatach.

94.2 OSPF

- Standardowo metryka = $\frac{\text{wartość odniesienia}}{\text{szerokość pasma}}$
- Standardowa wartość odniesienia to 100Mbps
- Ethernet 100Mbps ma metrykę 1, łącze 56kps ma metrykę 1768

Zasada działania:

- Każdy router posiada (albo buduje) informację o połączeniach w całej sieci

- Za pomocą algorytmu **Dijkstry** obliczane są najkrótsze ścieżki do węzłów. Jest to **zasobożerne**, więc tworzy się **obszary OSPF**:
 - Każdy obszar ma przypisany **numer**, z czego jeśli jeden to ma 0
 - **Stub Area** - nie ma tras zewnętrznych, sumy tras z innych obszarów są wprowadzane
 - **Totally Stubby Area** - nie ma ani tras zewnętrznych, ani sum tras z innych obszarów. Wyjście z takiego obszaru jest przez trasę domyślną
 - **Not So Stubby Area** - Stub Area, do którego wprowadzane są pewne, nieliczne trasy zewnętrzne. Trasy te wprowadzane są do innych obszarów tak jak sumy tras
 - **Not So Stubby Totally Stubby Area** - obszar połączenia NSSA i TSA
- **Rodzaje routerów**:
 - Router **desygnowany** - hosty mają z nim relację przyległości, z pozostałymi sąsiadami nie (on reaguje na zmiany, hosty na niego).
 - Routery wewnętrzne, **brzegowe** (Area Border Router), szkieletowe, brzegowe AS (Autonomous System Border Router).
 - **Gateway of last resort** - router do którego idziemy kiedy nie mamy trasy.
- Na początku pracy sieć jest zalewana informacją o stanie łączy (**flooding**)
- Przesyłane są tylko informacje o **zmianach** w stanie łączy - nie całe tablice
- **Ruch** w wielodostępnych fragmentach sieci jest **ograniczany** przez wybranie **DR (designated router)** i **BDR (backup designated router)**
- W przypadku istnienia kilku tras o jednakowym koszcie realizowane jest **równoważenie obciążeń** (load balancing).

95 Szyfrowanie z kluczem publicznym, podpis cyfrowy, certyfikaty.

Szyfrowanie z kluczem publicznym (asymetryczne)

- szyfrowanie i odszyfrowanie jest realizowane przy pomocy pary kluczy - **prywatnego i publicznego**.
- A wysyła do B zaszyfrowane kluczem publicznym B – wtedy tylko B może odszyfrować (swoim prywatnym)
- A wysyła do B zaszyfrowane swoim kluczem prywatnym – wtedy B ma pewność, że to A szyfrowało

Skrót (Hash) – skrót wiadomości tworzony za pomocą deterministycznej **funkcji haszującej** (MD5, SHA); jeśli w oryginalnej wiadomości zmieniony zostanie chociaż jeden bit, to skrót będzie zupełnie inny; odtworzenie oryginału praktycznie niemożliwe.

Podpis cyfrowy – **zaszyfrowany kluczem prywatnym skrót** wiadomości dołączony do niej jest gwarancją tego, że wiadomość pochodzi z odpowiedniego źródła.

Certyfikaty

- Klucze mogą być **generowane lokalnie** przy pomocy odpowiedniego oprogramowania i powinny być podpisane przez jakieś **centrum certyfikacyjne**.
- Centrum certyfikacyjne (CA) wydaje tzw. **certyfikaty cyfrowe** zawierające m.in:
 - Identyfikator osoby/firmy/obiektu oraz CA, który wydał certyfikat
 - Numer identyfikacyjny certyfikatu
 - Cel stosowania (np. podpisywanie bezpiecznych stron WWW albo podpisywanie listów elektronicznych)
 - Wartość klucza publicznego
 - Okres ważności
 - Podpis cyfrowy wydawcy
- W systemach operacyjnych oraz różnych programach jest wpisana lista zaufanych CA.
- Niezależnym standardem opisującym tworzenie kluczy, rejestrowanie i wykorzystywanie certyfikatów jest PGP (**Pretty Good Privacy**).

96 Wirtualne sieci prywatne, protokół IPsec.

Wirtualna Sieć Prywatna (VPN)

- sieć przekazu danych korzystająca z publicznej infrastruktury telekomunikacyjnej
- dzięki stosowaniu protokołów tunelowania i procedur bezpieczeństwa zachowuje **poufność danych**
- opiera się na dynamicznych, wirtualnych połączeniach (trasy przesyłu danych oraz przepływność pasma dla połączenia ustawiane dynamicznie), czy też tunelach – nie mających fizycznego bytu i istniejącymi tylko wtedy, gdy jest nimi przenoszony ruch.

- Powinna zapewniać: **poufność, integralność, autentyczność**.
- Trzy rodzaje:
 - **Zaufany VPN** – przesyłane dane są odpowiednio chronione, ale z technicznego punktu widzenia nie ma gwarancji, iż urządzenia w sieci operatora, przez które przesyłane są dane nie zostaną przejęte przez osoby nieupoważnione.
 - **Bezpieczny VPN** – bazuje na urządzeniach i oprogramowaniu klienta, transmisja odbywa się przez Internet a do jej ochrony stosuje się **techniki kryptograficzne**; zaszyfrowany ruch jest widoczny jako tunel między dwoma sieciami; wykorzystuje zwykłe łącze internetowe – niski koszt, ale też łącza publiczne – brak gwarancji parametrów sieci na całej drodze transmisji.
 - **Hybrydowy VPN** – integrujący dwa powyższe, stosowany do przesyłania poufnych informacji wrażliwych na opóźnienia w sieci; transmisja realizowana przez **łącza dzierżawione** (wysoki koszt), odpowiednio zabezpieczone za pomocą technik kryptograficznych.
- **Zalety VPN**
 - **niezależność od fizycznej realizacji połączeń**
 - **niskie koszty** ponoszone za dzierżawę łącza - opłaty ponoszone są głównie za łącze dostępne
 - **dostępność** - poza ewentualnymi problemami z szybkością transmisji
 - **szybkość zestawiania nowych** bezpiecznych połączeń - zestawienie nowych "połączeń" konfigurowane jest lokalnie, bez udziału dostawców usługi

Protokół IPsec (ang. Internet Protocol Security, IP Security)

- zbiór protokołów służących implementacji bezpiecznych połączeń oraz wymiany kluczy szyfrowania pomiędzy komputerami,
- mogą służyć do **tworzenia VPNów** – taki VPN składa się z dwóch kanałów komunikacyjnych pomiędzy połączonymi komputerami: **kanał wymiany kluczy** (UDP) oraz kanału (jednego lub więcej), który **niesie pakiety** transmitowane poprzez sieć prywatną.
- **Usługi zapewniane przez IPsec**
 - Poufność (szyfrowanie) transmisji
 - Zapewnienie integralności danych w pakiecie

- Uwierzytelnianie pochodzenia danych
- Zabezpieczenie przed atakami polegającymi na powtórnym przesłaniu pakietu
- Ograniczone zabezpieczenie przed analizą ruchu
- Kontrola dostępu
- IPsec jest jednym ze sposobów na zbudowanie VPN
- Przed przesyłaniem danych strony komunikujące się uzgadniają szczegóły takie jak sposób uwierzytelniania, wymiana kluczy, algorytmy szyfrowania. Uwierzytelnienie może być oparte o **certyfikaty** lub **klucz dzielony**.
- **Authentication Header (AH)** - nagłówek dodawany do pakietu IP, zawierający dane służące do uwierzytelnienia (tajny klucz) i kontroli integralności (sumy kontrolne) – sprawdzenie **autentyczności i integralności**.
- **Encapsulating Security Payload (ESP)** - szyfrowanie (symetryczny współdzielony klucz) oryginalnego pakietu IP i obudowanie w pakiet ESP – **szyfrowanie** danych, oraz autentyczność i integralność danych.
- **Tryby** działania IPsec (zarówno AH jak i ESP):
 - **Tryb transportowy** - w sieci lokalnej; podmiana zawartości pakietu IP na zaszyfrowaną (szyfrowany nagłówek i ładunek)
 - **Tryb tunelowy** - w niezabezpieczonej części sieci, zanurzenie pakietu ESP w nowy pakiet IP (szyfrowany tylko ładunek)
- **Polityki stosowania IPsec**
 - **Client** (respond only) - transmisje bez IPsec, chyba że druga strona zażąda IPsec
 - **Server** (request security) - żądanie transmisji IPsec, ale jeśli druga strona nie implementuje IPsec, to komunikacja bez IPsec
 - **Secure server** (require security) - żądanie transmisji IPsec, jeśli druga strona nie implementuje IPsec, to komunikacja nie jest kontynuowana.
- **Filtry IPsec** – filtr pozwala na automatyczne przepuszczenie datagramów IP, blokowanie lub użycie negocjacji (i w konsekwencji użycie IPsec) w zależności od źródła i miejsca docelowego IP, protokołu transportowego, portów źródłowych i docelowych.